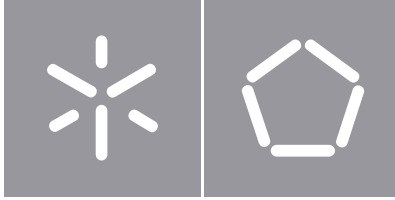


University of Minho
School of Engineering

Carlos Filipe Coelho Ferreira

Tackling Real-World Problems with Lince



University of Minho
School of Engineering

Carlos Filipe Coelho Ferreira

Tackling Real-World Problems with Lince

M. Sc. Dissertation
M. Sc. in Informatics Engineering
Dissertation supervised by
Renato Jorge Araújo Neves
José Miguel Paiva Proença

Copyright and Terms of Use for Third Party Work

This dissertation reports on academic work that can be used by third parties as long as the internationally accepted standards and good practices are respected concerning copyright and related rights.

This work can thereafter be used under the terms established in the license below.

Readers needing authorization conditions not provided for in the indicated licensing should contact the author through the RepositóriUM of the University of Minho.

License granted to users of this work:



CC BY

<https://creativecommons.org/licenses/by/4.0/>

Acknowledgements

To my supervisor at Uminho university, Renato Neves, I give my sincere thanks for his knowledge, availability and helpfulness on this final journey of my student career.

To all the people involved in the creation and development of Lince, it goes my full admiration for their technical knowledge, vision and commitment in contributing to the future of the field of hybrid systems.

To my colleges, who I have met on my academic life, from which so many discussions and support made my journey at Uminho much more enjoyable and enlightened.

But most important, to all my family, my parents, my brothers and my sister, and to all my friends, who supported me throughout all my life, I thank them for their presence, both in good and bad moments, I dedicate this dissertation to all of them.

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

University of Minho, Braga, october 2024

Carlos Filipe Coelho Ferreira

Abstract

Hybrid programs orchestrate classic computation with physical processes in order to reach prescribed goals. Examples of this are diverse and range from small medical devices, such as pacemakers and insulin pumps, to autonomous vehicles and district-wide electric/water grids. Their rapid development in the last decades lead to a flurry of research on suitable languages, semantics, and tools for their design and analysis.

However, even though great progress has been made on this area, several important challenges remain largely open. A prominent case is that many crucial hybrid systems have not been tackled nor fully analyzed via hybrid programming techniques and tools such as Lince. In fact, the area of hybrid programming is quite new and still generally lacks well-tested methodologies and techniques for a disciplined modeling/analysis of hybrid programs.

Therefore, in this dissertation, we introduce not only a comprehensive study of the different behaviors of hybrid systems, but also developed a rich new taxonomy of hybrid programs, using it to fully explore the recently introduced tool Lince, its strengths and limitations, drawing some conclusions about further improvements of the tool. In our exploration, we focused on different categories of hybrid programs, and examples that are of crucial importance to society, typically found e.g. in medicine, autonomous piloting, and the energy sector. This exploration clarified the potentialities and limitations of Lince, and provided interesting solutions to real-world problems in ‘hybrid systems engineering’. The fact that our exploration was systematic and based on the established taxonomy, we delineated some methodologies and techniques in hybrid programming including the simulation of hybrid automata in Lince.

Keywords Cyber-Physical Systems, Hybrid Systems, Hybrid Programming, Lince

Resumo

Os programas híbridos orquestram computação clássica com processos físicos para atingir objectivos prescritos. Os exemplos são diversos e vão desde pequenos dispositivos médicos, como pacemakers e bombas de insulina, a veículos autónomos e redes eléctricas/água de âmbito distrital. O seu rápido desenvolvimento nas últimas décadas levou a um foco de investigação de linguagens, semânticas e ferramentas adequadas para a sua conceção e análise.

No entanto, embora se tenham registado grandes progressos neste domínio, vários desafios importantes continuam em aberto. Um caso proeminente é o facto de muitos sistemas híbridos cruciais não terem sido abordados nem totalmente analisados através de técnicas e ferramentas de programação híbrida como o Lince. De facto, a área da programação híbrida é bastante recente e, de um modo geral, ainda carece de metodologias e técnicas bem testadas para uma modelação/análise disciplinada de programas híbridos.

Por isso, nesta dissertação, apresentamos não só um estudo abrangente dos diferentes comportamentos de sistemas híbridos, mas também desenvolvemos uma nova e rica taxonomia de programas híbridos, usando-a para explorar completamente a ferramenta Lince, seus pontos fortes e limitações, tirando algumas conclusões sobre possíveis melhorias futuras da ferramenta. Na nossa exploração, concentrámo-nos em diferentes categorias de programas híbridos e em exemplos que são de importância crucial para a sociedade, tipicamente encontrados, por exemplo, na medicina, na pilotagem autónoma e no sector da energia. Esta exploração clarificou as potencialidades e limitações do Lince, e forneceu soluções interessantes para problemas do mundo real em 'engenharia de sistemas híbridos'. O facto de a nossa exploração ter sido sistemática e baseada na taxonomia estabelecida, delineou algumas metodologias e técnicas de programação híbrida, incluindo a simulação de autómatos híbridos em Lince.

Traduzido com a versão gratuita do tradutor - DeepL.com

Palavras-chave Sistemas Ciberfísicos, Sistemas Híbridos, Programação Híbrida, Lince

Contents

I	Introductory material	1
1	Introduction	2
1.1	Motivation and context	2
1.1.1	Cyber-Physical Systems	4
1.2	Contributions	5
1.3	Document structure	6
2	State Of Art	7
2.1	Concepts	7
2.1.1	Hybrid Systems	7
2.1.2	Hybrid Programming	9
2.2	Tools for Modeling and Analysis of Hybrid Systems	10
2.2.1	Modeling approaches	10
2.2.2	Uppaal	13
2.2.3	Simulink	14
2.2.4	KeYmaera X	15
2.2.5	Modelica	16
2.2.6	Lince	17
2.3	Hybrid Formalisms	20
2.3.1	Hybrid Automaton	21
2.3.2	Hybrid Automata Simulation Difficulties.	23
2.3.3	Deterministic Hybrid Automaton Definition	29
2.3.4	Classic Automata Definition	30
2.4	Summary of Previous studies of Hybrid Systems Classifications	32

2.4.1	Causality of flow conditions and switches	32
2.4.2	Hybrid Phenomena	35
II	Core of the Dissertation	39
3	Hybrid Automata to Lince	40
3.1	Automata to Regular Expressions	41
3.1.1	Kleene Algorithm	41
3.1.2	Example	42
3.2	Deterministic Hybrid Automata Compiler	44
3.2.1	Syntax Compiler	44
3.2.2	Example	45
3.3	Hybrid Automata Compiler	47
3.3.1	Syntax Compiler	47
3.3.2	Example	48
3.4	Conclusion	50
4	Taxonomy	51
4.1	Continuous Dynamics Linearity	54
4.1.1	Linear Continuous Dynamic	54
4.1.2	Non-Linear Continuous Dynamic	54
4.2	Braching Types	55
4.2.1	Distributed Systems	55
4.2.2	Deterministic Systems	57
4.2.3	Non-Deterministic Systems	59
4.2.4	Stochastic and Probabilistic Systems	63
4.3	Triggered Actions	64
4.3.1	Time Triggered Actions	64
4.3.2	Event Triggered Actions	67
4.3.3	Controlled Triggered Actions	69
5	Discussions	71
5.1	Investigation Methodology	71

5.2	Automata	72
5.2.1	Water Tank	72
5.2.2	Thermostat	75
5.2.3	Gas Burner	78
5.3	2D Movement	81
5.3.1	Dubins Path	81
5.3.2	Wall Collision	85
5.4	Autonomous Driving	88
5.4.1	Automatic Emergency Breaking	88
5.4.2	Adaptive Cruise Control	92
5.4.3	Missile and Target	94
6	Conclusions and Future Work	99
6.1	Conclusions	99
6.2	Future Work	101
III	Appendices	105
A	Automata to Lince	106
A.1	Haskell Automata Compiling	106
A.2	Urgent Methodology algorithm	113
A.3	Examples	113
A.3.1	Deterministic Semaphore	113
A.3.2	Automaton Example	113
B	Lince Examples	114
B.1	Taxonomy	114
B.2	Discussions	117

List of Figures

1	Light Switch in Uppaal	14
2	Simple Speaker example in Simulink	15
3	Bouncing Ball Verification in KeYmaera X	16
4	Velocity Variation Example in Lince	18
5	Online Lince Example	19
6	Hybrid automaton for a car that can accelerate or brake	21
7	Exemple 1: Deterministic Automaton	26
8	Example 2: Deterministic Automaton	26
9	Example 3: Non-Deterministic Automaton in Switching	26
10	Example 4: Non-Deterministic Automaton in Time	27
11	Example 5: Non-Deterministic Automaton	27
12	Example 6: Non-Deterministic Automaton in Time	27
13	Example 7: Non-Deterministic Automaton in Switching	28
14	Example 8: Non-Deterministic Continuous Dynamics	28
15	Deterministic Automaton of 2.4	29
16	Example of DFA that search words web and ebay	30
17	Event Iteration - Ball Collision Sequence	36
18	Deterministic Semaphore	45
19	Semaphore output in Lince	46
20	Automaton - Compiling Example	48
21	Lince output of simulating "afd" transitions of 20, with precision 0.1	49
22	Taxonomy	51
23	Projectile motion	54
24	Dubins aircraft dynamics	55

25	Distributed Breaking System	57
26	2D Cruise Control - Lince	58
27	Choosing to break or accelerate	60
28	Non-Deterministic Thermostat	60
29	Brownian Motion in a 2D Space [18]	62
30	Breaking and Accelerating - Lince	65
31	Breaking If - Lince	66
32	Bouncing Ball [15]	67
33	Bouncing Ball Lince	68
34	Investigation Methodology	72
35	Water Tank hybrid automaton [15]	73
36	Water Tank Lince Output	75
37	Thermostat hybrid automaton [15]	75
38	Thermostat Lince Output	78
39	Leaking Gas Burner Hybrid Automaton	79
40	Gas Burner Modified Automaton	79
41	Gas Burner Lince Output	80
42	Gas Burner Second Lince Output	81
43	Discrete Self Driving Lince Output	83
44	Linear Dubins Path Lince Output	85
45	Wall Collision	86
46	Wall Collision Lince Output	87
47	Automatic Emergency Breaking	88
48	Automatic Emergency Breaking - Lince Output	91
49	Adaptive Cruise Control [20]	92
50	Adaptive Cruise Control - Lince Output	94
51	Missile and Target	96
52	Missile and Target - Lince Output	97

List of Tables

1	Classification of hybrid systems by G. Labinaz in Modeling and control of hybrid systems: a survey.[8, 17, 15]	12
2	Classification of hybrid systems in "A unified framework for hybrid control: Background, model, and theory" by M.S. Branicky, V.S. Borkar and S.K. Mitter.[22, 15, 17]	34
3	Classification of hybrid simulation phenomena in "An overview of hybrid simulation phenomena and their support by simulation packages. [23]" by Pieter J. Mosterman [15, 17]	38
4	Kleene Algorithm - 1st Step	42
5	Kleene Algorithm - 2nd Step	43
6	Kleene Algorithm - 3rd Step	43
7	Automaton - Functions	48
8	Lince Extended Syntax	53
9	Examples of hybrid systems and their categories	102

Glossary

Continuous System Dynamic system whose state flows according to a continuous function over time.

Cruise Control System that maintains a vehicle's previously programmed driving speed.

Cyber-physical System System with cybernetic and physical components to solve problems that neither party could solve alone.

Differential Equations Equations that contain the derivatives of one or more dependent (velocity) variables with respect to one or more independent (time) variables.

Discrete System Dynamic system whose state changes at given time instants.

Hybrid Automaton extension of classical automata with state variables that evolve continuously while in a state and change discretely at state transitions.

Hybrid Program Hybrid system specifications written in a programming-oriented style.

Hybrid System Mathematical models of dynamical systems that combine discrete and continuous dynamics.

Lince Hybrid program interpreter originating from the extension of differential equations to a classical while-language.

Part I

Introductory material

Chapter 1

Introduction

Key Points:

- Hybrid systems have great potential in all sectors of society.
- The development of these systems is recent as are the tools involved in such.
- The tool Lince has the ability to help in the development of these systems.
- The main objective of this dissertation is to contribute to the critical analysis of the tool through the development of a diverse taxonomy of hybrid programs.

1.1 Motivation and context

Programming and technology in general is becoming more and more intrinsic to the physical world in order to help solve problems once considered impossible. This trend, however, comes with its own set of difficult problems, at the root of this issue is the fact computer science is still a long way from being able to represent the complexity of the physical world in a confident and reliable way. The reality around us is governed by complex and connected numerous factors like time, space, randomness and many others that one must take in to consideration for achieving prescribed goals.

Not surprisingly, the realization of software systems that closely interact with the physical world presents immense challenges and complexities, especially considering that the failure of such systems can lead to catastrophic consequences, not only economically but also in the health of the individuals.

Example of such systems are found in autonomous driving, drone control, power plants and pace-makers among many others present in various areas and sectors of our community and with prospects of proliferating even more in the future years. Of these examples, we emphasize autonomous piloting, classic tasks in this area include following a leader, avoiding an obstacle, and safely landing a spacecraft.

The abstraction of these systems into mathematical models creates the opportunity for their design, analysis and verification using the right tools. However, there is still a general lack of tools for these models, or sometimes there's no adequate models to represent a certain type of systems, which arises from the fact that this is an upcoming field of study. Many of the few existing tools that have flaws and limitations that hinder a disciplined/systematic modeling and analysis. The tool Lince tries to combat this problem by offering an environment capable of interpreting hybrid programs, an emerging computation paradigm that aims to model hybrid systems by melding techniques of programming theory with physical processes, Lince programming language is based on a well-established semantics, and consequently promotes the development of these programs with confidence in their safety.[10]

To fully study the capacities of Lince, it is important to follow an approach not focused on case by case of hybrid systems, but an approach dictated by what classes of hybrid systems Lince is able to handle. In a perfect world, Lince would be able to fully and comprehensively simulate all existent categories of hybrid systems, unfortunately that is not the case, not for Lince, and arguably not for any Hybrid system tool existent in the market, mainly because of how each one of them were created with a clear distinct goal in mind and with very distinct approaches in functionality. Taking all of this into account, in this dissertation a personal taxonomy of hybrid systems was made, mainly to highlight the categories that can show the limits of Lince, what is impossible, what is difficult, and which opposite behaviors in contrast are easy to simulate, finally, which alternatives the user can implement when faced with those limits, and most importantly, what are the possible future goals Lince can improve to resolve these issues.

This taxonomy aims to make the modeling of hybrid system much easier, by quickly identifying the behaviors of a hybrid system, and compare them with a tool's capacities, the user will be not only able to recognize the limitations of the tool while modeling the system in question, but also observe what errors could arise before even modeling it, and plan accordingly.

1.1.1 Cyber-Physical Systems

As every part of our society keeps getting integrated with the cyber space in an ever increasingly large net of interconnected devices that constantly interact with the physical world, a new concept has risen that aims to explain this phenomena and the scale of connection between both worlds, named cyber-physical systems, or CPS for short. This concept is an excellent way to take a glimpse about the possible future of hybrid systems and why is it so important to keep innovating on the study of these.

A cyber-physical system (CPS) is an orchestration of computers and physical systems. Embedded computers monitor and control physical processes, usually with feedback loops, where physical processes affect computations and vice versa. [19]

Cyber-physical systems applications include automotive systems, manufacturing, medical devices, military systems, assisted living, traffic control and safety, process control, power generation and distribution, energy conservation, HVAC (heating, ventilation and air conditioning), aircraft, instrumentation, water management systems, trains, physical security (access control and monitoring), asset management and distributed robotics (telepresence, telemedicine). [19] Because these systems present a wide array of behaviors, specially distributed and non-deterministic, they contrast an intellectual challenge on the construction of models and methods from the area of computer-science, that's where hybrid systems come in, however as a new upcoming challenge, the existence methods of hybrid systems, don't present necessarily a capacity able to handle cyber-physical systems and therefore CPS constitutes a new engineering discipline that demands its own models and methods. CPS connects strongly to the currently popular terms Internet of Things (IoT). IoT reflect a vision of a technology that deeply connects our physical world with our information world. The term "CPS" is more foundational and durable, because it does not directly reference either implementation approaches, nor particular applications. It focuses instead on the fundamental intellectual problem of conjoining the engineering traditions of the cyber and the physical worlds.[19]

1.2 Contributions

The first objective of this dissertation was to test the functionalities and limitations of Lince, using it to model the possible different categories of hybrid systems in our world. The improvements made in that regard are:

- **Small study of existent hybrid system tools:** This include a small presentation of each different software tool available to design and validate hybrid systems, their main functionalities and modeling approaches. The tools discussed include Uppaal, Simulink, KeYmaera, Modelica and Lince
- **Analysis of hybrid system behaviors by previous studies:** The identification of different behaviors that hybrid systems can present regarding classifications previously done on established research papers, introducing some of the complexities of hybrid systems and some examples.
- **Study and modeling of hybrid automata in Lince:** The introduction of a study about hybrid automata and the creation of a possible modeling approach of these when using Lince, presenting the limitations found on said approach.
- **Creation of a personal taxonomy:** The introduction of a taxonomy of hybrid systems, proposal of a theoretic new syntax language in Lince in order to improve it's modeling capabilities to encapsulate new types of hybrid systems, followed by respective examples.
- **Modeling of real life examples in Lince:** Usage of the developed taxonomy to present case studies of complex real life hybrid systems in Lince.

1.3 Document structure

This document will be divided in 6 parts:

- **Chapter 1: Introduction**

The current chapter introduces the theme of the dissertation in a simple and motivating way, presenting as well the contributions it intended to offer to the area of hybrid systems.

- **Chapter 2: State of the Art**

The second and following chapter will effectively be the state of the art, with the objective to present to the reader theoretical information and support about the concepts related to the project.

- **Chapter 3: Automata into Lince**

The third chapter will contain a study about converting and simulating automata into Lince.

- **Chapter 4: Taxonomy**

The forth chapter will contain a proposed taxonomy of hybrid systems developed during this thesis, and how identifying these categories helps in the process of modeling and programming these systems not only in Lince but other tools as well.

- **Chapter 5: Discussions**

The fifth chapter, discussions, will have some practical examples of programming in Lince, testing it's capabilities, and taking conclusions and observations.

- **Chapter 5: Conclusions and Future Work**

In the final chapter, possible future work and conclusions are drawn from this dissertation and discussed.

Chapter 2

State Of Art

In this chapter we intend to present several concepts and definitions that will be useful for the reader to understand during the presentation of the developed work in the next chapters, we introduce the definition of Hybrid systems, how they integrate both discrete and continuous dynamics and how important they are to the future development of technology and we discuss the definition of hybrid programming, computational techniques that aim to simulate or validate hybrid systems. In another section we introduce how different modeling approaches, and different simulation/validation techniques of hybrid systems led to the creation of several tools throughout the years, giving small examples of existent tools in the market and their main characteristics, we talk about Uppaal, Simulink, Keymaera, Modelica and our tool Lince. We present several definitions of Hybrid Automata as formalisms of hybrid systems, giving special distinction to their possible non-deterministic characteristics, these definitions will be later used in further chapters of our dissertation. Finally, we discuss some known classifications and phenomena of hybrid systems from distinguished researched papers.

2.1 Concepts

2.1.1 Hybrid Systems

Programs and physical processes are combined more and more often in programming practice, creating a necessity for software tools that closely interact with physical processes, like velocity, movement, energy, and time. Examples of such systems are numerous, ranging from small medical devices, like pacemakers and infusion pumps, to surgical robots, autonomous vehicles driving, and district-wide electric grids[25], this development however is extremely challenging considering the complex behaviors of the physical world. Hybrid systems, come as a way to try and define the intrinsic behavior of physical processes being mathematical models of dynamical systems that combine discrete and continuous dynamics. Specifically,

their behavior includes both aspects that change discretely one step at a time and aspects that change continuously over time. In the previous example of autonomous vehicle driving, the vehicles' position and velocity continuously evolve according to a system of differential equations. However, the pilot, or in this case the computer, has the ability to change this trajectory or change the speed in discrete decisions to prevent collisions and/or follow a destiny. In this case, the car schedules a decision in a specific instant of time being a discrete dynamics. Then the car follows the result of that decision continuously for a period of time until it finds itself in a new situation that leads to re-evaluate a new decision to be made. Other examples of hybrid systems can usually be found in the areas of control theory, event-based mechanics, software incorporation, and biology, including for example disease propagation and personalized cancer treatments. [15, 28]

During the conversion from physical processes to hybrid systems, despite initial deductions, processes often implemented in a continuous dynamics do not necessarily imply representation of a physical process, and processes implemented in a discrete dynamics are not necessarily a programming component. For example, some computational processes are computed so quickly and frequently that it is more advantageous to interpret them with a continuous dynamics even if it is not completely true. In fact, this is one of the advantages of hybrid systems, their mathematical principles are so basic and fundamental that they allow us to interpret processes either continuously or discretely, offering a trade-off depending on the goal we want to achieve. For some purposes, it may be better to model the landing of a plane as a discrete change of state from air to ground. For other purposes, such as developing an autopilot program for landing that same plane, it is important to include a much more refined view, because the aircraft will simply take off again if it has a high speed.

In conclusion, hybrid systems are mathematical models of complex physical processes interpreting both discrete and continuous dynamics, their uses are many, and they allow us to simulate, validate and study programs that interact with the physical world.

It is important to note as well that certain biological processes such as genetic networks can be captured as hybrid models. Hybrid system can be extended to represent other different behaviors presented in the real world, for example exhibit random or uncertain behavior derived from choices influenced by variability in the environment, exhibiting a so-called stochastic dynamics if good distribution of available choices is known. Other dynamics such as non-deterministic, adversarial or distributed may also be necessary to be represent.

Since hybrid systems are built around dangerous environments where lives can be in danger if they malfunction, safety is an important if not the most important factor in their development. A bad-designed

hybrid system can lead to the opposite of contributing to efficient and safe decision making leading to many catastrophes. Safety is then key to ensure the quality of one of these systems. This entails the need to involve simulation and validations processes to enhance the development of these.

To develop a completely fail-safe system would require a complete and correct understanding of the physical world and a complete catalog of all actions that can affect that world as well as a correct understanding of the consequences of those actions. While such a purpose is not currently achievable, the point is that the safety of a hybrid system and the validation of its objectives depends directly on how it is computationally programmed and how well it has been evaluated for errors. Assurances originating from analytical foundations in the form of computational languages are then necessary, and it is appropriate that these tools are adaptable and standardized to various or all situations to establish a firm foundation in the area of hybrid systems. Practical testing although still important, proves to be insufficient as they cannot efficiently cover all possible scenarios. Without a mathematical and logical proof system it's impossible to offer a sufficiently qualified safety proof.

2.1.2 Hybrid Programming

Hybrid programming is an emerging computational paradigm that aims to use principles and techniques from programming theory to develop computational components that interact closely with physical processes.[7, 11, 28, 25] The distinction between a hybrid system and a hybrid program is emphasized: hybrid systems are characterized by having both discrete and continuous components; hybrid programs on the other hand are specifications of hybrid systems written in a programming-oriented style. An easy way to recognize the distinctions between the two is to keep in mind that a single hybrid system system can have several different programs developed to represent it.

Consider the following example of a hybrid program:

```
1      while true do {if v <= 10 then (v' = 1 for 1) else (v' = -1 for 1)}
```

This example represents a digital motion controller (Cruise Control) that periodically measures and regulates the speed of a vehicle (v): if it is less than or equal to 10, the controller accelerates for 1 unit of time, as dictated by the program instruction $v' = 1$ for 1 ($v' = 1$ is a differential equation representing the rate of change of velocity per time). Otherwise, it decelerates for the same period of time ($v' = -1$ for 1).[10]

Notice that, unlike normal languages, this cruise controller does not only contain normal constructs like *while-loops* and *conditionals* but also differential constructs. The use of differential equations is a

characteristic feature of hybrid programming that allows the representation in a programming-oriented style of the various physical processes mentioned earlier, namely time, energy, velocity and motion. The usual programming constructs on the other hand can easily represent the discrete behaviors of hybrid systems.

There are also other formalizations of hybrid systems namely hybrid automata, an extension of the classical automata with state variables that evolve continuously while in a state and change discretely at state transitions. Hybrid automata have a rich theory and powerful tools, both typically focused on reachability analysis unlike hybrid programs.[2, 25]

2.2 Tools for Modeling and Analysis of Hybrid Systems

The immersion of hybrid systems led, throughout the years, to the creation of various powerful software tools capable of helping in their development, analyses and verification. Because of the complexity of dynamic systems there isn't a right or better tool, but rather the appropriate tool to a given problem and objective, it's then important to highlight why exactly the creation of Lince was beneficial to the field of study by giving a short introduction to some of the various tools available on the market, how they work, their characteristics and uses. These short explanations help understand the current state of the field and how despite all the progress made, there is still a necessity to keep innovating in order to achieve all the potential of hybrid systems.

2.2.1 Modeling approaches

Before we introduce some of the tools existent in the market, it is important to know why there is such a varied array of software created to develop and analyze hybrid systems and why, even so, each tool finds a way to be unique in it's approach and capabilities. Because hybrid systems are mathematical models of both continuous and discrete dynamics there's numerous different ways from which these can be modeled and simulated, the model choices of each tool have to take into account several characteristics of the hybrid systems they wish to encapsulate. One of the first studies trying to categorize modeling approaches was done in Modeling and Control of Hybrid Systems: A survey [8], from which it is purposed six specific characteristics, sampling, continuous Dynamics, determinism of continuous dynamics, determinism of discrete dynamics, control actions and specifications, all presented very briefly on table 1 [15, 17] Therefore, this classification of hybrid systems is most suited when used to discuss different hybrid system tools and the type of hybrid system behaviors they are able to represent. [17, 15, 8]: Here's some

of the concepts most important to take into account in this study:

- **Sampling:** Sampling refers to the mechanism of acquiring information from the physical process.

The following options are used for sampling in hybrid system modeling:

- **Continuous** - measurements (of states) are assumed to be known (and available) at all times, this allows for a complete validation of the entire hybrid system, this include tools that use proof theory algorithms like some we'll mention afterwards.
 - **Regular** - measurements (of states) are taken based on some predetermined, fixed sampling period.
- **Continuous Dynamics:** The Continuous Dynamics of an hybrid system takes into reference the type of equations that represent the continuous it's behavior. A continuous dynamic can be linear when the differential equation presents no variables with higher power then 1. A clear distinction is made because simulate non-linear continuous dynamics can be a problem for some languages and tools, causing a clear visible restriction.
 - **Determinism:** In the field of mathematics a deterministic function is defined by always returning the same results if given the same input values. A nondeterministic function may return different results every time it is called, even when the same input values are provided. In a similar way, a hybrid system is said to be deterministic if it presents only one behavior through his simulation time as long as it's given the same initial state and inputs. This means that at any given time during the system execution, there will only exist one possible result for all the variables of the system aka it's state. For a hybrid system to be deterministic both of his dynamics need to be deterministic:
 - **Continuous Dynamic:** For each state of the system, the continuous flow off all it's variables are always singled value and can never flow in more then one way (multi-value). This behavior is usually shown when the differential equation that represents the continuous behavior of a system is non-deterministic, an example will be given much later in the dissertation.
 - **Discrete Dynamic:** Discrete occurrences map each state to a single consecutive state, meaning that, at every possible execution, a discrete action will always happen at the same moment in time and once made, the result of that action is always only one single next state and never multiple possible distinct states.

The clear definition of determinism is wide, depending much on the formalism used to represent the hybrid system, the choice to include the validation or simulation of non-deterministic hybrid

systems in a tool leads to a completely new set of problems, something that will also be discussed later.

Category	Sub-category	Explanation
Sampling	Regular	Measurements are taken based on some predetermined, fixed sampling period
	Continuous	Measurements of states are assumed to be known (and available all the time)
Continuous Dynamics	Linear	Dynamics are defined by linear equations
	Nonlinear	Dynamics are defined by nonlinear equations
Determinism (Cont.dyn.)	Deterministic	The evolution of systems is single valued
	Nondeterministic	The evolution of system is multi-valued
Determinism (Discr.dyn.)	Deterministic	Each state is mapped to single next state
	Nondeterministic	State is mapped to a set of states
Control action	Continuous	Continuous-valued control functions with continuous domain and range
	Discrete	Function with discrete range and continuous or discrete domain
	Combined	Combination of discrete and continuous control
Specifications	Continuous	Specifications based on continuous time behaviour and/or variables
	Discrete	Specifications based on discrete event behaviour and/or variables
	Combined	Combination of continuous and discrete

Table 1: Classification of hybrid systems by G. Labinaz in Modeling and control of hybrid systems: a survey.[8, 17, 15]

2.2.2 Uppaal

Just like all the tools in the field Uppaal has the objective to help develop dynamic systems. The difference present is in how the tool allows the representation of these systems, and giving the available abstraction what dynamic components of a hybrid system can be integrated. The structure in cause is a web of timed automaton extending to the usage of different language data types (strings, integers), clocks and operations between those. Each timed automata represents a process and consists of locations or nodes and transitions or edges, each node and edge can be paired with a set of invariants being then capable of representing certain conditions that make a state or transition possible and therefore portray the behavior of the system.

Timed automaton allow us to represent various different dynamics previously mentioned. In a location, the system have a continuous dynamic represented by the flow of time in the form of clocks, in an edge it's possible to update variables and clocks, altering the state of the system and representing a discrete dynamic. The usage of channels and shared variables allow us to synchronize transitions of different automata being one of the few tools to easily be able to study distributed systems. In addition, a timed automata contains a non-deterministic behavior meaning that, the simulation of a system can lead to different results or paths depending on the different possible transitions taken in that simulation. Because of the usage of timed automaton, Uppaal is built using at it's core a pseudo-formal semantics, allowing to logically determinate the valid actions of a model in a certain state, being delay transitions made to advance clocks or action transitions.

Although a bit slow, by taking in to account the simulation of all possible depth limited paths, Uppaal makes available a powerful verification, query language capable of expressing different type of properties:

- **Reachability.** A specific condition holds in some state of the model potential behaviors.
- **Safety.** A specific condition holds in all the states of an execution path.
- **Liveness.** A specific condition is guaranteed to hold eventually.
- **Deadlock.** A deadlock is possible or not in the model.

This verification has the characteristic of being realized through a continuous sampling mechanism, meaning it holds true to all states of a system.

One very simple example, just for demonstration purposes, of a system in Uppaal is a Light Switch where the user is able to press a button two consecutive times in less than a specific time to turn the

bright light and only one to turn the low light. A distributed system with two timed automata, the lamp and the user.

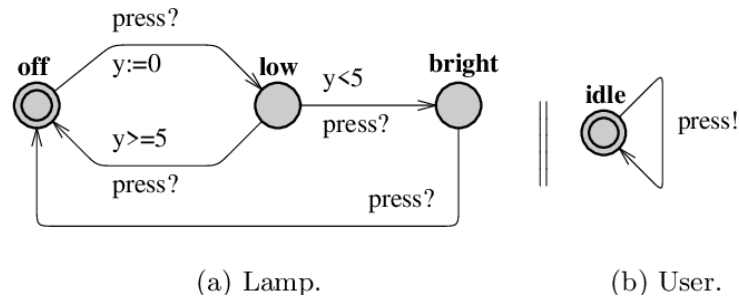


Figure 1: Light Switch in Uppaal

In conclusion, Uppaal is a very powerful tool, especially to specify time based distributed systems, for example, entities competing for resources, but its graphic interface is not very flexible and can become very complex very easily. The continuous dynamic available is restricted to clocks, not being able to represent for example space (trajectory and velocity) or energy based hybrid systems, much less those with non-linear mechanics.

2.2.3 Simulink

Simulink is an extra component of Matlab that allows rapid and interactive graphical construction of virtual block orientated models that simulate and analyze dynamic systems. The objective is to be able to produce a running model that would otherwise require hours to build in the laboratory environment. These models work by the directional and continuous or sampled flow of numeric/signal values, originating from one or more source blocks and terminating in final blocks, being re-directed, altered or filtered in the blocks in-between.

These models are built recurring only to predefined blocks categorized in:

- **Sources:** Used to generate various signals. Ex: Constant input, Sine Wave, Step, etc.
- **Sinks:** Used to output or display signals. Ex: Scope, Display, To Workspace, XY Graph, etc.
- **Discrete:** Linear, discrete-time system elements. Ex: Discrete Filter, Discrete State-Space, Discrete Transfer Fcn, Discrete Zero-Pole, Unit Delay, etc.
- **Continuous:** Continuous-time system elements. Ex: Integrator, State-Space, Transfer Fcn, etc.
- **Signal Routing:** Contains useful blocks to build a system. Ex: Mux, Demux, Switch, etc.

- **Math Operations:** Contains common math operations. Ex: Abs, Gain, Product, Sign, Sum, etc.

The next simple example, as we can see, serves to represent a speaker. A sinusoidal wave (sound) is produced in a source, multiplied by the math operation block Gain, and both the original and the altered sound are displayed in a sink Scope.

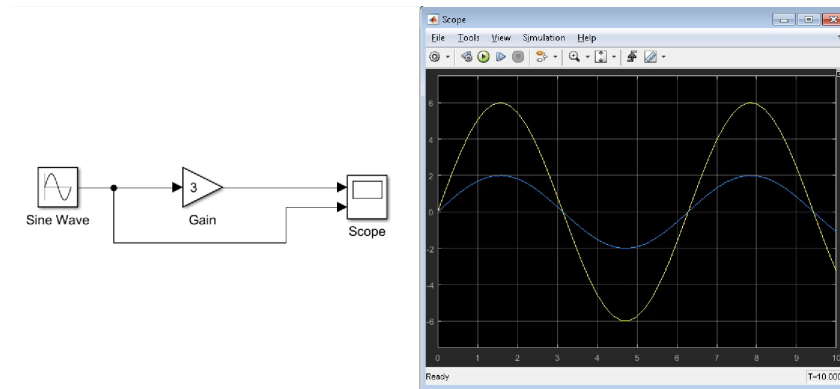


Figure 2: Simple Speaker example in Simulink

Simulink is a useful and easy tool, but it represents certain disadvantages derived from the fact of involving graphical, interactive and predefined component design, although the reuse of models is possible, it's not very scalable or flexible representing several difficulties for example in abstracting hybrid systems involving three dimensional movement, something essential in the field. Although very restrictive Simulink present some clear advantages, for example it can generate executable code for languages like C and C++.

2.2.4 KeYmaera X

KeYmaera makes available a very lightweight environment capable of analyze **Hybrid Program**, being one of the few examples where the analyses of these programs is not made by simulation but by proof deduction, recurring to the use of a well defined and established differential dynamic logic (dL), a logic for specifying and verifying properties of hybrid systems with mixed discrete and continuous dynamics. The major advantage of KeYmaera is the ability to use aggressive proof search to verify with confidence certain safety conditions. A safety condition that once again, holds in the continuous sampling period of the system, at all states.

One example of a hybrid program in KeYmaera and its corresponding verification is the bouncing ball, a system that tracks the movements of a ball jumping into the ground. After defining the assumptions and behavior of the program, it's then possible, using this tool to verify for example the invariant related to the conservation of mechanical energy when the damping coefficient equals to 1.

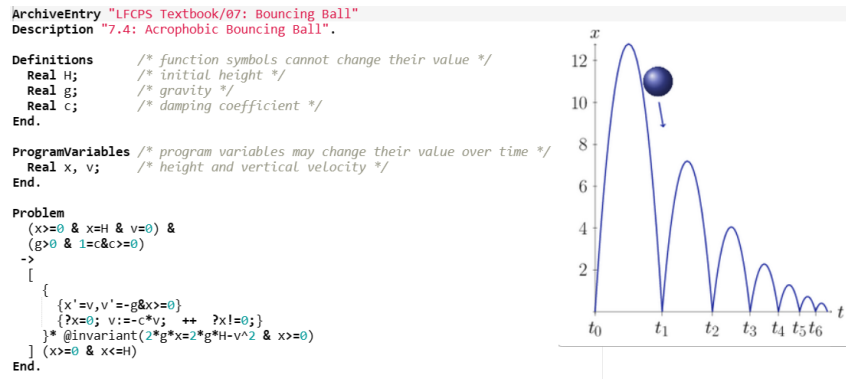


Figure 3: Bouncing Ball Verification in KeYmaera X

Because there is no simulation involved, the tool is useful to verify predefined hybrid programs but doesn't help during the development of those, not being able to study the course of the system and making changes accordingly.

2.2.5 Modelica

Modelica is a software language used for modeling physical systems, the abstraction of these systems is made by connecting different components that represent physical objects like turbines or pumps, being these reused or created using mathematical equations to represent a set of rules that define its principals.

What makes this tool so distinguished from the others in the field is that the language is acausal meaning the components are described by relationships between system variables, there is no clear sequence or procedure of actions, and the tool will sort and compile this relationships into execution code to suit the system. This presents a grand advantage in the development of more practical systems like electric, magnetic, hydraulic or thermal systems being used a lot in automotive companies to design energy efficient vehicles. Modelica is a testament to the scope of hybrid systems and their possible mathematical model abstractions.

2.2.6 Lince

Lince is a newly created tool with the objective of combating the difficulties found today in the hybrid programming environment. This tool, functions as a hybrid program interpreter and simulator originating from the extension of the use of differential equations to a classical *while-language*, thus promoting a composed vision that remains in touch with classical programming theories. This approach differs from various other hybrid system development and analysis programs on the market precisely because of this characteristic of being orientated to a programming style, as opposed to the usual construction of graphical diagrams, therefore being more adaptable, flexible and more easily integrated with other languages or tools.

One of the desired goals is that in the future Lince can be used as a complement to other debugging tools, especially those that work in the hybrid domain. However, the main feature and advantage of the tool is that it has an extremely well defined, established and formalized semantic being therefore built on logically deduced and theoretical foundations taking into account the composition of its different components (*while-language* and *differential*) and the desired functionalities. In fact, one of the main concerns during its development was to make sure that the interpretation of hybrid programs at the implementation level coincides precisely with the intended semantics calculated that integrates both the *while-language* and its continuous behavior in a rigorous and safe way, however, the demonstration and explanation of this semantics are not within the scope of this dissertation.

Other tools mentioned before like *SIMULINK* and *MODELICA* that are standard in the hybrid systems analysis and development industry lack these formalized semantics, even though, there have been efforts to provide these for subsets of both tools, so that simulation errors caused by numerical approximation methods are avoided. It is also important to note that, other advantage in comparison with other tools is that in the future, by extending its semantics it's possible to take into account the addition of other physical behaviors such as a non-deterministic behavior mentioned before. In spite of all that, its recent creation carries various disadvantages to the use of the tool, the lack of testing and well defined techniques paired with its core basic function and the absence of well defined purposes can prevent its application in real life problems, something that this projects intends to address.

In summary, Lince acts as a practical framework to formally specify, analyze and simulate hybrid programs and present relevant properties regarding its behavior, such as the cruise controller previously shown:

```
1      while true do {if v <= 10 then (v' = 1 for 1) else (v' = -1 for 1)}
```

In this example in particular, Lince has the capacity to safely answer questions such as: What is the maximum speed the vehicle will reach? Does it ever reach an unsafe speed? And if so, for how long?

As can be demonstrated by the following image 4 that represents the result of running the cruise control in the tool, in this context Lince is, in a very specific way, a tool that given a hybrid program and an instant of time t , returns the value to which the program evaluates for time t .

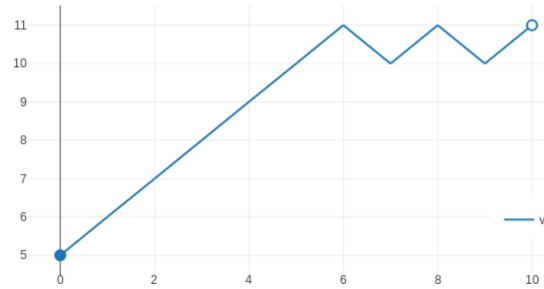


Figure 4: Velocity Variation Example in Lince

However, notice that despite being quite obvious, because of the simplicity of the system, Lince makes its observations in a sampled period of time, not able to make full validations like Keyamera and Uppaal, so although there's a clear loop in the system, for more complex systems Lince may not be able to provide valid safety validations.

Because of that, Lince offers a lighter verification method that can be complementary to a heavier verification tool like KeYmaera, considered then to be a basic building block not only for developing programming tools in the hybrid scenario, but also for simulating hybrid system executions, believed to be, at best of our knowledge, the first tool of its kind to be created.

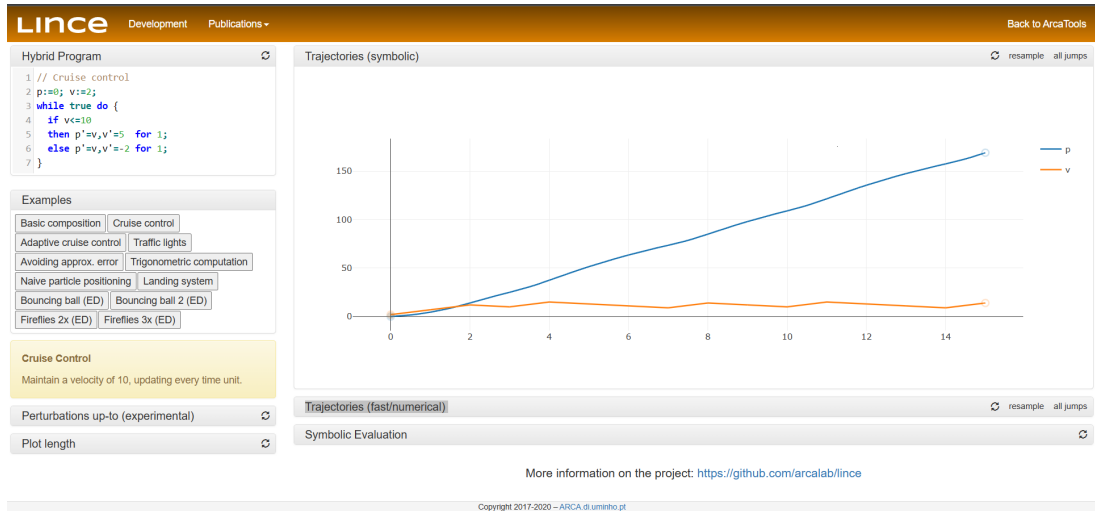


Figure 5: Online Lince Example

At this moment Lince finds itself available for use to everyone [3] and for installation [9], giving the user an easy interface (figure 5) and several examples of developed hybrid systems to interpret.

Take into account some of the model approaches discussed in the previous section 1, Lince programming syntax is not able to simulate any non-deterministic behavior, it's sampling period is as discussed before, Regular, however, as mentioned before, because Lince syntax can be extended to include new model characteristics, there's already been studies to include non-linear dynamics [33], something that previously wasn't possible. As another clear limitation is important to know that as a new upcoming tool Lince still has a rigid syntax with few supported arithmetic operators and mathematical functions.

2.3 Hybrid Formalisms

As it could be seen in the introduction of different tools for hybrid systems, there's numerous ways from which a hybrid system can be represented and modeled, the structure from which a hybrid system is defined is described as a formalism, a set of rules, syntax and restrictions that are able to model both continuous and discrete dynamics of a hybrid system. The flexibility of these formalism are extensive, so there won't be a deep discussion on the different ways these exist, but it is important to mention the main areas, considering this chapter presents many hybrid systems represented in the form of hybrid automata for example.

It is not surprising that different formalisms were devised to handle hybrid phenomena, mainly considering the different competing communities and their respective focus, in a very brief way, these modeling approaches can be categorized in three very different groups: [17]

Dynamical systems Hybrid *dynamical systems* are classical dynamical systems extended to handle hybrid phenomena. They consist on a system of structured equations that dictate the continuous behavior of an hybrid system, yet some drawbacks to this approach:

- Discrete behavior is often oversimplified or even neglected. It is somehow transacted to the continuous world, mostly in ad-hoc fashion, thus expensive and error-prone
- Formalisms usually are not modular, and it complicates work with big and complex systems.

Automata, process algebras and transition systems These models are grouped considering their focus on representing the discrete behavior of hybrid systems, contrary to Dynamical Systems, therefore, they focus on transactions between states and the rules that dictate them, their strengths are:

- Elaborated techniques to specify and analyze discrete behavior.
- The approaches are modular and support parallel composition.

But there are some serious drawbacks:

- Often tools and even algorithms for these formalisms are missing.
- Continuous behaviour is oversimplified or even neglected

Simulation languages As the name indicates, these formalisms are mostly used for modeling and simulation, and less for theoretical research. They have tool support, and often are used in practice, Lince

for example uses a simulation language to model it's systems, employing a mixture of famous operations in habitual coding languages like **repeat**, **If** and **While** and mixes them with differential functions for the continuous behavior in operators like **for** (continuous flow the system for a given amount of time), **until** (continuous flow the system until a condition is met) and **wait** (advance the clock for a given amount of time where all the differential equations are all 0, presenting no change during that time).

2.3.1 Hybrid Automaton

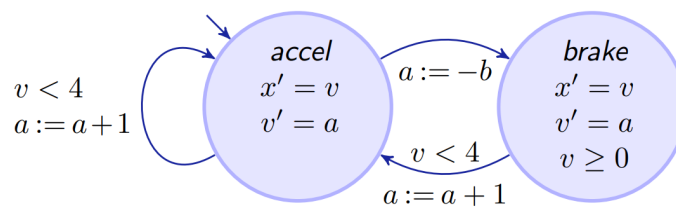


Figure 6: Hybrid automaton for a car that can accelerate or brake

The hybrid automaton model is a formalism of hybrid system and it's one of the most popular approaches to model and analyze them, considering their graphical design and very easy to understand representation as it can be seen in the previous figure. Essentially, a hybrid automaton encodes the two characteristics of hybrid systems. [17]

1. **Discrete** changes are described by transitions, which are decorated with action names, guards and resets. The guards define conditions allowing to take the transition. The resets define changes of continuous state made by the transition and the action names are used as references in synchronization when used mainly while representing distributed systems as it could be seen in the previous example of the light switch when showing the capacities of Uppaal.
2. **Continuous** behavior is described in locations. It is given by flow conditions, which define continuous change of the continuous state and usually are given by differential equations, and invariant, which restrict evolution in the location and usually are given by some kind of inequalities.

Take a closer look to the automaton in figure 6 [28], In this example of a hybrid automaton, there's two locations, one from which the car is accelerating, both following a similar continuous behavior $x' = v$; $v' = a$, the position of the car x flows continuously accordingly to it's velocity v , and the velocity v

flows accordingly with the the car acceleration a , the brake location however has an invariant where it's not possible to reach velocities bellow 0 $v \geq 0$. There's three discrete changes possible in the system aka transitions, when the car is accelerating, if the guard $v < 4$ is satisfied then the system can make a transition to increase it's acceleration by one. At any moment in time in this location, because the guard has no invariant, the system can also change it's acceleration for $-b$, a value that represents the force of the breaks. Once in the break location, if the car has velocities bellow 4 $v < 4$ then the car can increase it's acceleration again. Notice that a guard being satisfied doesn't necessarily mean the transition will be made.

Here's a formal definition of hybrid systems that we will be using in this dissertation, to clarify all it's properties, we were inspired by the definitions provided in [17, 31] Hybrid automaton is a collection

$$H = (X, L, \text{Init}, \text{Inv}, f, E, \text{Guard}, \text{Assign}, \Sigma) \quad (2.1)$$

- $X \subseteq \mathbb{R}^n$ is a finite set of real-valued variables $\{x_1, x_2, \dots, x_n\}$, where $x_i \in \mathbb{R}$, This set of variables, and their values, represent the world from which our system will evolve and change, both continuously and discreetly.
- L is a finite set of locations.
- $\text{Init} : C(X)$ is an expression built with language C over the variables X , that indicates the initial attributions of all variables in X .
- $\text{Inv} : L \rightarrow P(X)$ assigns to each location l an invariant to be satisfied by the system while in the location l , where $P(X)$ denotes the language of predicates made around the set of variables X , in this case all Boolean expressions.
- $f : L \rightarrow C(X \cup X')$ assigns to each location l an expression where $X' = x'_1, \dots, x'_n$ represents the first derivatives of the variable in X . In this case $C(X \cup X')$ represents a language built around the set of variables X and X' . This map is used to dictate which continuous evolutions may occur in a given mode.
- $E \subseteq L \times \Sigma \times L$ is the set of transitions, also called switches, where Σ is a set of transition labels.
- $\text{Guard} : E \rightarrow P(X)$ assigns to each transition a guard that has to be satisfied by the system if the transition is taken.

- **Assign** : $E \rightarrow C(X)$ assigns to each transition a predicate that dictates how each variable $x \in X$ will be altered to when the transition is taken.

Take for example some grammar rules of expressions φ built with language $P(X)$.

$$\varphi \ni \text{true} \mid \varphi \ \&\& \ \varphi \mid t < t \mid t == t \quad t \ni t + t \mid t - t \mid x \mid r \quad (x \in X, r \in \mathbb{R}) \quad (2.2)$$

And some examples of grammar rules expressions φ built with language $C(X)$

$$\varphi \ni \varphi; \varphi \mid \varphi; \mid \varphi, \varphi \mid t := t \mid t = t \quad t \ni t + t \mid t - t \mid x \mid r \quad (x \in X, r \in \mathbb{R}) \quad (2.3)$$

These languages will make possible the creation of predicates like $y' = 3, x' = y$ for the f function of the automaton 2.1, $x := 5; x := y + 10$; for the Assign and Init functions, and $(x == 3 \ \&\& \ y < 3)$ for the guard and invariant functions. Clearly these aren't the entirety of all rules, which we won't give too much attention into developing, but remind that both languages, $P(X)$ 2.2 and $C(X)$ 2.3 are supposed to accept all Boolean predicates and all assign and flow expression that are also accepted by Lince, considering we'll be working mainly with that language. More of Lince syntax can be read on [10, 33].

2.3.2 Hybrid Automata Simulation Difficulties.

Hybrid automata, as a formalist of hybrid systems, differ from the normal automata from which they are derived, in this chapter we intend to study many of the hybrid systems behaviors that hybrid automata are able to represent, and compare which one of these characteristics need to be taken into account during our intended objective of simulating automata in Lince, specially considering the limitations of our tool. Take a consideration into the next identified behaviors of hybrid automata that need to be taken into attention when trying to correctly simulate these in Lince or any other hybrid system simulation tool.

- **Detection of Guards and Invariants:** Lince, as many other simulation languages of the type, can't, in a confident way, detect and locate certain exact conditions that require extremely precision, as a possible solution, the system resorts to a time triggered discretization of it's continuous behavior, always associated with a certain precision, an interval of time from which the condition is repeatedly checked, for example, checking if $x == 2$ each 0.5 seconds in a system that starts with $x = 0$ and flows with $x' = 2$. Considering that in automata there's a need to detect transition

guards and location invariants, when simulating automata in Lince, these conditions need to be easy to detect at a reasonable precision. This restriction is perhaps one of the most dangerous we will encounter in our study, one, from which there's little to no solution besides being attentive to it's flaws. In fact, the detection of these conditions is a problem that surges since the inception of Lince [10]. We'll later discuss this behavior of trying to detect conditions with practical examples.

- **Non Linear Differential Equations:**

Definition 1. *Non Linear automata are those who continuous behavior of any of it's locations is described by differential equations that are non-linear*

An example of a non-linear differential equation is $x' = y^2, y' = 1$, the reason why this is an important behavior to take into account is because some tools have limitations about simulating non-linear continuous dynamic, this is a very easy restriction to detect, but important because, once again, Lince still has some difficulties in simulating non-linear hybrid systems. We'll also talk about Non-Linear Hybrid Systems later in this dissertation.

- **Non-Determinism:** A Deterministic Automaton is one that for each initial state and variable values, only presents one possible execution, by this definition it's easy to understand that the only way to truly know if an hybrid automaton is deterministic is by simulating it. Despite the possibility of defining a set of rules restricting the modeling of automata in order to make these always deterministic, which we will be doing later 2.4, this set of rules won't cover for all possible deterministic hybrid automata, instead, let's make a study and take a look at all the different ways an automaton can present non-deterministic behaviors, and be aware that the following section will provide some practical examples of these definitions:

- **Non-Deterministic in Continuous Dynamic:**

Definition 2. *If an automaton has at least one location from which the continuous behavior is represented by a non-deterministic differential function, then that automaton is by default non-deterministic in it's continuous dynamic.*

However, it is very rare to find an automaton with that characteristic, a deterministic continuous dynamic in each location is actually a fairly common assumption.

- **Non-Deterministic in Switching:**

Definition 3. *If, during an automaton execution, there's two or more transitions possible out of one location, meaning at least two transition guards are satisfied during the continuous behavior of that location, then the system is said to have non-deterministic discrete switching*

Deterministic Switching is a concept briefly mentioned in the study of hybrid systems [8]. Because it's not pre-determined which guards will be true during the execution of the automaton, it's hard to define if a hybrid system is deterministic in it's range, unless we simulate it. An hybrid system with maximum one transition out of each location is necessarily deterministic in range, however the opposite is not true. Furthermore, an automaton can also be non-deterministic in range if a made transition has a non-deterministic assign function connected with it. An hybrid automata with deterministic switching means that given the same input it will always execute the same sequence of transitions throughout it's execution.

– Non-Deterministic in Time :

Definition 4. *If a transition is not executed at the exact same time from which the continuous behavior of the location stops being possible then a automaton is said to be time non-deterministic*

Time deterministic is a famous studied property of hybrid transition systems [8, 31, 24]. This behavior derives from the possibility of a transition to be delayed, causing a choice between executing the transition or staying in the location, and therefore resulting on different possible executions of the system. A true deterministic system always has it's transitions guards become available at the same time from which the invariant of the location becomes false and therefore the system can't stay in that location anymore, this restricts the behavior of the system to execute the transition at only one possible instant. An easy way to model a time deterministic is to define all guards by a given equality like $t = 1$, contrary to have for example $1 > t > 0$ where the system could make it's transition at any given time as long as t is between these two values, and pair that guard with an invariant of value $t \leq 1$ or $t \geq 1$ to make sure that once the guard is verified no more continuous behavior is possible.

Examples

Because the determinism of an automata can be a rather difficult concept to understand unless some examples are given, consider the next simple examples of automata who have the characteristic previously explained.

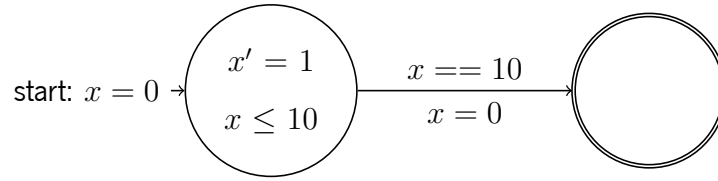


Figure 7: Example 1: Deterministic Automaton

The automaton in the figure 7 is Deterministic both in time 4 and switching 3. There's only one transition possible out of the location, and that transition can only be made at the exact same instant from which the system can no longer stay at the location, the instant where $x == 10$. The simulation of this system will always have an output from which x will continuous evolve with flow 1 until it reaches 10 at exact 10 seconds and then realize the transition discreetly changing the value of x to 0.

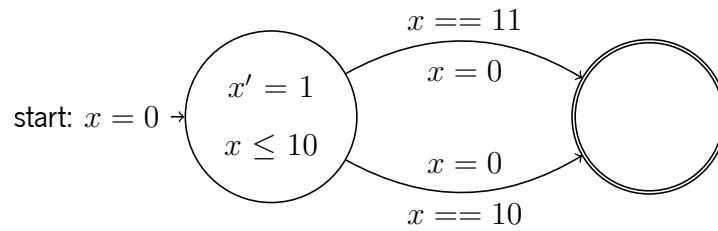


Figure 8: Example 2: Deterministic Automaton

This automaton 8 is also deterministic both in switching 3 and time 4, although there's 2 transitions out of the location, it is easy to notice that only one of them is possible, because the guard of the second transition $x == 11$ will never come true and therefore the transition will never be made. Therefore the execution of this system is deterministic and completely equal to the previous one.

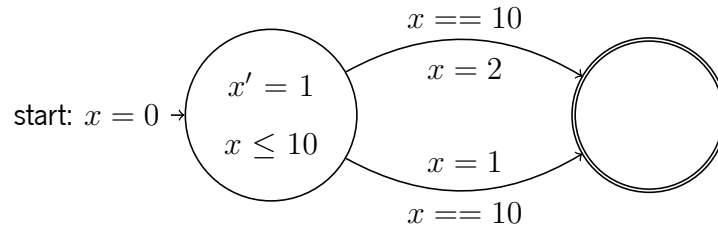


Figure 9: Example 3: Non-Deterministic Automaton in Switching

In figure 9, both transitions can be executed at the same instant from when the continuous behavior is not possible in the location, however there's two transitions possible out of the location, both with different

assign functions so this system is deterministic in time 4 but not in switching 3 causing a non-deterministic behavior overall.

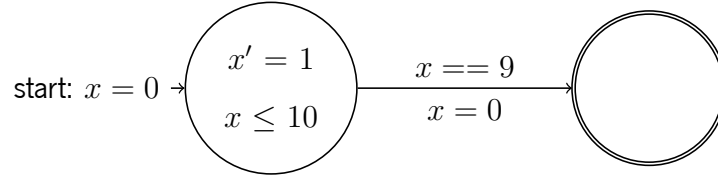


Figure 10: Example 4: Non-Deterministic Automaton in Time

During the execution of this automaton 10, because there's a clear choice between doing the transition when $x == 9$ or stay in the location until $x \leq 10$ then this automaton is non-deterministic in time 4.

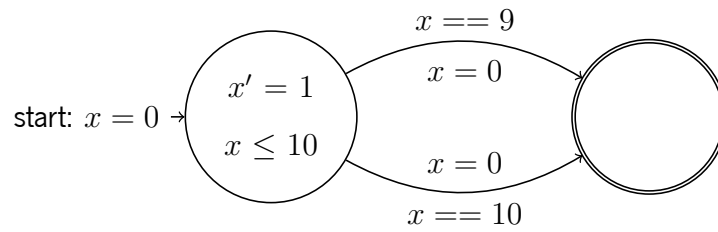


Figure 11: Example 5: Non-Deterministic Automaton

Considering there's two transitions possible out of the location and one of them is not made at the same instant from which the continuous behavior stops being possible then this automaton 11 is both non-deterministic in time and switching..

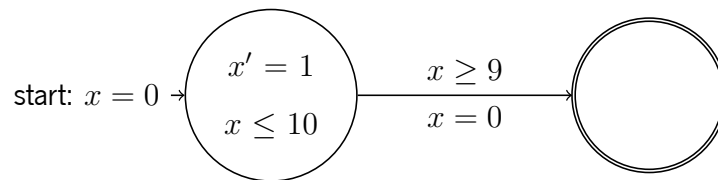


Figure 12: Example 6: Non-Deterministic Automaton in Time

This automaton 12 is non-deterministic in time 4, but it differs from the previous examples because there's a continuous interval between $9 \leq x \leq 10$ from which the transition can be executed. This non-deterministic behavior, that are possible to be modeled automata, are one of the most difficult to

simulate considering it's infinite possibilities from which the transition can be made, resulting in a system with theoretically infinite possible executions.

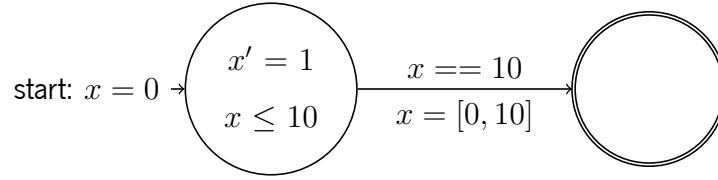


Figure 13: Example 7: Non-Deterministic Automaton in Switching

Finally, this automaton 13 is non-deterministic in switching considering the assign function of the executed transition is non-deterministic, attributing to the variable x any value between 1 and 10, this type of automata usually doesn't appear when modeling hybrid systems, but it is within the real of possibilities, and it causes the automaton to present infinite executions.

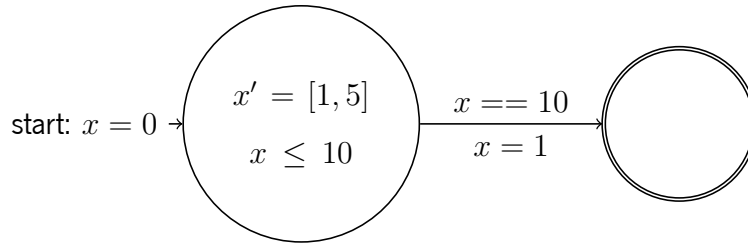


Figure 14: Example 8: Non-Deterministic Continuous Dynamics

The automaton in the figure 14 is an example of an extremely unusual automaton that is non-deterministic in it's continuous dynamics 2. This behavior of having the x variable flow in a random way between two values while in the location, is not something very studied in the field, and very difficult to simulate, therefore, there won't be a detailed explanation and study of this phenomena but it will be discussed later in the dissertation.

2.3.3 Deterministic Hybrid Automaton Definition

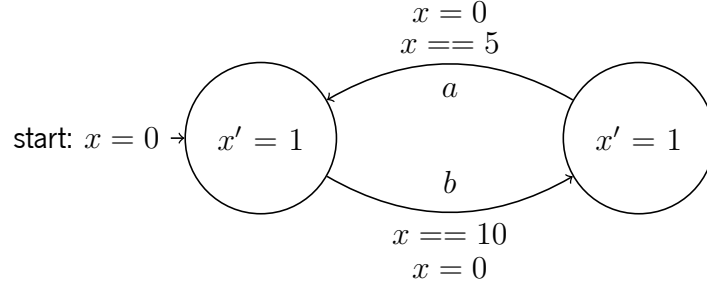


Figure 15: Deterministic Automaton of 2.4

Considering the problematic behaviors explained in the previous section, we now understand why Lince is not able to fully simulate all automata, so in this dissertation we will be working not only with the previous definition of the Hybrid automaton 2.1 but also with a new derived definition applying some restrictions, mainly to make sure that the automaton in question is deterministic, these restrictions are quite common in the study of hybrid automata in an coalgebraic perspective [31].

$$H_d = (X, L, \text{Init}, f, E, \text{Guard}, \text{Assign}, \Sigma) \quad (2.4)$$

This new definition is close to the previous definition, however the following assumptions are made:

1. There is only one transition out of each location, and the assignment function of that transition is deterministic, this will restrict examples like 13 and 9, and will make sure the automata has deterministic switching 3.
2. For every location of the automaton, the function f returns a system of differential equations with exactly one solution for each variable, and all those equations are linear. This is actually a common assumptions as most examples of hybrid automata rarely involve non-linear 1 or non-deterministic continuous 2 differential equations.
3. There won't be considered any invariant for any of the locations, as it can be seen in the previous definition 2.4, there's a lack of the function Inv .
4. Finally, and most important of all, as soon as an edge outgoing a location become enabled (i.e. the associated guard is satisfied) the system must execute the transition, never before or after. This will satisfy the property of *time deterministic* 4.

In the figure 15, there's an extremely simple automaton where it can be observed that it satisfies all restrictions of this new definition 2.4. There's only one transition out of each location, all assign functions are deterministic, the differential functions are linear and deterministic, there isn't any invariant. Assuming each transition is done as soon as it's guard is satisfied, and the variable x behaves like a clock because it flows always with a differential equation of 1, we can deduce that this system is a simple loop that stays in each location for 5 and 10 seconds respectively. Note also that each transition has a label belonging to the set Σ .

This set of rules makes the automaton easy to simulate in Lince, and despite being very restrictive, there's multiple examples of simple and studied real life examples of hybrid automata that follow these restrictions, some of which it will be mentioned later.

2.3.4 Classic Automata Definition

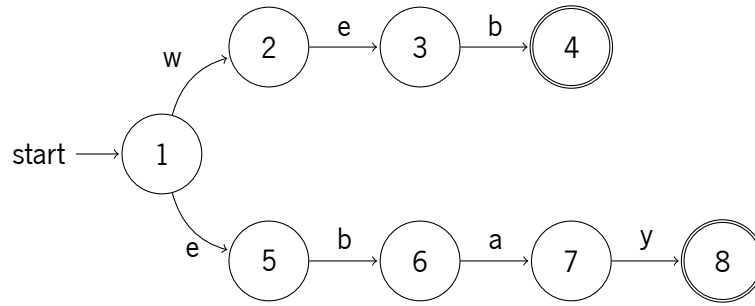


Figure 16: Example of DFA that search words web and ebay

In this dissertation, it's also important to take into account the following definition of a classic automata, the version from which hybrid automata derive from: [13]

$$A = (Q, \Sigma, \delta, q_0, F) \quad (2.5)$$

where:

1. Q is a finite set of *states*.
2. Σ is a finite set of *input symbols*.
3. q_0 , a member of Q , is the *start state*.
4. F , a subset of Q , is the set of *final (or accepting) states*.

5. δ , the *transition function* is a function that takes a state in Q and an input symbol in Σ as arguments and returns a subset of Q .

This definition can be extended depending if the automata is intended to be non-deterministic (NFA) or deterministic (DFA), in a deterministic finite automaton the δ formula will map a state to a single next state, while a non-deterministic finite automaton will map a state to a set of different states. This extension comes from the fact that a NFA can have different transitions with the same input symbol leaving the same location, and also can possess transitions with no input symbol at all. Although a distinction is made between a DFA and NFA, it has been proved that any NFA can be represented with a DFA although the process is quite inefficient. [13]

The purposes of classic automata can vary, but consider one of its many uses, text search. When offered an input equal to a sequence of symbols, these being labels to transitions of the automaton, the automaton will be able to calculate a sequence of states resulting from doing that offered sequence of input symbols, if the last transition ends in a final state then the input string is accepted by the language that the automaton represents, the set of strings accepted by the automata is denominated as a language, therefore, automata can be used to search specific keywords in large amounts of data similar to the known and famous structures of regular expressions.

Take the previous example of a deterministic finite automaton 16, there's 8 states, the initial state is one, $q_0 = 1$, there's two final states, state four and eight, $F = \{4, 8\}$, and the set of input symbols accepted are $\Sigma = \{w, e, b, a, y\}$. For the transition function the full table won't be shown, but for example, the δ of five and input symbol 'b' is state six because there's an arc with label b that leaves from five to six. Overall, this DFA allows for the acceptance of the two words: ebay and web.

2.4 Summary of Previous studies of Hybrid Systems Classifications

Even if hybrid systems are mathematical abstractions they need to consider the complex behavior of the physical world, making them hard to classify, interpret and simulate, at least compared to more usual algorithms in programming. That is why throughout the years a huge importance has been given to finding ways to identify and classify these types of characteristics, the objective being, to find an universal vocabulary that can be used to efficiently evaluate hybrid systems, their modeling approaches, and also judge the tools that are made to validate and/or simulate them. There are several approaches from which Hybrid systems can be classified, one possible approach is for example, to which the systems in question may be of interest:

1. A certain Industry (chemical, automotive etc.)
2. An Organization which can provide financing for research (government, military, industry/industries etc.)

Although this approach leaves a lot to be desired considering how informal it is. For research purposes, it is much more favorable to consider an approach that views to classify the internal properties of hybrid systems, taking into account aspects like the hybrid system **complexity** or the properties of their **continuous** or **discrete** dynamics. [15]

With this approach in mind, although the scientific community hasn't come to an absolute understanding of an official categorization to hybrid systems, several research papers have come forward with possible solutions, so that, in this thesis, three distinct classifications studies will be addressed. This chapter will allow the formation of the necessary vocabulary useful to the consequent testing of Lince, being able to more easily identify which behaviors of hybrid systems Lince is able to simulate, and which it can't.

2.4.1 Causality of flow conditions and switches

One of the first proposed hybrid systems classifications was given in Branicky et al. [1994] by M. S. Branicky, V. S. Borkar and S. K. Mitter [22, 17, 15], this classification grouped hybrid systems into 4 categories as shown in table 2. To make such division, the author took into account the discrete changes happening in variables that otherwise flux continuously in time, having two different characteristics into account, namely:

- The causality of the discrete event:

The causality of the events refers to when a discrete change is acted upon in our system, according to this classification two types of signals can cause alterations into a hybrid-system:

- **Internal Signals** (Autonomous) happen when the system variables reach a specific region border predefined by a set of inequalities, e.g a ball hitting the ground, in this case the region in question being the height of the ball equal to zero.
- **External Signals** (Controlled), are, as the name points to, signals that come out of the system, via by controllers, buttons, switches, or human actions, for example a human hitting the break in a moving car. This type of signals only makes sense on the modeling of **distributed systems**.

- What type of changes are done to the continuous behavior:

Once a discrete change is decided to happen, according to this classification there's 2 different alterations that a continuous variable can suffer:

- **A Switch:** A variable is said to suffer a switch when the continuous flow of that variable changes during a discrete change, for example, when a ball hits the ground it's position stays the same but it's flow aka it's velocity changes direction and intensity.
- **A Jump or Reset:** A variable is said to suffer a jump, when it's value is directly altered during a discrete change. In a position/time system, the position of a particle suffering a jump would be akin to teleportation because it's position would change instantly, without time passing.

Remark: Most hybrid systems are likely to exhibit different combinations of the classes proposed, the bouncing ball example refereed has both switching in it's position, and jump in it's velocity once it hits the ground, some hybrid systems even have pretty complex behaviors, presenting discrete changes happening out of a combination of both Internal and External signals.[\[17\]](#)

The final classification takes into account the combination of these 4 types of behaviors, Autonomous Switching, Autonomous Jump, Control Switching and Control Jump.

Category	Explanation
Autonomous switching	Continuous flow changes on hitting specified region border
Autonomous jump (reset)	Continuous component changes on hitting specified region border
Controlled switching	Continuous flow changes in response to a control command
Controlled jump (reset)	Continuous component changes in response to a control command

Table 2: Classification of hybrid systems in "A unified framework for hybrid control: Background, model, and theory" by M.S. Branicky, V.S. Borkar and S.K. Mitter.[22, 15, 17]

This classification is very basic but somewhat useful in understanding the behavior of discrete dynamics in hybrid systems, mainly to understand that discrete actions can cause sudden changes on the continuous behavior of our system (how it flows), or direct sudden changes (jumps) in the value of its variables, furthermore, the time from which these discrete actions happen need to be encoded with signals/conditions, mostly internal. Out of the signal classifications, the external signals, as discussed before, make more sense in a environment of **distributed systems**, for example, in the Uppaal section, the shown example of the light switch 1, it was modeled the receiving of an external signal with the variable *press?*, representing receiving a signal from the channel *press*.

2.4.2 Hybrid Phenomena

The second proposed Hybrid system classification that we will be diving in was first given by Pieter J. Mosterman [23, 15, 17], and stands out because of its different approach when coming to studying hybrid systems. This established taxonomy aims to identify certain phenomena that can be visualized during the execution of various hybrid systems, therefore being something that happens instead of a core characteristic of the system, however, it is still useful to identify which tools and model approaches can handle the representation of these phenomena and how they do it. The full classification and its summary can be seen in table 3. Here's some of the hybrid simulation phenomena that could be important to remind in this thesis:

- **Events:** In hybrid system, an event is a certain occurrence condition that triggers a discrete change. As the reader may have noticed, Events is a concept that is close in meaning with Internal Signals from previous classification 2, however Pieter J. Mosterman makes the following distinction of Events:
 - **Time Events:** when the event is triggered at a predetermined time.
 - **State Events:** when the event is triggered by the system variables crossing threshold values and the time of their occurrence is not known a priori, in this case, state events need to be detected (if the event occurred) and located (at which time the event occurred), something that is not very easy for hybrid systems to do.
- **Simulation Model:** A phenomenon in which the system of equations that dictate the continuous flow of the system may change. Again, a similar concept with Switching from the previous classification 2. This change on the continuous behavior of the system can be modeled as:
 - Blocks of presorted and solved equations that appear or disappear.
 - Full Replacement of the system of equations.
 - The new system of equations is determined by giving an algebraic constraint between state variables.
- **Reinitialization:** A phenomenon in which the system variables suffer from a discontinuous and discrete change, similar to a Jump 2. This alteration in the variables may be:
 - Explicitly specified by the user given a pre-determined function.

- The new values need to be extracted from the system of equations in a way that is physically consistent to the behavior of the system.
- **Event Iteration:** A phenomena in which two or more events occur in the same instant of time, being that the second event occurs because of the changes caused by the previous event. A normal example given to explain an event iteration is the following [23]:

A ball is moving through a surface at a positive velocity, x_1 is its position, and v_1 its velocity, the ball follows in a direction against two distant balls, both not moving, once its position coincides with the second ball (a collision) following the condition $(x_1 \geq x_2 \wedge v_1 > v_2)$ a state event will trigger, making the second ball gain positive velocity, this discrete change will cause the other condition to be satisfied $(x_2 \geq x_3 \wedge v_2 > v_3)$, triggering an immediate second event, without the system evolving continuously between the events.

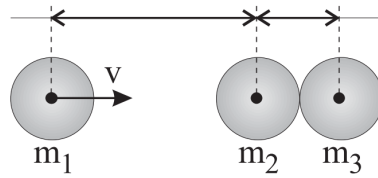


Figure 17: Event Iteration - Ball Collision Sequence

In modern literature, the concept of event iteration has been refereed more as **super dense time**.

Definition 5. *Super dense time is an important tool for modeling simultaneity in discrete event simulations. [26].*

The first use of super dense time was by Maler, Manna, and Pnueli [27] to distinguish simultaneity from sequences of instantaneous actions in a study of hybrid automata. This phenomena causes an importance in detecting and simulating a sequence of instantaneous discrete actions, that overall, in our hybrid system, appear to happen in simultaneous but computationally, each discrete action is done as consequence of previous discrete changes that happened in the same instant in time..

- **Chattering:** Chattering is a phenomena in which consecutive events happen at a fast pace in a small interval of time, moving back and forward between several modes of the system. Imagine the example of a bouncing ball, that re-bounces at a fraction of its velocity each time it hits the ground, at a certain time in our system, the ball will re-bounce with extreme low velocity, making so

the next event (hitting the floor) happens faster and faster until it's so close, we consider the ball to be not moving anymore. This particular phenomena presents a difficulty in simulating hybrid systems, locating so many state events at a small interval of time takes computer power, and causes continuous integration to become excessively slow, this chattering performance problem in the bouncing ball example can also be visualized in Lince.

The phenomena of chattering is actually a very studied and old one, this phenomena can also be known as **Zeno behavior**.

Definition 6. *A system that undergoes an unbounded number of discrete transitions in a finite and bounded length of time. [21]*

Zeno behavior can be thought of as a type of instability of hybrid systems that constitutes a major impediment to “proper” system behavior, giving plenty study about it's detection and safely control algorithms to stop them or bypass them. [21, 1, 32]. For example, in the bouncing ball system, which is a strong zenoness system, a safety control system would be to identify the point of zeno behavior and stop the ball at height zero.

Overall, when using this hybrid system phenomena classification 3 to evaluate the performance of hybrid system tools, like Lince, normally it's tested which of these phenomena the simulation can portray, and how it's capable of doing so. In Lince, for example, events are a must, with time events being very easy to do, but having a lack of algorithms to detect and locate state events, something it will be discussed much later. On a smaller note, in Lince, the simulation model works normally by replacing the entire system of equations, and the reinitialization only by giving an explicit function by the user.

Category	Type	Explanation
Events	Time	Events are generated at predetermined times
	State	Events occur because the system crosses some thresholds, the time of their occurrence are not known a priori - they need to be detected
simulation model	Dynamical blocks	Blocks of sorted and solved equation may simply appear or disappear (vehicle entering or leaving highway), and, therefore, can be dynamically added/removed
	Changeable cont. dynamics	In some cases equations can be replaced by others, changing computational causality, and the system of equations may have to be sorted again
	Constraint changes	In other cases, algebraic constraints between state variables may become active and the system of equations needs to be solved again
Reinitialization	Explicit	Change explicitly specified by user by a new initial state
	Integrated	The system of equations may have to be integrated to derive physically consistent initial values for a new mode. This ensures conservation of the thermodynamic extensivity holds
Event iteration	Invariant	State vector is invariant across the entire iteration
	Updated	State vector is updated after each iteration step
Chattering		Fast switching between several modes
Dirac pulses		Non-continuous changes for continuous variables; if their magnitudes are numerically approximated, comparison may be affected by non-Dirac type variables

Table 3: Classification of hybrid simulation phenomena in "An overview of hybrid simulation phenomena and their support by simulation packages. [23]" by Pieter J. Mosterman [15, 17]

Part II

Core of the Dissertation

Chapter 3

Hybrid Automata to Lince

A study about the transformation of hybrid automata into executable simulation code in Lince gives a clear path of analysis about the limitations of the tool, possible future improvements and an overall study of the characteristics of hybrid systems. At a first glance, the main difficulty of simulating automata into Lince, as described previously [2.3.2](#), besides the detection of event triggered actions, is their possible non-deterministic behavior, a characteristic Lince is not able to simulate, if the user pre-established a system to be deterministic, for example, using the previous definition of automaton [2.4](#), then its transformation into Lince is pretty simple and possible, however, this process can be tricky, and to push the full capabilities of Lince we intend to create an automatic middle algorithm that tries to simulate both definitions previously mentioned [2.1,2.4](#), even in non-deterministic behaviors, just to fully push what Lince is able to do and could do in the future. The first step of this study is to create a process for the transformation of an hybrid automata into a more understandable syntax language, and use that language to represent an automaton execution, and, if possible, encapsulate all possible executions of a non deterministic automaton. In fact, the oldest studies about interpreting classic automata [\[13\]](#), involve the transformation of these into regular expressions, arguably a much more easier syntax language to interpret for the intended objective. Take the previous example of the deterministic finite automaton [16](#), this automaton has an equivalent regular expression of $R = web + ebay$, and this regular expression represents all possible sequence of transitions that go from the initial state to ending states. Modernly, even in hybrid automata [2.1](#), the research community regularly tries to interpret these using their own versions of hybrid regular expressions that are extended to represent continuous and discrete behaviors. [\[28\]](#)

3.1 Automata to Regular Expressions

3.1.1 Kleene Algorithm

In this thesis, we will work mainly with the most classic definition of a regular expression, taking inspiration from the study done by John Hopcroft [13], who casually proves that every language represented by a classic automaton 2.5 can be represented by a classic regular expression, and vice-versa, to turn automaton into regular expressions a certain algorithm was purposed, by the name of Kleene Algorithm. Calculating a regular expression from an automaton can be tricky and the way it's done it's by building expressions that describe sets of strings that label certain paths although through only a limited subset of states. Every regular expressions is calculated inductive by expressions of smaller set of states, starting by the one from which paths are not allowed to pass through any state.

Consider the following expression R_{ij}^k that represents the regular expression whose language is the set of strings with path from state i to state j without passing through any state higher than k , being passing the act of entering the state and leaving it, when k equals to the number of states of an automaton then there's no restriction at all.

To construct the base case R_{ij}^0 there's two condition to take into account:

- $i \neq j$
 - a) If there's no transition between i and j then $R_{ij}^0 = \emptyset$
 - b) If there is exactly one symbol a then $R_{ij}^0 = a$
 - c) If there are symbols $a_1, a_2, \dots, a_k$, that label arcs from i to j then $R_{ij}^0 = a_1 + a_2 + \dots + a_k$
- $i = j$

Paths of length 0 represented by the regular expression ϵ adding to the various expressions devised in (a) through (c) above.

- a) If there's no transition loop then $R_{ij}^0 = \epsilon$
- b) If there's one symbol then $R_{ij}^0 = \epsilon + a$
- c) If there are symbols $a_1, a_2, \dots, a_k$, that label arcs from i to i then $R_{ij}^0 = \epsilon + a_1 + a_2 + \dots + a_k$

To construct R_{ij}^k for any value of k where $k \geq 0$ then the algorithm makes an induction case. Suppose there is path from from state i to state j that goes through no state higher then k . There are two possible cases to consider:

1. The path does not go through the state k at all. In this case the label of the path is the language

$$R_{ij}^{k-1}$$

2. The path goes through state k at least once, from which we can break the path into several pieces, a path that goes from state i to state k without passing through k , a path that goes from state k to state j without passing through k , and all the other several middle pieces go from state k to itself.

The result of all this pieces is represented by the expression $R_{ik}^{k-1}(R_{kk}^{k-1})^*R_{kj}^{k-1}$

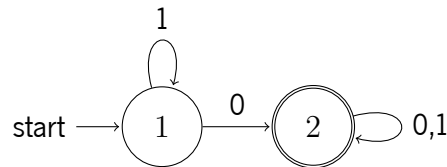
Remark: The kleene star of a language L is denoted L^* and represents the set of strings that can be formed by taken any number of strings, even zero, from L and concatenating all of them.

When we combine the expressions for the paths of the two types above, we have the expression:

$$R_{ij}^k = R_{ij}^{k-1} + R_{ik}^{k-1}(R_{kk}^{k-1})^*R_{kj}^{k-1}$$

3.1.2 Example

Let's convert the following deterministic state automaton into a regular expression. This DFA accepts all strings made with 0's and 1's that have at least one 0 in them.



The base case for the inductive process of the algorithm is to calculate the regular expression for every order of 2 states belonging to the automata when $k == 0$ using the rules above.

$R_{11}^{(0)}$	$\epsilon + 1$
$R_{12}^{(0)}$	0
$R_{21}^{(0)}$	$\emptyset$
$R_{22}^{(0)}$	$(\epsilon + 0 + 1)$

Table 4: Kleene Algorithm - 1st Step

For example $R_{11}^{(0)}$ has the term ϵ because the beginning and ending states are the same, and the term 1 because there's an arc connecting to itself on input 1. As another instance, $R_{21}^{(0)} = \emptyset$ because there's no arc connecting the states. The next step is to apply the inductive formula to compute the regular expression when $k = 1$ following the expression:

$$R_{ij}^1 = R_{ij}^0 + R_{i1}^0 (R_{11}^0)^* R_{1j}^0$$

	By substitution	Simplified
$R_{11}^{(1)}$	$\epsilon + 1 + (\epsilon + 1)(\epsilon + 1)^*(\epsilon + 1)$	1^*
$R_{12}^{(1)}$	$0 + (\epsilon + 1)(\epsilon + 1)^*0$	1^*0
$R_{21}^{(1)}$	$\emptyset + \emptyset(\epsilon + 1)^*(\epsilon + 1)$	$\emptyset$
$R_{22}^{(1)}$	$\epsilon + 0 + 1 + \emptyset(\epsilon + 1)^*0$	$\epsilon + 0 + 1$

Table 5: Kleene Algorithm - 2nd Step

The table above has regular expressions simplified using some simple deductions to obtain less complex expressions. For example, note that in $R_{12}^{(1)}$ if R is any regular expression then $(\epsilon + R)^* = R^*$ and in a similar way $(\epsilon + R)R^* = R^*$. Two other rules used in the simplification of the table above are:

- $\emptyset R = R\emptyset = \emptyset$.
- $\emptyset + R = R + \emptyset = R$

Because the automaton possesses only two states, then there's only one more iteration to perform for the regular expression to not be bounded to any restriction, therefore, as the last step we need to calculate R_{ij}^2

	By substitution	Simplified
$R_{11}^{(2)}$	$1^* + 1^*0(\epsilon + 0 + 1)^*\emptyset$	1^*
$R_{12}^{(2)}$	$1^*0 + 1^*0(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$1^*0(0 + 1)^*$
$R_{21}^{(2)}$	$\emptyset + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*\emptyset$	$\emptyset$
$R_{22}^{(2)}$	$\epsilon + 0 + 1 + (\epsilon + 0 + 1)(\epsilon + 0 + 1)^*(\epsilon + 0 + 1)$	$(0 + 1)^*$

Table 6: Kleene Algorithm - 3rd Step

The final regular expression that represents the language accepted by the automaton is constructed by the union of all the expressions where the first state is the initial state and the last state is an accepting state. In this case, considering the initial state is 1 and the only accepting state is 2, the regular expression is $R_{12}^{(2)} = 1^*0(0 + 1)^*$. The language of the automaton consists of all strings that begin with zero or more 1's then have a 0, and then any string of 0's and 1's, put another way, the language is all strings of 0's and 1's with at least one 0.

3.2 Deterministic Hybrid Automata Compiler

3.2.1 Syntax Compiler

Knowing that a regular expression can represent a sequence of transitions done by a certain automaton and even identify loops with the kleene star, we now introduce here a function named *compile* that aims to easily transform any regular expression coupled with a deterministic hybrid automaton discussed previously 2.4, and result in the equivalent Lince code that will be able to execute and correctly simulate that said automaton. First, remember the previous functions *Init*, *f*, *Assign* of an automaton 2.1, consider R to be a regular expression, $R_1.R_2$ to be the concatenation of two regular expressions, R^* to be the kleene star applied to the regular expression R , R_a to be the regular expression of a single symbol correspondent with a transition with label a and L_a the location from which that transition leaves from, so that $(L_a, a, L) \in E$. Additionally, consider p , a timed precision on which Lince will try to detect the automaton guards

$$(\text{start}^{\rightarrow}) \quad \text{compile } R \rightarrow \text{Init } \text{compile}' R \quad (3.1)$$

$$(\text{star}^{\rightarrow}) \quad \text{compile}' R^* \rightarrow \text{while (true) do } \{ \text{compile}' R \} \quad (3.2)$$

$$(\text{conc}^{\rightarrow}) \quad \text{compile}' R_1.R_2 \rightarrow \text{compile}' R_1 \text{ compile}' R_2 \quad (3.3)$$

$$(\text{symb}^{\rightarrow}) \quad \text{compile}' R_a \rightarrow f(L_a) \text{ until_p Guard}(a); \text{Assign}(a) \quad (3.4)$$

The rules are quite simple, 3.1 defines the initial state of our system applying the assignment of the initial variables of our automaton, 3.2 transforms the kleene star into an infinite loop to execute the order of transactions indefinitely, 3.3 simply states that in a concatenation there's a clear order of actions to be executed, and finally and most important 3.4 simulates the execution of a transition that is represented by a certain label a , in this case the system will continuous evolve following the rules of the location from

which the transition leaves from, until it finally detects the transition guard and apply the discrete changes associated with the transition.

3.2.2 Example

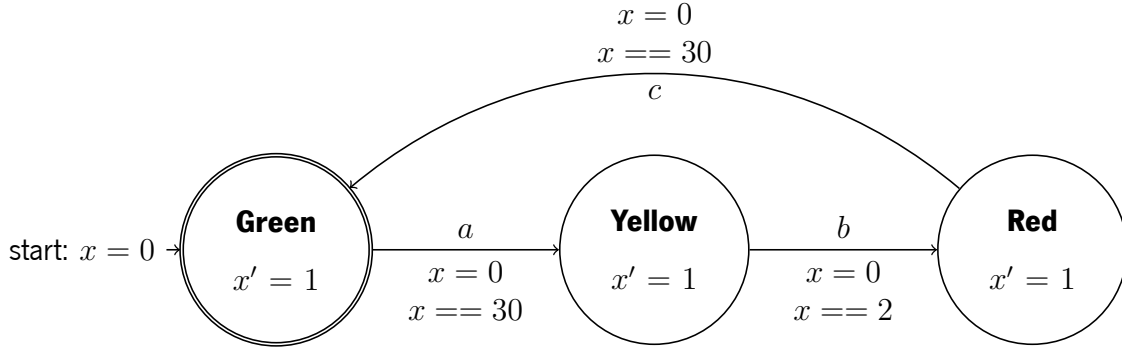


Figure 18: Deterministic Semaphore

Observe the simple example of a simple semaphore, that stays 30 seconds on the location green, 2 on yellow, and 30 on red 18, first of all, it clearly follows all the restrictions of 2.4, secondly, even without kleene algorithm, it's intuitively that the regular expression of the automaton is $R_s = (abc)^*$, therefore, we can apply the developed method to compile the automaton into Lince. Observe the following application of the rules:

$$\begin{aligned}
 & \text{compile } (abc)^* \rightarrow \\
 & \quad \{\text{by the rule (start}^{\rightarrow}\}) \\
 & x := 0; \text{ compile}'(abc)^* \rightarrow \\
 & \quad \{\text{by the rule (star}^{\rightarrow}\}) \\
 & x := 0; \text{ while true do } \{\text{compile}'(abc)\} \rightarrow \\
 & \quad \{\text{by the rules (conc}^{\rightarrow}) * 2\} \\
 & x := 0; \text{ while true do } \{\text{compile}'(a) \text{ compile}'(b) \text{ compile}'(c)\} \rightarrow \\
 & \quad \{\text{by the rules (symb}^{\rightarrow}) * 3, p = 1\} \\
 & x := 0; \text{ while true do } \{x' = 1 \text{ until_1 } (x == 30); x := 0; \\
 & x' = 1 \text{ until_1 } (x == 2); x = 0; x' = 1 \text{ until_1 } (x == 30); x = 0; \}
 \end{aligned}$$

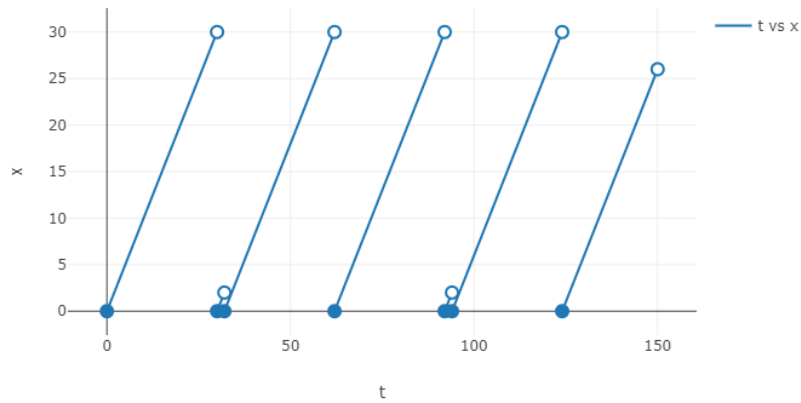


Figure 19: Semaphore output in Lince

The figure 19 is the result of simulating the semaphore automaton 18 by 150 seconds using the defined conversion method. The results of our study are promising, however, it's important to notice all it's flaws that are inherent to the method and Lince, first of all, following these rules, all automata must be bound by the restrictions defined in 2.4 making sure the automaton is deterministic, secondly, all transitions must be uniquely labeled, meaning there can't be 2 transitions with the same label, and last of all, the algorithm uses the Lince function **until** for the detection of the transitions guards and that function must be paired with a precision p , a time interval from which the condition will be checked, and this could cause serious troubles, too little precision and perhaps the guard won't be detected, too much and it will take from the simulation efficiency, some guards may even be impossible to detect, no matter the precision, this weakness is very clear since the inception of the tool [10].

3.3 Hybrid Automata Compiler

3.3.1 Syntax Compiler

With the objective of trying to simulate in Lince Hybrid automata that are not necessarily restricted to be deterministic 2.1. We now pretend to present a set of rules that instead of producing code from a regular expression, it does so from a string accepted by the regular, and therefore obtain the output of simulation that particular sequence of transitions.

Consider all automata functions previously, added with the *Inv* function, furthermore, consider a to be the label of the transition at the beginning of the string and t the tail of the string, and again p a pre-determined precision from which to detect the guards and invariants.

$$(\text{start} \rightarrow) \quad \text{compile } s \rightarrow \text{Init } \text{deadlock} := 0; \text{ compile}' s \quad (3.5)$$

$$(\text{symbol} \rightarrow) \quad \text{compile}' a \rightarrow f(L_a) \text{ until_} p \text{ (Guard}(a) \parallel \neg(\text{Inv}(L_a))); \quad (3.6)$$

$$\text{if (Guard}(a)) \text{ then Assign}(a) \text{ else deadlock} := 1$$

$$(\text{head} \rightarrow) \quad \text{compile}' (a : t) \rightarrow f(L_a) \text{ until_} p \text{ (Guard}(a) \parallel \neg(\text{Inv}(L_a))); \quad (3.7)$$

$$\text{if (Guard}(a)) \text{ then } \{ \text{compile}' t \} \text{ else deadlock} := 1;$$

$$(\text{delay} \rightarrow) \quad \text{compile}' (a : ' : t) \rightarrow f(L_a) \text{ until_} p \text{ (Guard}(a) \parallel \neg(\text{Inv}(L_a))); \quad (3.8)$$

$$\text{if (Guard}(a)) \text{ then } \{ f(L_a) \text{ for } p; \text{ compile}' (a : t) \} \text{ else deadlock} := 1;$$

The rules are actually quite simple and similar to the previous compiler 3.2.1, however, because of the existence of invariants, it is now possible for a transitions to not be executed if the invariant becomes invalid before it's guard is detected. In order to take this behavior into account a new variable named *deadlock* was automatically added. 3.6 is to compile the last symbol of the string and simply states to stay on location from which the transition leaves until the transitions is possible or the invariant of that location is no longer true, in this case entering a deadlock. 3.7 simplify executes a transaction calling the recursive to compile for the rest of the string. Finally, and perhaps the most complex 3.8 tries to delay a transaction, this rule tries to cover the case of a hybrid automata when it is time non-deterministic 2, in this rule, once the guard is detected the simulation won't execute the transition but it will evolve continuously in the same location for an amount p (Our precision), and try to detect the guard of the transition again, a transition can be delayed several amounts of time, " $a \dots$ " for example, will try to delay transition labeled a three times.

3.3.2 Example

Consider the following example of a non-deterministic automaton, we plan to compile in Lince the execution of the sequence of transitions "afd" with no delays, meaning all transitions are made as soon as enabled.

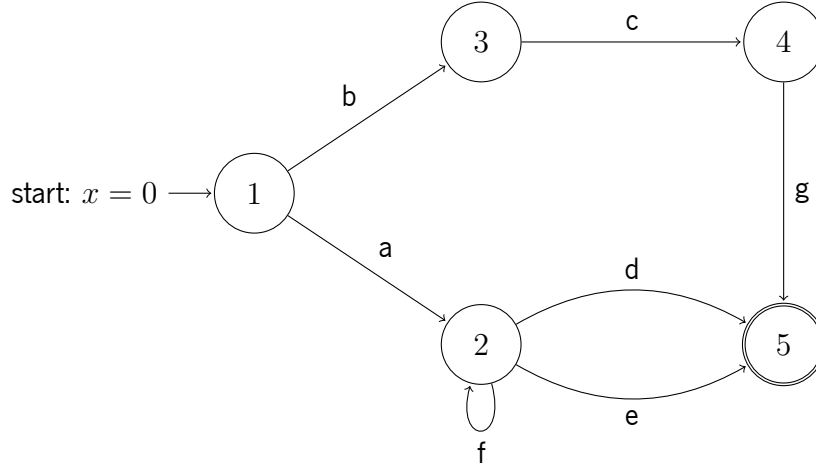


Figure 20: Automaton - Compiling Example

Σ	Guard	Assign			
a	$x == 3$	$x := 0;$	L	Inv	f
f	$x == 1$	$x := x + 1$	1	$x \leq 5$	$x' = 1;$
d	$x == 1$	$x := 1$	2	$x \leq 5$	$x' = 1;$

Table 7: Automaton - Functions

In the execution of this automaton following the sequence of transitions "afd" with the compilation rules 3.3.1, Lince will be able to simulate the following behavior 21, x starts in 0 following the Init function, then, x continuously evolves in location 1 until it detects the guard of transition a $x == 3$, afterwards, it enters location 2 resetting the value of x to 0. In location 2, x will evolve until it detects the guard of transition f when $x == 1$, following from there, x will stay in locating 2 but only after executing the assign function of f making x assume the value of 2, finally, x will continuously evolve from 2 until 5 where the invariant of location 2 stops being true entering a deadlock, never being able to execute transition d which guard is $x == 1$, making to the user, very clear that the sequence of transitions "afd" is not a valid execution of the system.

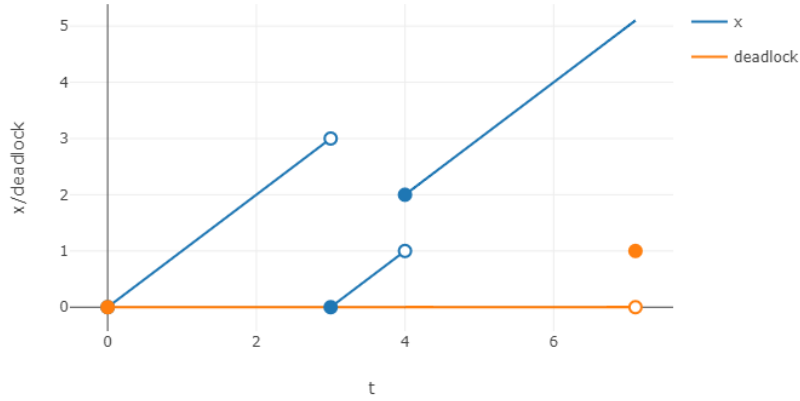


Figure 21: Lince output of simulating "afd" transitions of 20, with precision 0.1

Following these compiling rules, one string accepted by the regular expression, once compiled to Lince presents a possible run of the hybrid system, an execution that necessarily is deterministic by predetermine which transaction to make, and by doing that transitions as soon as it's possible or delaying that transition multiple times by p amounts of time where p is our precision. This methodology shows it's true potential when simulating all sequences of transitions up to certain size, as long as they are accepted by the regular expression, and therefore, get a clear view of the possible executions of the hybrid automaton. Compiling and simulating all those possibilities could lead to an understanding of the automaton and even if the system is deterministic or not.

However, the main problem with this proposal is that it's astronomically inefficient, if we add the possible delays of a transition, specially with low precision, there would be countless possible executions, simulating all these would create a process that is extensive and slow, basically impossible, making this method good as a case study, or to simulate mainly automata with very limited possible outputs.

3.4 Conclusion

Hybrid automata are a modeling approach to hybrid systems, with a visual representation and very easy to interpret, being one of the most used formalisms to study and validate classic and simple hybrid systems present in our day to day, like factories control mechanisms, home appliances and daily electronics. As a modeling approach of hybrid systems, automata can be converted to the simulation language used in Lince, by logically interpreting their behavior and using regular expressions as a middle mechanism, however, these automata may possess varied characteristics that Lince is not able to simulate directly like a non-deterministic behavior, a restriction that was partially solved in this study by the construction and interpretation of multiple simulations of the same automata in Lince [3.3.1](#). Other behaviors like non-linear continuous dynamics and automaton guards and invariants that require a precise detection also lead to a necessity in being extremely careful while modeling automata in Lince, on the other hand, with certain restrictions applied on the automata [2.4](#), it's simulation is quite easy and trivial. [3.2.1](#) As a case study, all methodologies mentioned in this chapter, including the kleene algorithm are all presented more practically in a computational environment encoded in Haskell with a simple example [A.1](#).

Tools that verify or simulate automata are already present in the scientific community, we mentioned Uppaal before, but notice that Uppaal and Lince work in two very different ways, while Uppaal, for example, is able to model non-deterministic systems, Uppaal works only with timed automata, contrary to Lince that allows to more complex continuous dynamics, so it's safe to say that despite it's serious drawbacks, using Lince to simulate hybrid automata could be very useful, specially when these are not very complex in the number of locations and transitions.

Chapter 4

Taxonomy

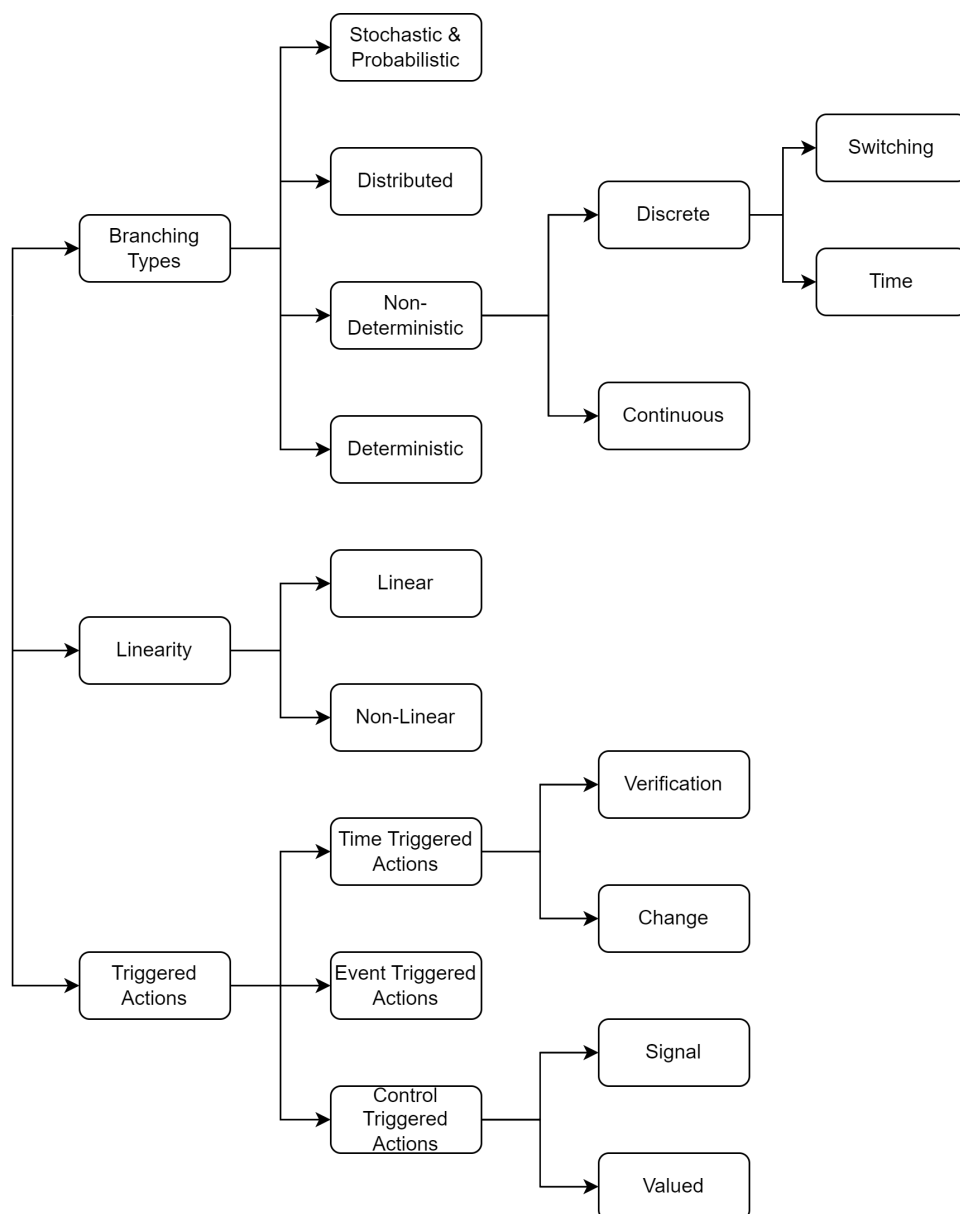


Figure 22: Taxonomy

As scarcely mentioned throughout all this document, Hybrid systems have an array of different characteristics which renders their validation and simulation demanding. Specifically, several of these behaviors, like its determinism, were already mentioned in an automata environment 2.3.2 and previously hybrid system modeling approaches studies made by the scientific community 2.2.1, we present here a concise group and subgroups of identified hybrid systems taking into account the different characteristics observed while modeling and testing hybrid systems in Lince. We take into account for example system branching type, its linearity and types of triggered discrete actions, the summary of full taxonomy structure can be observed in the figure 22.

However, before we dive in into explanations and examples of the different purposed categories of hybrid systems, in order to offer a concise demonstration we need an all encompassing model approach to explain each example and behavior, for that goal, we take into account the simulation language of Lince we use in this thesis and we purpose an extended set of operators that would make Lince able to represent all of the hybrid systems desired in this taxonomy. Nonetheless, notice that, one, these new operators are not actually implemented in Lince, being a theoretic possible future improvement, and second, most of these syntax operators are actually not new and heavily inspired on predefined algebraic language approaches to hybrid systems, specially one by the name of Hybrid Communicating Sequential Processes (HCSP) [34], Hofner's work in Algebraic Calculi for Hybrid Systems [12] and André Platzer approach to stochastic hybrid systems (SHPs) [30]

Take special attention to the next table 8, with each new purposed operator and their small explanation, knowing that each one will be included in the examples of the next sections:

Operator	Explanation
$a + b$	Internal and non-deterministic system choice between executing program a or executing program b , causing the possibility of multiple outputs.
$x := [a, b]$	Non-deterministic assignment of an arbitrary real number between a and b to variable x .
$f(X) \text{ until+ } (E)$	Extension of the until function in Lince. Evolves the system continuously, choosing to stop at any possible instant in time as long as the condition E holds, instead of the original function that stops as soon as the condition holds.
$(x' = a + x' = b) \text{ for } t$	Extension of the for function in Lince. Evolve the system continuously for t amount of time, in which x flows possibly alternating between differential equation a and differential equation b .
$a \parallel b$	Parallelism operator, run program a and program b in parallel, sharing the same world space (time and variables).
$f(X) \text{ until } (x)ch?$	Extension of the until function. Evolve the system continuously, until it synchronizes at the receiving end of the channel labeled ch . Assign to variable x the value received in the channel.
$f(X) \text{ until } (x)ch!$	Extension of the until function. Evolve the system continuously, until it synchronizes at the sending end of the channel labeled ch . Sends the value of the variable x through the channel.
$\lambda a +_p b \beta$	Internal choice between executing program a or b , but providing probabilities of occurrence to each path, λ and β .
$x := \text{random}(a, b)$	Assigning to variable x any value between a and b following a continuous uniform distribution. $X \sim U(a, b)$.
$x := \text{normal}(\mu, \sigma)$	Assigning to variable x a value following a Gaussian distribution with expected value μ and standard deviation σ . $X \sim \mathcal{N}(\mu, \sigma^2)$.
$x' = \mu + \sigma W \text{ for } t$	Extension of the function for in Lince. Represents a stochastic continuous movement with flow μ an volatility σ using the Wiener Process, for a t amount of time

Table 8: Lince Extended Syntax

4.1 Continuous Dynamics Linearity

4.1.1 Linear Continuous Dynamic

Definition 7. A system is said to be *Linear* if it's continuous evolution is always represented by linear differential equations.

In this case, all the system variables that can flow continuously with time are linked to a differential equation that is a polynomial of either degree 0 or degree 1. This behavior, however, doesn't mean that the variable in question will have a plotted evolution through time that can be represented with a simple line. Take the next example of a projectile motion [23](#) (a ball) making a trajectory in a 2D space, the ball moves affected by it's initial velocity, both horizontally v_x and vertically v_y , and it's also continuously affected the gravity force g .

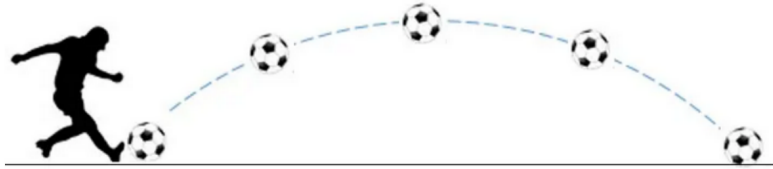


Figure 23: Projectile motion

$$x' = v_x, y' = v_y, v'_x = 0, v'_y = -g \quad (4.1)$$

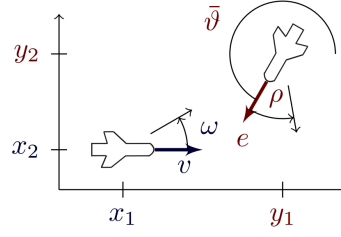
As it can be observed [4.1](#), each variable has a linear differential equation to represent it's continuous flow throughout time, therefore, this system is linear.

4.1.2 Non-Linear Continuous Dynamic

Definition 8. A system is said to be *non-linear* when there's an existence of a non-linear differential equations to dictate how a system variable flows through time.

Take the following famous example of an Dubins aircraft dynamic [\[28\]](#), In this example, the movement of an aircraft is explained by two variables, an angle ϑ that varies through time equals to an angular velocity ω and a constant velocity v , furthermore, simple trigonometry is used to separate the movement in both axis, resulting to the following continuous behavior:

Figure 24: Dubins aircraft dynamics



$$\begin{aligned} x' &= v \cos \vartheta & y' &= v \sin \vartheta & \vartheta' &= \omega \end{aligned} \quad (4.2)$$

This system presents non-linear dynamics because the position in both axis continuous movement are represented by non-linear differential equations, causing some tools to have difficulty in simulating or validating these types of systems.

4.2 Braching Types

4.2.1 Distributed Systems

Definition 9. A distributed system is modeling approach that aim to define multiple hybrid systems executing in the same world space.

As a very studied approach to hybrid systems, a distributed system represents a class of hybrid systems that are modeled as two or more separate programs that execute in parallel, sharing the same continuous world (time and space), and sometimes synchronizing discrete actions with each other. Take the following example, two cars are moving in a one dimensional space, the car on the left starts at position $p_1 = -10$ with constant velocity of $v_1 = 1$, the car on the right starts at $p_2 = 10$ and velocity $v_2 = -1$, both cars want to reach position zero, however, both will try to stop in order to avoid a collision, the first car has faster sensors and can make decisions each second, the second car has sensors that reach farther being able to detect objects at longer distances, each car will break first?, each car will reach position zero first?, will the cars collide with each other? Take the following modeling approach in the extended language of Lince where $a \parallel b$ means program a running in parallel with program b :

1. Declare the initial state of our system, first car is on position -10 , second car on position 10 , both moving against each other with constant speed.

```
1  p1:=-10; p2:=10; v1:=1; v2:=-1; a1:=0; a2:=0;
```

2. Program of first car, runs a sensor at each 0.5 seconds that detects if the other car is at distance less than 10 meters, choosing to decelerate if yes.

```
1  {  
2    while true do {  
3      p1'=v1, v1'=a1 for 0.5;  
4      if (p2-p1<10) then a1:=-1; else a1:=1;}  
5    }
```

3. Usage of the parallel operator $||$ from the proposed syntax [8](#) for the program of both cars to run in the same space.

```
1    ||
```

4. Program of second car, runs a sensor at each second that detects if the other car is at distance less than 15 meters, choosing to decelerate if yes.

```
1  {  
2    while true do {  
3      x2'=v2, v2'=a2 for 1;  
4      if (p2-p1<15) then a2:=1; else a2:=-1;}  
5    }
```

This particular distributed system can be carefully modeled in Lince [25](#) as a singular system by combining the time and content of the discrete actions of each system, however, this manual technique is not advisable for more complex systems, requiring a more systematic methodology, a field of study that could be explored upon in later improvements of the tool. We can observe that, in this system, it's the car on the left that reaches the position zero first, and we can also observe that both cars don't collide.

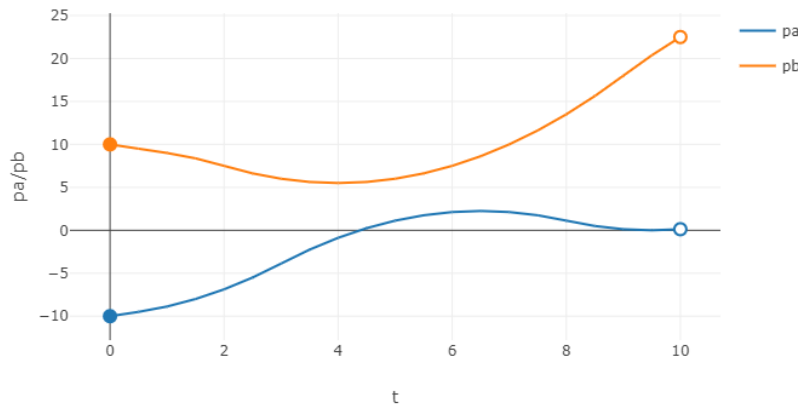


Figure 25: Distributed Breaking System

4.2.2 Deterministic Systems

Definition 10. A deterministic system is a system that given the same input always presents the same behavior, meaning that for each specific time the system evolves to, there's only one possible result shown by all its variables.

Deterministic systems are varied and simple, they can be explained by the absence of any choice in our system. We highlight here the difference between a choice and a decision [28], a decision is deterministic for it takes into account information about the system to decide which change to apply, a simple **if then else** statement in Lince is a decision, a choice however, is non-deterministic, and it's modeled with no information besides the possibility of multiple outcomes. Take the following example of a deterministic system modeled in Lince, an improved version of the cruise controller presented in 2.2.6, extended to two dimensions and it's respective output in Lince 26.

1. Initial state of the system, a car is on position (0,0), moving in both axis, at the same velocity and acceleration.

```

1    // Initial state of the system
2    x:=0; y:=0;
3    vx:=2; vy:=2;
4    ax:=5; ay:=5;

```

2. The system enters a loop, it checks if velocity in both axis has achieved critical values to decide if it should accelerate or break, the cruise control tries to maintain a velocity of $10m/s$


```

1   while true do {
2       if vx<=10 then ax:=5; else ax:=-4;
3       if vy<=10 then ay:=7; else ay:=-2;

```

3. The system continuously evolve for 1 second until the sensor runs again.

```

1   x'=vx, vx'=ax,
2   y'=vy, vy'=ay for 1;
3   }

```

In a deterministic system, all discrete actions happen at predetermined instants in time, in this case, discrete actions happen always at each second, the execution of these discrete actions always result in the same output, in this case, a decision is made taking into account the information about the state of our system, with the **if then else** statement. Furthermore, all the differential equation representing the continuous movement are deterministic.

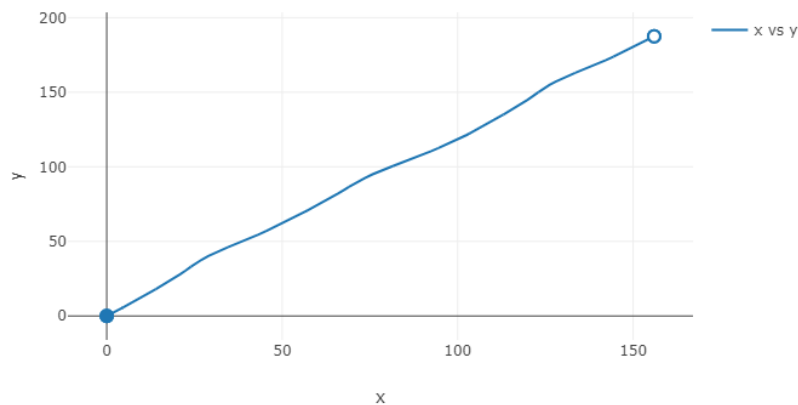


Figure 26: 2D Cruise Control - Lince

4.2.3 Non-Deterministic Systems

Non-Deterministic Discrete Dynamic

Usually, a system is non-deterministic because of its discrete dynamics. For example, a system can have non-deterministic choices about the time of its discrete actions and about what changes those actions can apply to the system.

Switching

Definition 11. *A Non-deterministic switching system is a hybrid system in which its discrete actions may cause possible different outcomes.*

For a system to be deterministic each discrete action must map a state to a single next state, so when a discrete action maps one state to multiple states then that action causes a non-deterministic switching behavior, similar to the concept explained before in automata [3](#).

Imagine an individual is driving a car and suddenly spots a moving obstacle almost reaching the road, without having the time or ability to think to make a decision based on the information of the system, the person is forced to make one of several random choices: The person could accelerate and try to reach the intersection before the obstacle or they could hit the breaks and try to stop on time, In this case there's two programs the system can execute, changing the outcomes of the system and making it non-deterministic.

1. Initial state of the system, moving at a constant velocity in a one dimensional space

```
1      x:=0; v:=2; a:=0;
```

2. The system continuous evolve for 30 seconds where it spots an obstacle in the distance

```
1      x'=v, v'=a for 30;
```

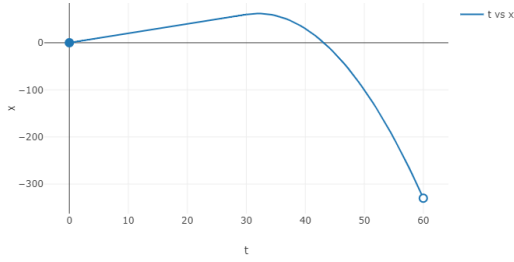
3. Once the obstacle is spotted, the system makes a non-deterministic choice between accelerating or breaking

```
1      (a:= -1) + (a:=1)
```

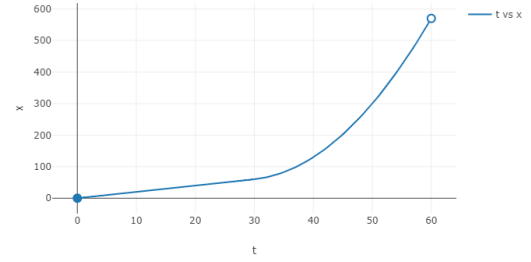
4. Afterwards, the system continuous evolve by consequence of the choice made.

```
1      x'=v, v'=a for 30;
```

Because this system has a **discrete** amount of possible executions, two, each possibility can be modeled in Lince separately [27](#), however, there's also the possibility of a **continuous** non-deterministic choice of being made, using for example, the previous function $a := [-1, 1]$ in [8](#) to model all executions in which the acceleration can become all values between the interval.



(a) Choosing to break



(b) Choosing to accelerate

Figure 27: Choosing to break or accelerate

Time

Definition 12. A Non-deterministic system in time is a hybrid system in which it's discrete actions may happen at distinct moments in time.

When a discrete action doesn't happen at an instant in time that is solely determined by the current state and inputs of the system, then a system is said to be non-deterministic in time. A non-deterministic system in time occurs when it's discrete actions can be delayed, causing the system to make a choice between continuously evolving or stop and execute a discrete action. Imagine the following famous example of a thermostat [\[15\]](#) in the form of an automaton and it's respective purposed modeling in Lince following the previous syntax [8](#).

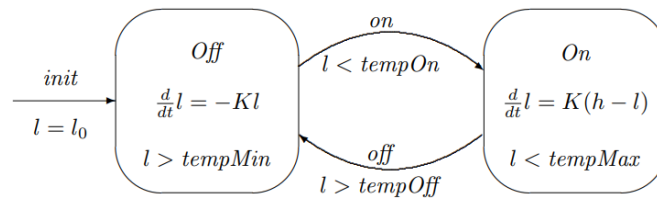


Figure 28: Non-Deterministic Thermostat

1. Initial state of the system, starts with temperature of 10, all other variables are constants, h is the power of the heater, k a property of the room.

```

1      l:=10; h:=5; K:=1;
2      tempMin:=5; tempMax:=20;
3      tempOn:=10; tempOff:=10;

```

2. Enters a loop, starts by executing the continuous dynamic of the location *Off*, where the temperature decreases until it chooses to turn on, It can do at any instant as long as the temperature is between the minimum temperature (Invariant) and the guard of the transition (tempOn)

```

1      while (true) do {
2          l' = -K * l until+ (l>tempMin && l<tempOn);

```

3. Executes the location *On*, where the temperature increases until it chooses to turn off similar to previously

```

1      l' = K*(h-l) until+ (l>tempOff && l<tempMax);
2  }

```

As it could be seen in the previous automata study [4](#), this thermostat automaton is non-deterministic because the transitions *on* and *off* can both be made not at a specific instant in time, but a continuous interval, for example, the transition *on* can be made at any given time as long as the temperature l is between $tempMin$ and $tempOn$, causing the system to present a non-deterministic behavior in time.

Non-Deterministic Continuous Dynamic

Definition 13. A hybrid system is said to be non-deterministic in its continuous dynamics when the flow of its variables are dictated by an equation that is non-deterministic.

$$x' = 1 + x' = -1 \text{ for } 10; \quad (4.3)$$

Imagine the following continuous dynamic system of equations [4.3](#). In this example the variable x is switching between two different continuous behaviors throughout its execution. This hybrid system could present an execution where x would evolve by 1 for 10 seconds, or flow with rate of -1 for 10 seconds, or switch between 1 and -1 each 2 seconds, or any output in between these possibilities, presenting a behavior truly non-deterministic with infinite possibilities, and being incredibly difficult to simulate or validate. It may sound counterproductive on why it would be useful to model a system that is so unpredictable, however,

in certain field this random and uncertain continuous dynamic is actually a very present and studied behavior, it may simulate noise in movement or unpredictable variances like stock-markets. We present here a classic example to explain why non-deterministic continuous hybrid systems are a field of study worth pursuing. The Brownian motion is a perceived random movement of a particle suspended in a liquid or gas medium, A certain particle presents a Brownian motion when it suffers from an seemingly infinite and random number of collision with an unspecific amounts of smaller particles causing it's movement to continuously switch between different velocities and direction [29](#).

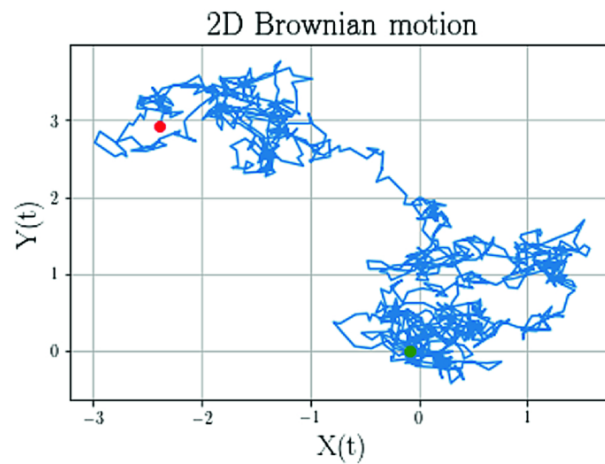


Figure 29: Brownian Motion in a 2D Space [\[18\]](#)

Although it may seem that such collisions would be better modeled as an infinite amount of discrete events that happen at non-deterministic time and causes non-deterministic changes, this would actually be quite unproductive, in fact, a Brownian particle can undergo a number of collisions roughly around 10^{14} collisions per second [\[5\]](#), causing the study of these systems to be condensed into continuous dynamics. The study of non-deterministic continuous behaviors like the Brownian movement is so chaotic and boundless that the only way to do so, and that it's been done so in the community, is by applying stochastic analysis that aims to explain such notions via the use of probabilities, this will be the next type of hybrid systems that we will discuss.

4.2.4 Stochastic and Probabilistic Systems

Definition 14. A stochastic/probabilistic system is a type of a non-deterministic system from which the outcome is random and can be quantified by probabilities.

So for example, in a practical setting, an execution of a stochastic/probabilistic system would involve the use of inner computational functions like for example the ones provided in the syntax table 8 *random*, *normal* and $+_p$, that use probabilistic distributions to make random choices to provide a probable outcome, because of the usage of probabilities some outcomes could be probable then others and a study could be made about the frequency of occurrence of each outcome. This contrasts with usual non-deterministic concept on which the choices of the system are not governed by randomness and simply state the possibility of different executions.

In the field of hybrid systems, a **probabilistic** hybrid system is a concept used to describe systems with discrete changes that follow probabilistic distributions from which we have information about. An example of a probabilistic system would be the previous model 1 of a vehicle making a choice between accelerate or breaking, but this time, provide a probability of accelerating and a probability of breaking, using the operator $+_p$ discussed before 8. In this case, another probabilistic system would be to randomize the value of the acceleration between minimum -1 and maximum 1 using for example the following operator: $acceleration := random(-1, 1)$.

On the other hand, a **stochastic** hybrid system is a concept usually used to describe the usage of probabilistic distributions to model the continuous dynamic of a hybrid system. Remember the previous example of the Brownian motion, mathematically, one of the most used solutions to simulate this phenomena is called the Wiener process also known as Brownian motion, a continuous-time stochastic process that indexes to each time t a random value W_t , and it works with the following properties [4]:

1. $W_0 = 0$ almost surely.
2. W has **independent increments**: for every $t > 0$, the future increments $W_{t+u} - W_t$, $u \geq 0$, are independent of the past values W_s , $s < t$.
3. W has **Gaussian increments**: $W_{t+u} - W_t$ is normally distributed with mean 0 and variance u , $W_{t+u} - W_t \sim \mathcal{N}(0, u)$.
4. W has almost surely continuous paths: W_t is almost surely continuous in t .

The Wiener process uses a probabilistic Gaussian distribution in small increments of time to calculate how much a value has varied. In the field of hybrid systems, continuous-time stochastic processes like

the Wiener Process are important because they can be integrated to define **stochastic differential equations (SDE)**, these equations could be then represent a stochastic continuous behavior in our variables. For example, in the proposed Lince syntax 8, the following example 4.4. represents a particle movement in a one dimensional space, where it's position x flows with a certain velocity v but with small stochastic variations (noise) with intensity σ .

$$x' = v + \sigma W \text{ for } 10; \quad (4.4)$$

On the other hand, a popular stochastic differential equation to study the movement of stock prices would assume a similar behavior 4.5, however, in this case, while the asset price s would assume an expected deterministic variation of μ and a volatility of σ , the more the value of the stock, the more variations it will present, reason why there's the multiplication of the asset value to it's variation e.g. $s \cdot \mu$.

$$s' = s \cdot \mu + s \cdot \sigma W \text{ for } 10; \quad (4.5)$$

In fact, André Platez, in "Stochastic Differential Dynamic Logic for Stochastic Hybrid Programs" [30] has already proposed a syntax to stochastic hybrid systems that uses stochastic differential equations with the Wiener Process similar these examples.

4.3 Triggered Actions

A hybrid system has a continuous dynamic that changes discretely throughout time, these discrete changes can be awoken by the presence of a triggered action, similar to the concept of an event in 3. Here we present the different factors that can be modeled as provokers of discrete change.

4.3.1 Time Triggered Actions

One of the factors that can trigger a discrete action and consequently possibly change the state of an hybrid system is the passage of a predetermined amount of time.

Change

The time triggered action can present a case from which it always causes a change in our system. For example, imagine a very simple cruise control, that changes it's acceleration each 30 seconds, switching between breaking and accelerating. In this case, at every 30 seconds the system will present a clear change that will cause the system to continuously flow differently from before the action was triggered.

1. Initial state of the system, a car starts at position 0 moving in a one dimensional space at a certain velocity and acceleration.

```
1      x:=0; v:=2; a:=1;
```

2. Enters a loop triggering a discrete action that is predetermined by the passage of 30 seconds.

```
1      while true do {
2          x'=v, v'= a for 30;
```

3. Makes a certain change by inverting it's acceleration. If the car was accelerating in the last 40 seconds it decides to break, if it's decelerating decides to accelerate.

```
1      a := - a;
2  }
```

As a system with time triggered actions, the simulation of this system is possible in Lince, as shown in the figure 30

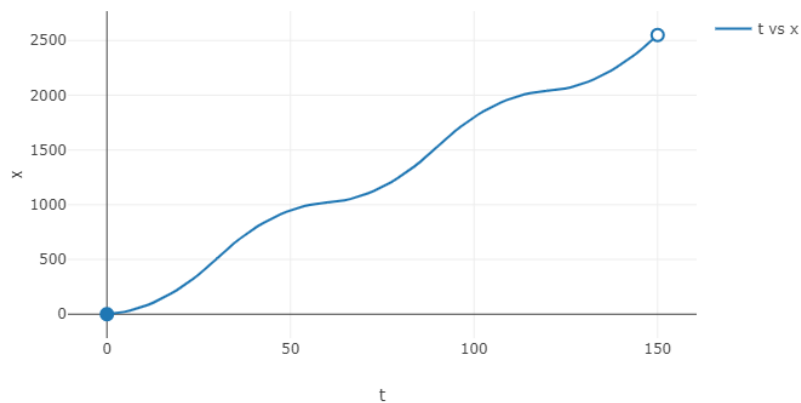


Figure 30: Breaking and Accelerating - Lince

Interrogation

On the other spectrum of a time triggered action, these may be useful not to applies certain changes, but to evaluate the current state of a system, checking if a certain condition is met, and applying the change only if it's deemed necessary, and seemingly continuously evolving as previously if not. In this case, the system is modeled as continuous evolving in small increments of time, waiting to detect a

certain state. This methodology is mostly common when modeling technological processes like sensors, computer communication or polling. Take the previous example of a cruise control 1, but this time simpler, a car has a sensor that detects it's velocity each second, if the velocity has reached dangerous values then it will decelerate.

1. Initial state of the system, a car starts at position 0 moving in a one dimensional space at a certain velocity and acceleration.

```
1  x:=0; v:=2; a:=1;
```

2. Enters a loop triggering a discrete action that is predetermined by the passage of 1 second.

```
1  while (true) do {
2    x'=v,v'= a for 1;
```

3. Decides to apply a change (accelerate) only if a certain condition is checked. Otherwise the system stays with the same flow equation.

```
1    if (v>10) then a:=-1 else a:=a;
2  }
```

The figure 30 represents the output of running such program in Lince.

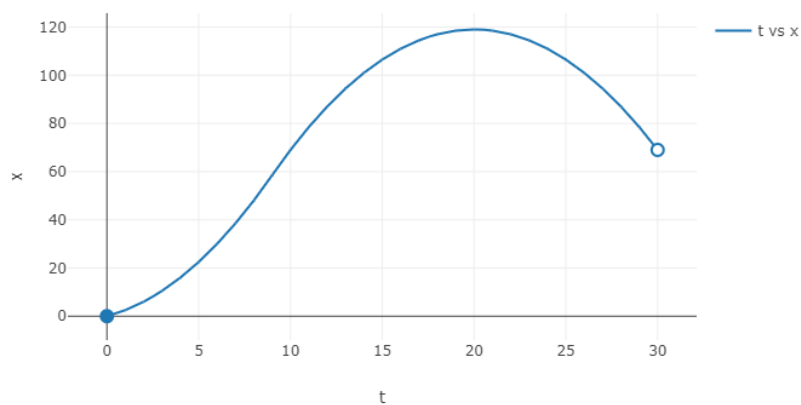


Figure 31: Breaking If - Lince

4.3.2 Event Triggered Actions

An event triggered action is a change in the system modeled to restrict the continuous behavior of the system to contain only situations that make physical sense. For example, the famous bouncing ball that collides with the ground: the exact moment when the ball hits the ground has to be detected to stop the system from continuously evolving through the ground, a behavior that we wish to preclude. An event triggered action requires an absolute precision in detecting the respective condition so that the system is simulated correctly.

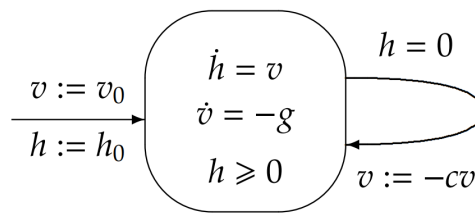


Figure 32: Bouncing Ball [15]

1. Initial state of the system, the ball starts at maximum height with no velocity, the gravitational force is $-9.8m/s^2$

```
1      h:=10; v:=0; a:=-9.8;
```

2. Enters a loop, continuous evolve until a discrete action is triggered by the detection of a condition (collision with the ground)

```
1  while (true) do {
2    h'=v, v'= a until (h == 0);
```

3. After a collision is detected, it re-bounces at half of it's velocity.

```
1      v:=-0.5*v;
2  }
```

Event trigger actions are important in hybrid system simulation tools like Lince for their respective detection difficulty, for example, the previous code 1, despite being theoretically, a faithful conversion of the automaton 32, it won't present a correct simulation in our tool because of the way the **until** function works, that turns an event triggered action into a time triggered interrogation 4.3.1 with a certain precision

associated. This restriction doesn't mean some version of the bouncing ball example can't be modeled in Lince [10, 3], as long as it's modeled taking into attention that the height of the ball can reach values bellow zero for small amounts of time 4.3.2.

1. Initial state of the bouncing ball, equal to above

```
1  h:=10; v:=0; g:=-9.8;
```

2. Enters a loop, continuous evolve by small increments in time (time interrogations 4.3.1) until it detects a broader condition (height is bellow the ground). In this case, Lince uses inner algorithms to try and calculate with maximum precision the moment in which the height becomes less then zero.

```
1  while (true) do {
2  h'=v, v'= g until (h < 0);
```

3. Adjusts the height to zero and re-bounces it's velocity.

```
1  h:=0; v:=-0.5*v;
2  }
```

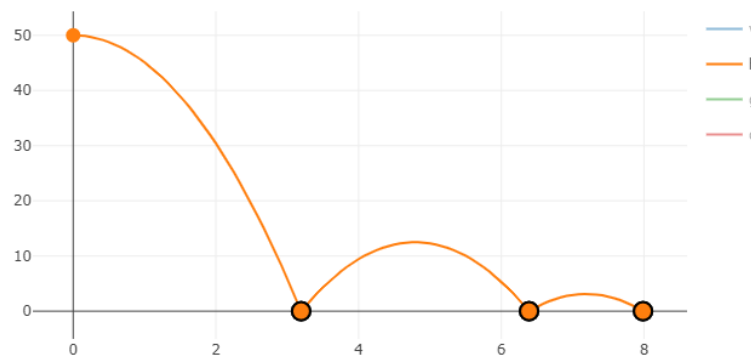


Figure 33: Bouncing Ball Lince

4.3.3 Controlled Triggered Actions

Another factor that can trigger a discrete action is an external signal, a concept previously mentioned [2](#), these triggered mechanism are mostly modeled on distributed systems [4.2.1](#).

Signal

The external signal can be a simple one, like a switch, it doesn't transmits to the system any information besides the time of occurrence, synchronizing both systems, the previous lamp example on the Uppaal section [1](#) belongs to this category. Take the following example of a very simple light switch.

1. Initial state of the system, $c1$ is an internal clocks of the first system, the lamp, $c2$ is an internal clock of the second system, a switch. the variable *light* represents if the lamp is on 1 or off 0.

```
1      c1:=0; c2:=0; light:=0;
```

2. Program of the first system, the lamp, the clocks continuously evolve until it detects an external signal from channel *ch*, this external signal triggers a discrete action that changes variable *light*.

```
1  {
2    while true do {
3      c1'=1 until ch?
4      light = 1 - light;
5    }
6  }
```

3. Parallelism operator [8](#) that makes both systems run in parallel.

```
1      ||
```

4. Program of the second system, the switch, each 10 seconds, the system tries to synchronize in the channel *ch*, sending a signal to the lamp.

```
1  {
2    while true do {
3      c2'=1 for 10;
4      c2'=1 until ch!
5    }
6  }
```

Valued

In a similar way, these channels can sometimes carry information through them, for example, a controller joystick for a remote vehicle that transmits the value of the acceleration to the vehicle. Modeling channels as mechanisms capable of sending and receiving information is actually not a new concept, being a prominent factor of Hybrid Communicating Sequential Processes language (HCSP) [34]

1. Initial state of the system, internal clocks for both systems $c1$ and $c2$, furthermore, the position, velocity and acceleration of the remote controlled vehicle.

```
1      c1:=0; c2:=0; x:=0; v:=0; a:=0;
```

2. The program for the vehicle, a loop that continuously evolve it's movement until it detects an external signal, once it does so, updates it's acceleration variable according to the received value from the channel ch .

```
1  {
2      while true do {
3          x'=v; v'=a until (a)ch?
4      }
5  }
```

3. Parallelism operator [8](#) that makes both systems run in parallel.

```
1  ||
```

4. The program for the controller, each second tries to synchronize in the channel ch , sending through it a new value for the acceleration.

```
1  {
2      while true do {
3          c2'=1 for 1;
4          a:= a+1;
5          c2'=1 until (a)ch!
6      }
7  }
```

Chapter 5

Discussions

In this chapter we intend to present a deep approach into practical and more complex examples of hybrid systems, focusing on autonomous driving and small control devices, consequently, we intend to model these in Lince following a modeling approach that aims to identify each system characteristics according to the concepts of the developed taxonomy and contrasting these behaviors with Lince's capabilities, and finally, we intend to take found conclusions for future improvements or behaviors to be attentive about when modeling in Lince like non-determinism, event triggered actions and non-linearity, as well as highlight Lince usefulness when modeling critical hybrid systems. The previous methodologies for automata conversion will also be used and tested in a more practical environment in classic automata like the water tank, the thermostat and the gas burner .

5.1 Investigation Methodology

Considering the existence of a taxonomy where each hybrid system category was identified and explained, a clear methodology is now presented to be followed in this chapter regarding the modeling of hybrid systems using Lince. This methodology will contain a clear order of steps to be taken into account in order to facilitate not only the conversion of hybrid systems into Lince but also the identification of Lince's limitations and it's advantages. In the following examples, each hybrid system will be presented in two main parts:

The first part will introduce the system, specially it's use and inner function, most importantly we'll classify the system taking into account the developed taxonomy, identifying what behaviors could cause difficulties while modeling these on the tool and what methodologies can and are used. The second part consists on showing the output Lince provides and how such can be used to optimize and study or validate the hybrid system in question, additionally, a discussion regarding possible improvements to be made on Lince to turn the representation of similar type of hybrid systems easier.

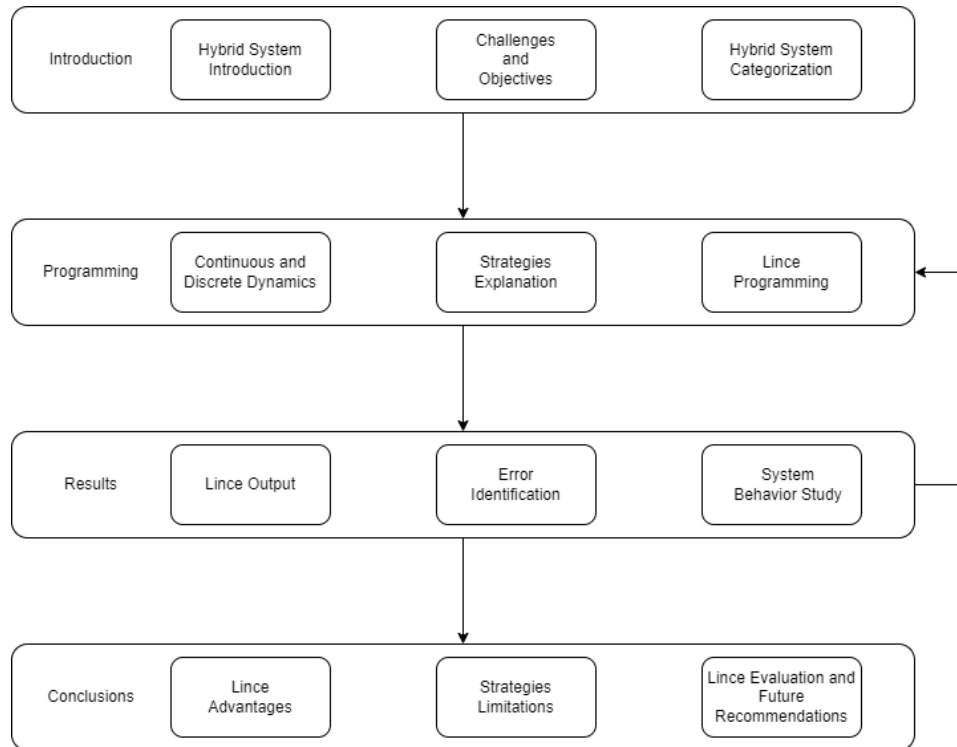


Figure 34: Investigation Methodology

5.2 Automata

5.2.1 Water Tank

Introduction and Methods

To start the study let's dive in on the following example of hybrid system, a fluid level control. A fluid level control encapsulates a wide class of different examples, from managing the level of a trivial single tank, to a wide array and complex network of tanks with several valves and plumps with different flows of liquid, the main idea of the fluid control is maintaining a required fluid level by controlling the fluid input and drain speed, opening and closing it's several valves and plumps, being these actions sometimes with a certain delay. More complex fluid level control involves tanks being interconnected occasionally at different heights, the existence of only one input of fluid being schedule between several tanks, and variable input and drain rates.

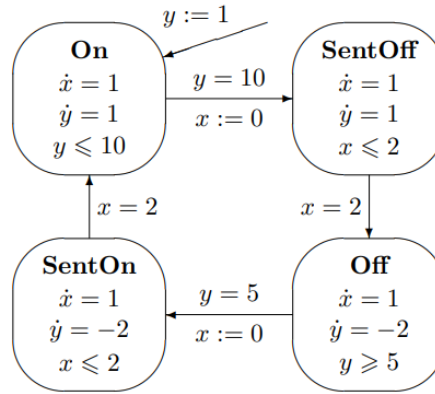


Figure 35: Water Tank hybrid automaton [15]

For the sake of this study let's observe the following very simple fluid level control hybrid system [15], A water level in a tank is controlled through a monitor which has the capacity to sense the water level at all times having zero delay, ignore for now, the fact that this is very unrealistic, this controller has the simple capability of turning a pump on and off, the water level changes by simple constant rates, when the pump is off the water level, designated by the letter y , decreases by 2 cm per second, when the pump is on the water level raises by one centimeter per second. The requirements of the system is to maintain at all times a level between 1 and 12 cm, however the signal between the controller and the pump is delayed by two seconds, thus the commands should be sent earlier. At which thresholds should the controller send the signal for the requirement to be true? That is something the validation of this hybrid system allows to do, which may seem trivial in this simple example, but can be very complicated on more complex fluid level controls. The hybrid automaton model that represents this system has the following locations:

- **On** - the pump is On
- **SendOff** - The pump is on, but the signal to stop the pump is sent
- **Off** - the pump is off
- **SendOn** - the pump is off, but signal to star the pump is sent.

With the automaton represented, the following question is, how can we turn it's behavior to Lince and do the following validation of the requirement, for that, we have to identify which type of system this fluid control is considering the developed taxonomy. First of all, all differential equations that represent the flow of the system in all locations are linear, which is good, second of all, the system is deterministic both in regards to switching, considering one location leads a single next location, and it's also deterministic in

regards to time, this determinism is modeled in automata by the junction between the invariant of each location and the guard of the transition out of that state, for example:

- On the location **On**, the invariant $y \leq 10$ together with the guard $y == 10$, indicates that the system leaves the location **On** always when $y = 10$, never before, never after, and it also means it can't stay in the location, forcing the system to leave, therefore this transition is modeled in the automaton as an event triggered action, with border condition $y == 10$, although this only happens because the unrealistic expectation that the sensor detects continuously the level of the fluid, without any delay. In reality, detection of the level fluid at the exact point of $y == 10$ is impossible for any sensor, there's always a precision associated. We'll label this transition with a .
- In a similar way, all other transitions behave in similar manner making the automaton deterministic in time. We'll label these transition b , c and d in order.

In the end, considering the deterministic behavior of the automaton, it is possible to interpret it following the restrictions in definition 2.4, where the invariants are ignored and each transition is made as soon as enabled, therefore, it is possible to apply the conversion method previously mentioned 3.2.1 to calculate a valid program in Lince that correctly simulates this automaton. Consider the obvious regular expression $R = (abcd)^*$ to represent the water tank automaton.

Results and Conclusion

1. The initial state of our system where the water level is one and the clock starts at 0.

```
1      x:=0; y:=1;
```

2. A loop while created by the kleene star of the regular expression of this automaton 3.2.1

```
1      while (true) do {
```

3. The code correspondent to the compilation of the first transition a leaving location **on** (3.2.1), it triggers once the water level equals to 10 resetting the value of the clock, consider the precision 0.5 seconds.

```
1      x' = 1, y' = 1 until_0.5 (y==10); x:=0;
```

4. Similar to the above, the compilation of all other 3 transitions b , c and d in order.

```
1      x' = 1, y' = 1 until_0.5 (x==2);
2      x' = 1, y' = -2 until_0.5 (y==5); x:=0;
```

```

3      x' = 1. v' = 1 until 0.5 (x==2):
4      }

```

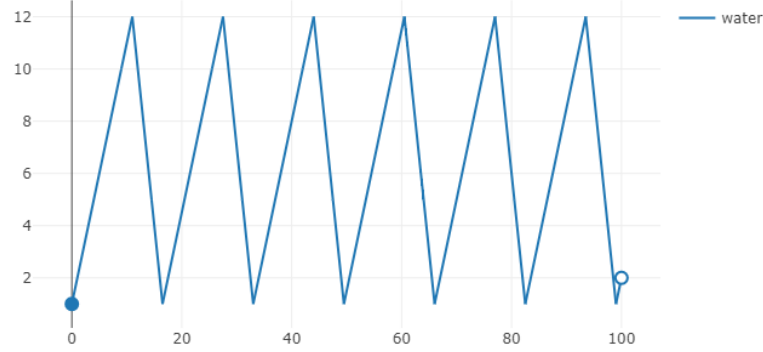


Figure 36: Water Tank Lince Output

As it can be observed, Lince proved the requirement, and the fluid level never surpasses the values out of 1 and 12 cm, validating the hybrid system. With this example we trivially proved that one: automata can be coded in Lince, for both of them are just different modeling approaches to hybrid systems, two, this transaction is easily made if the automaton doesn't have characteristics that Lince can't represent like non-linear continuous dynamic and/or non-determinism, three, this conversion can be done intuitively or automatically by interpreting the behavior of the automaton.

5.2.2 Thermostat

Introduction and Methods

The next hybrid systems we will be modeling in Lince is a thermostat, the same one that was previously mentioned [28](#). A thermostat is one the simplest examples of hybrid systems, showing how these are present in the day to day operation of the our life.

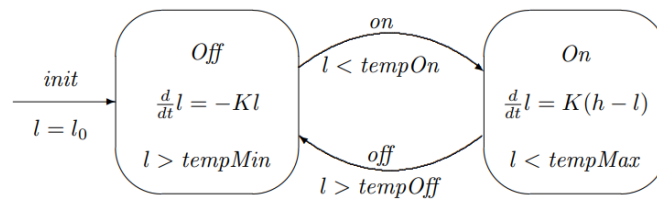


Figure 37: Thermostat hybrid automaton [\[15\]](#)

Consider that the temperature of a room is controlled by a thermostat, which, again, unrealistically, continuously senses the temperature and has the capacity to signal a heater to turn on and off.

When the heater is **Off** the temperature flow is dictated by the differential equation $l' = -K * l$, when **On**, the temperature is changed according to $l' = K(h - l)$ where l is the current temperature of the room, K is a constant determined by the room, and h is a constant that represents the power of the heater. The requirement of this system is to maintain a temperature bellow and above two temperature values, $[tempMin, tempMax]$, the heater can turn itself on or off according to detected minimum and maximum values of temperature denominated **tempOn** and **tempOff**. Some more complex thermostat can be modeled by taking into account delays, the use of several heaters in different rooms, and sometimes instead of a heater is an air conditioner. The previous example can be modeled by the hybrid automata shown in the figure 37 where two locations are considered

Again, in order to model this automaton into Lince, let's analyze it by it's categories using the developed taxonomy, the continuous behavior is once again **linear** and **deterministic** however, while the system in question is deterministic in regards to **switching** 3, the system is non-deterministic in regards to **time** 4, take in close attention, for example, that the triggered action that causes the transition **on**, is the temperature fulfilling the threshold of being bellow **tempOn**, and together with the invariant of the location **Off**, this transition can happen at any given time when the value of temperature is between $[tempMin, tempOn]$. Considering that Lince is not able to model non-deterministic hybrid systems, during the transaction between hybrid automaton into the simulation language of Lince, the thermostat needs to be modeled and transformed into a deterministic system where only one possible output can result from executing it.

There are some approaches that can be taken in order to make this possible, intuitively, the automaton can be changed to show a behavior close to the fluid control in the previous example, this can be done by changing the guard to $l == temp$ and the invariant to $l >= temp$, where $temp \in [tempMin, tempOn]$, making sure that the transaction will be made when the temperature equals to $temp$, however, this solution comes with some problems during the detection of event triggered actions. Instead we'll restrict the behavior of this automata to the previous definition 2.4, and we'll assume that a transaction is made as soon as enabled.

Results and Conclusion

Consider the simple regular expression $R = (ab)^*$ where a is the transition *on* and b is the transition *off*

1. Initial state of our system and some constants.

```
1      temp:=25; k:=0.2 ; h:=100;tempOn:=20; tempOff:=30;  
2      tempMin:=15; tempMax:=35;
```

2. Entering a cycle created by the kleene star [3.2](#)

```
1      while (true) do {
```

3. Compilation of transition *on* with precision of 1 second. [3.4](#)

```
1      temp' = -k * temp until_1 (temp<tempOn);
```

4. Compilation of transition *off* with precision of 1 second.

```
1      temp' = k * (h -temp) until_1 (temp>tempOff);  
2      }
```

At first glance, the user could manually change the constants of **tempOn** and **tempOff** to obtain different simulations of the thermostat, but even then, it wouldn't be a very truthful simulation of the original automaton. Notice that the automaton is a loop between the two locations, the transaction **on** can be made multiple times throughout it's execution, by simple changing **tempOn** to a single value throughout the entire loop the event triggered action would always be limited to that value, while a more broad simulation of the system would have different event triggered conditions throughout it's execution. That is why, of the many possible ways Lince can integrate a non-deterministic behavior to it's language is by providing a function that would return a value between **tempMin** and **tempOn**, the program would execute this function multiple times to generate different event thresholds each iteration throughout the loop. Another way to somewhat add non-determinism to the system would be to randomize the precision of the function **until**.

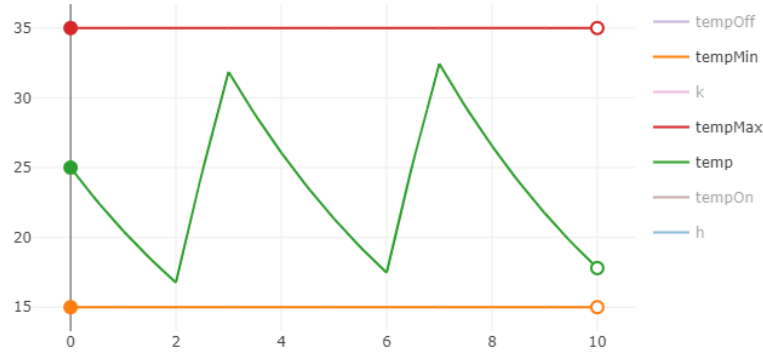


Figure 38: Thermostat Lince Output

In the figure 38 that shows the result of running the thermostat in Lince, it can be observed that the requirement is met and the temperature never surpasses **tempMin** and **tempMax**, however, Lince is only able to simulate the deterministic version of the system 2.4 making this validation only viable for one possible execution of the original automaton, the execution in which transitions are made as soon as enabled, and therefore, this not a true validation for the original automaton, trivially concluding that Lince is limited to the modeling of deterministic behaviors.

5.2.3 Gas Burner

Introduction and Methods

In the last automata examples, the approach to it's compilation in Lince was made by restricting these into a deterministic behavior 2.4. However, for the next example, we'll use the other method of compilation 3.3.1, this method will consist on showing the output of at least two possible executions of the automata using delays to it's transitions.

Let's consider one of the studied examples of hybrid systems, a gas burner [15], the following leaking gas burner consists of a simple valve, which controls a gas supply to a burner, when the valve is open the gas is flowing at a certain rate, when closed, there's no gas leaking. Some requirements to validate in these types of systems are:

- The gas shall not continuously leak for more then a specific amount of time (leaking time).
- The accumulated gas burned can't surpass more then a specific amount (accumulated leakage) at

any any interval of 60 seconds.

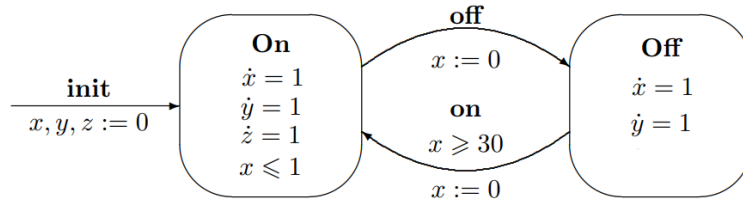


Figure 39: Leaking Gas Burner Hybrid Automaton

Consider the two-state model showed in the figure 39, in this model, the clock x records the time spent in the current location, z records the amount of accumulated gas leaked, and y records the total elapsed time of the system. The main problem of this automata is, just like the thermostat, the fact that it's non-deterministic in time. In order to model in Lince executions of this system, we'll have to take into account possible delays of it's transitions.

Results and Conclusions

We'll take some creative decisions and restrict the type of possible outputs this automata can present by limiting the transition guards. The transitions *off*, that could be done at any instant in which x is between 0 and 1 will now have a guard for when x is 0.2 or 0.5 or 1, meanwhile, the transition *on* will now have a guard of $x == 30$, turning the previous automaton into the following one:

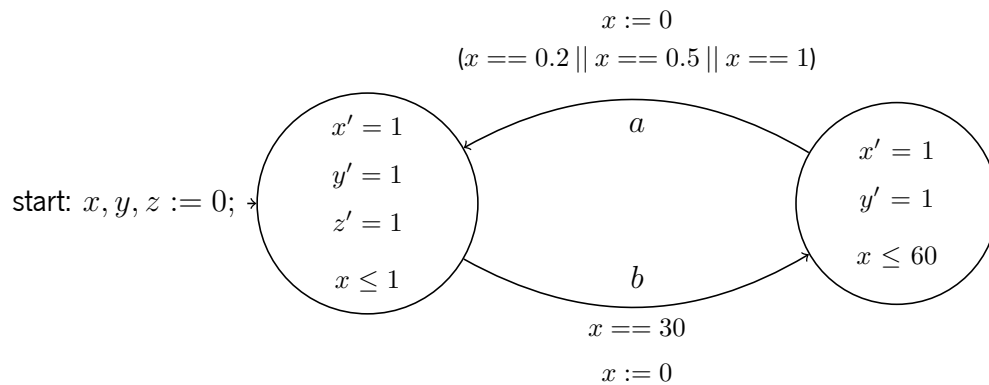


Figure 40: Gas Burner Modified Automaton

Using the previous automaton 40 and the compiling rules of 3.3.1 encoded in haskell A.1, it's now possible to define a string that will represent one possible execution of this gas burner, Observe a simplified

output for the compilation of the string "*a.ba*" when used to formulate a Lince program following the methodology and using precision of 0.1.

1. Initial state of our system, all variables start at zero, a new variable named *deadlock* is introduced in case an invariant is broken.

```
1  x:=0;y:=0;z:=0;
2  deadlock:=0;
```

2. Starts with compiling *a.ba* with rule of *delay* 3.8, continuously evolve until the guard of *a* is detected or the invariant. If guard was detected then delay the transition for 0.1 and compile the same string without the delay symbol, otherwise enters a deadlock because of the invariant.

```
1  x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x<=1));
2  if (x==0.2 || x==0.5 || x==1) then {
3      x'=1,y'=1,z'=1 for 0.1; Compile ("abab")}
4  else deadlock:=1;
```

3. Compilation of *aba* using the rule of *head* 3.7, continuously evolve until guard of *a* is detected or invariant, if guard is detected then precede, otherwise enters a deadlock.

```
1  x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x<=1));
2  if (x==0.2 || x==0.5 || x==1) then { Compile ("bab")} else
    deadlock:=1;
3
```

4. Follows the rest of the rules 3.3.1 to compile the rest of the string.

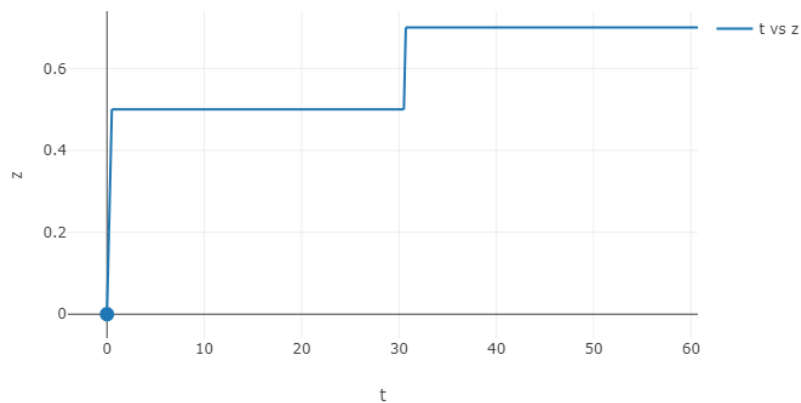


Figure 41: Gas Burner Lince Output

In the end, the Lince output from running this program is as shown 41, this presents one possible executions of the system, the objective being to run different strings to obtain different outputs, and observe a more complete valid simulation of the gas burner automaton 40. In this case, it can be observed that the transition is delayed as expected, being that the gas increases by 0.5 in the first transition, and only 0.2 in the second transition. A totally new simulation can be observed, for example, from executing the program created by string "aba.ba..b" 42.

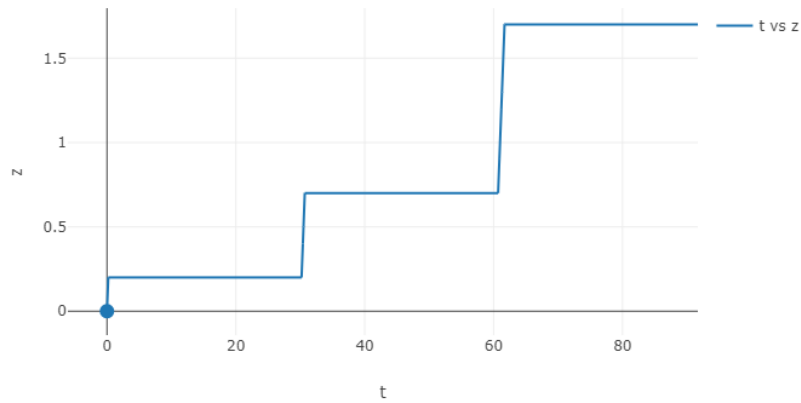


Figure 42: Gas Burner Second Lince Output

5.3 2D Movement

5.3.1 Dubins Path

Introduction and Methods

One of the categories of hybrid systems we intend to discuss more deeply is those whose continuous behavior are non-linear. Remember the previous example of Dubins path 24 where an object movement is described by it's constant velocity and the direction of that velocity. The previous presented non-linear hybrid system had the following continuous dynamics:

$$\begin{aligned} x' &= v \cos \vartheta & y' &= v \sin \vartheta & \vartheta' &= \omega \end{aligned} \quad (5.1)$$

- Where ϑ is the orientation of the plane, an angle that represents it's direction in a 2D space.

- ω is the angular velocity of the plane, how fast it can turn and it also represents the flow of the plane's orientation ϑ .
- x and y are the position in space of the plane.
- v is the plane modular velocity that remains constant during the whole movement.
- The equations that dictate the flow of x and y are the velocities of the plane re-parted for each axis, these velocities are calculated by it's modular velocity and it's angle using simple trigonometry.

Because this system is non-linear, it's simulation in Lince becomes very difficult, although some improvements were already been made to remedy such [33], however, we pretend to discuss alternatives from which such system can be implemented in Lince, even if not exactly correct. One of the ways to achieve that is **temporal discretization**, a very used method in the fields of mathematics.

Results and Conclusion

In this case, we divide the continuous behavior of the dubins path, into very small time increments, having the opportunity to represent the continuous dynamics of the system into linear equations. . Consider the next example:

1. Initial state of our system, the plane starts with angle a of 0, it's angular velocity is considered to be a tenth of π , and the constant velocity during the movement is $1m/s$.

```
1    px:=0; py:=0; vx:=0; vy:=0;
2    a:=0; w:=pi()/10; v:=1;
```

2. A loop where each iteration will be a small time interval in which to do the continuous behavior of our system.

```
1    while true do {
```

3. Discretely calculate the angle of the plane's direction, for example, using intervals of time by 1 second, we know the angle increases by w each time.

```
1    a:=a+w;
```

4. Discretely calculate the velocity of the plane in both axis using the new angle

```
1    vx:=v*cos(a);
2    vy:=v*sin(a);
```

5. Finally continuously evolve the system for a small amount of time.

```
1     px'=vx, py'=vy for 1;
2 }
```

As it can be observed [43](#), the plane makes the exact type of movement intended in the Dubins path. Temporal discretionary to convert non-linear system manually is not a perfect methodology, there's a precision associated and a need to calculate the best value for the time intervals, this conversion could lead to some errors if not given the special attention they need.

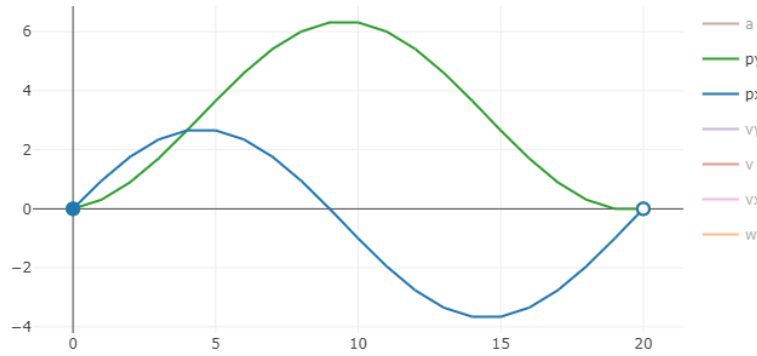


Figure 43: Discrete Self Driving Lince Output

Introduction and Methods

While a temporal discretization of our system is a possible way to deal with its non-linear behavior, the right way would be to turn a non-linear continuous dynamic into its linear equivalent, and fortunately, this is usually possible by using mathematical knowledge and understanding about the system, take the previous example, the process involves adding a new level of variables to the equation, (v_x and v_y), the velocities in both axis from which x and y flow, and calculate which equations dictate the continuous behavior of these new variables. This process was better introduced and explained by Andre Platez [\[28\]](#) as Axiomatic Ghosts. The calculation of the equations that bounds v_x and v_y in the continuous space are as following:

$$\begin{aligned}
x' &= v_x \wedge y' = v_y \rightarrow \\
\{x' &= v \cos \vartheta \wedge y' = v \sin \vartheta \text{ by 5.1}\} \\
v'_x &= (v \cos \vartheta)' \wedge v'_y = (v \sin \vartheta)' \rightarrow \\
\{\text{product rule: } (u \cdot v)' &= u' \cdot v + u \cdot v'\} \\
v'_x &= v' \cos \vartheta + v(\cos \vartheta)' \wedge v'_y = v' \sin \vartheta + v(\sin \vartheta)' \rightarrow \\
\{\text{constant linear velocity: } v' &= 0\} \\
v'_x &= v(\cos \vartheta)' \wedge v'_y = v(\sin \vartheta)' \rightarrow \\
\{\text{chain rule: } (f \circ g)' &= (f' \circ g) \cdot g'\} \\
v'_x &= v(\cos \vartheta)' \vartheta' \wedge v'_y = v(\sin \vartheta)' \vartheta' \rightarrow \\
\{(\cos \vartheta)' &= -\sin \vartheta \wedge (\sin \vartheta)' = \cos \vartheta\} \\
v'_x &= -(v \sin \vartheta) \vartheta' \wedge v'_y = (v \cos \vartheta) \vartheta' \rightarrow \\
\{v \cos \vartheta &= v_x \wedge v \sin \vartheta = v_y \wedge \vartheta' = \omega \text{ by 5.1}\} \\
v'_x &= -v_y \omega \wedge v'_y = v_x \omega
\end{aligned}$$

The calculation can be a bit complex, but extremely useful, because in the end, a system of linear equations can be used to represent the previous Dubins path:

$$x' = v_x \quad y' = v_y \quad v'_x = -v_y \omega \quad v'_y = v_x \omega \quad (5.2)$$

Results and Conclusion

1. Initial state of the system, equal to previously, because the orientation starts at 0 and the modular velocity is 1, then the initial velocity vector is fully horizontal where $v_x = 1 * \cos 0 = 1$ and $v_y = 0 * \sin 0 = 0$

```
1    px:=0; py:=0; vx:=1; vy:=0;w:=pi()/10;
```

2. Continuously evolve the system for a determined amount of time:

```
1    px'=vx, py'=vy, vx'=-w * vy, vy'=w*v x for 20;
2
```

The output in Lince of this program 44 is similar, arguably much better then the one obtained by temporal discretization, proving it's correctness. In the field of hybrid systems, the Dubins path is an

excellent way to model circulatory movements, mostly used in robotics and autonomous driving, a very useful and prominent field, however, it's important to understand why such system is usually represented by the equation given 5.2, specially when the tool doesn't have the ability to simulate or validate non-linear differential equations.

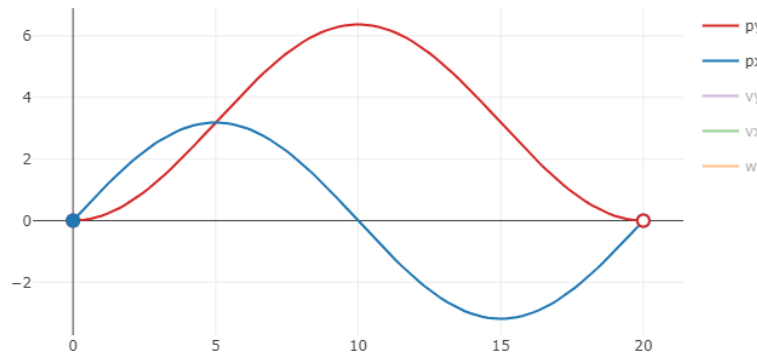


Figure 44: Linear Dubins Path Lince Output

5.3.2 Wall Collision

Introduction and Methods

Another type of hybrid systems we pretend to explore more deeply in Lince is event triggered actions, in the previous example of the bouncing ball 32, conclusions were taken about the limitations of Lince while detecting event triggered actions. In short, Lince is not able to safely detect and locate event triggered actions and the solution employed is, as usual in these cases, to model these extremely precise events into time triggered actions. This conversion is not completely safe and can result in some problems like the ones seen with the bouncing ball, it introduces a certain precision issue that the user needs to take into account. Take the following example of an object moving in a 2D space, colliding with a wall represented by a line in the form of $y = m \cdot x + b$ and bouncing back.

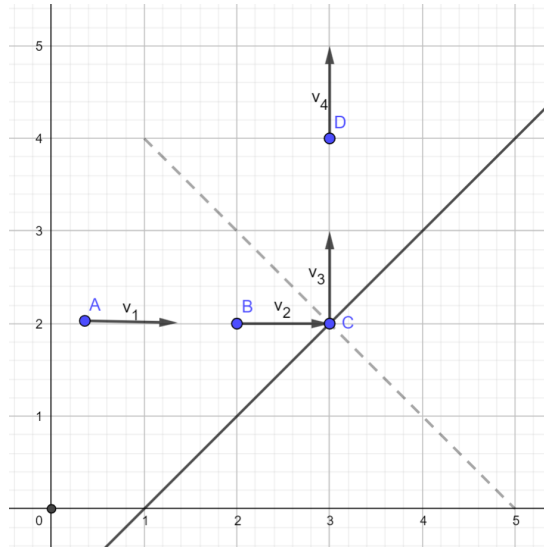


Figure 45: Wall Collision

Just like the case of the bouncing ball, the impact between the object and the wall is given when the coordinates of the object coincides with a point that belongs to the line, satisfying $y = m \cdot x + b$, where x and y are the coordinates of the object, m is the slope of the line, and b the interception of the line with axis y . Because of the difficulties in detecting this equality in a continuous dynamic, the exact time of impact can not be located in Lince, remember that, it's detection is realized by the the execution of periodic and small time triggered actions resulting on continuously evolving the system, step by step, trying to detect the condition at each discrete action. Therefore, the object hitting the wall needs to be turned into a more boarder condition, like the object crossing the wall, just like in the bouncing ball 4.3.2. The main problem with this solutions is that we need to take into account what's the initial position of the object to calculate what condition represents crossing the wall, imagine that the object is initially above the line, in this case, the event that triggers the collision becomes $y < m \cdot x + b$, while if the objects starts bellow the line, the condition of detection becomes $y > m \cdot x + b$. There is two main ways to model the detection of this condition in Lince using the function **until**: the first one is to not offer any precision and Lince will use inner algorithms to calculate the closest time of the event with maximum precision, but unfortunately, only works when the condition of the event isn't too complex. The second way is to offer a precision in the form, for example, of **until_0.1** (time triggered actions each 0.1 seconds).

Results and Conclusion

1. The system starts at position $p = (0, 3)$ and velocity $v = (2, 0)$.

```
1      x:=0; y:=3; vx:=2; vy:=0; vtemp:=0;
```

2. The object continuously evolve in small increments of time until it detects the condition of the object crossing the line, the line is represented by equation $y = 2 \cdot x$. Lince uses inner algorithms to calculate the best precision, trying to detect the condition as soon as possible.

```
1      x'=vx, y'=vy until (y<((2*x)+0));
```

3. Once upon impact, the velocity vector of the object v is reflected v_r . To calculate the phenomena, we use the trivial vector reflection formula where the normal vector is perpendicular to the slope of the line $n = (-m, 1)$ and $\hat{n}$ is the unitary vector where $\hat{n} = \frac{n}{||n||}$

$$v_r = v - 2(v \cdot \hat{n}) \cdot \hat{n}$$

```
1      vtemp:=vx;
2      vx := vx - (2*m*(vx*m - vy))/((m^2)+1);
3      vy := vy + (2*(vtemp*m - vy))/((m^2)+1);
```

4. Finally, the object continuously evolve after it's velocity vector was reflected.

```
1      x'=vx, y'=vy for 2;
```

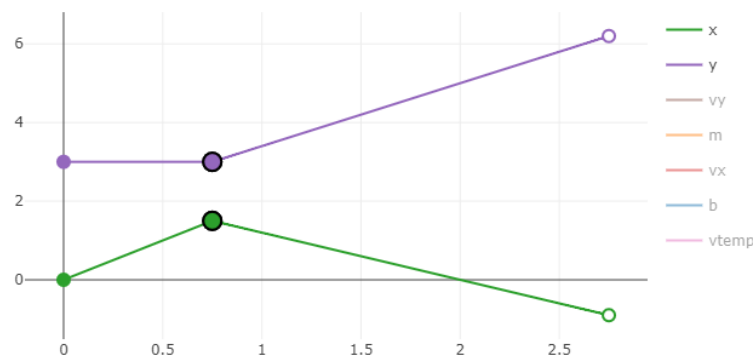


Figure 46: Wall Collision Lince Output

Event triggered actions present a clear limitation in the capacities of Lince, forcing the user to always think in a perspective of time triggered actions, the more precision it's required, the more actions will happen in the same amount of time, offering less computational efficiency, if the precision is not enough,

errors will be made and, sometimes, depending on the system, events won't be even detected. It's important, while using Lince, for the user to evaluate the complexities of his hybrid system and take conclusions in which if it is safe to model event triggered actions into time triggered actions, the more complex the hybrid system is, the more errors can stack up on each other.

5.4 Autonomous Driving

Autonomous driving is a very important section of hybrid systems for their extreme application in robotics, aiming to make every type of vehicle move in space in an efficient matter, following a specific route, choosing between routes, following a leader, changing lanes, the examples are numerous. We can apply these, for example, in search and rescue missions, warehouse automation, agriculture, factories, self driving cars, between many others. We'll dive in into some examples of autonomous driving and how Lince can be useful in the validation of such algorithms.

5.4.1 Automatic Emergency Breaking

Introduction and Methods

The automatic emergency breaking (AEB) is an autonomous driving mechanism that aims to make urgent decisions in order to stop collisions with any object by automatically and drastically reducing the velocity of the vehicle , it's a safety mechanism that is extremely useful to avoid tragic accidents and relies on various sensors, such as cameras, radars, ultrasound or infrared together with it's fast computational calculations in order to make decisions faster than what a human would be able to.

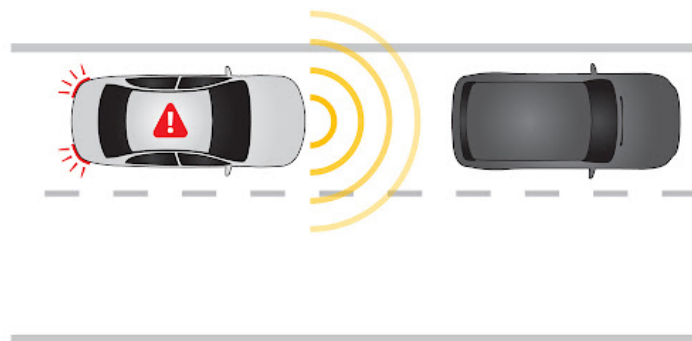


Figure 47: Automatic Emergency Braking

In an attempt to validate the capacities and usefulness of Lince and test a modeling approach for an automatic breaking system in several scenarios, we'll present a program that aims to represent this

particular useful hybrid system. Imagine the following situation of a car driving aggressively, meaning it's always accelerating, and an emergency breaking system that is doing periodic decisions about if it should force a deceleration of the vehicle in order to stop a collision. Consider the following variables:

- p_v and p_o are the positions of the vehicle and the obstacle respectively.
- v_v and v_o are the velocities of the vehicle and the obstacle respectively.
- t_r is the periodic interval in which the automatic break makes a decision.
- a_A and a_B are the acceleration values of the vehicle for acceleration and breaking respectively, while a_o is the acceleration of the obstacle.

The main difficulty of designing an automatic emergency breaking system is, of course, calculating the condition that satisfies a forceful automatic breaking of the vehicle. This condition is exactly why tools like Lince may be proven useful to understand what's the best approach and to validate that same approach in it's safety using several scenarios. Consider the next system of equations that are necessary in order to calculate if an urgent breaking is necessary:

1. The easiest and obvious approach is to predict the position of both entities (vehicle and obstacle) after a periodic time t_r , remember that the vehicle is accelerating so it's acceleration is a_A .

$$p_v(t_r) = p_v + v_v \cdot t_r + \frac{1}{2} \cdot a_A \cdot t_r^2 \quad (5.3)$$

$$p_o(t_r) = p_o + v_o \cdot t_r + \frac{1}{2} \cdot a_o \cdot t_r^2 \quad (5.4)$$

2. With both future position calculated 5.3 and 5.4, the first condition verifies if the vehicle can surpass the obstacle in the next t_r seconds, if yes, then a collision was made.

$$p_v(tr) < p_o(tr) \quad (5.5)$$

3. However, the first condition 5.5 is not enough, the condition just states that the vehicle won't collide in the next t_r seconds, what it doesn't do is to take into account the fact the vehicle can reach velocities that will make impossible to break in time to stop a collision in the future. Therefore consider the equations to calculate the velocity of both entities after a periodic time t_r

$$v_v(t_r) = v_v + a_A \cdot t \quad (5.6)$$

$$v_o(t_r) = v_o + a_o \cdot t \quad (5.7)$$

- Next, consider the movement equations of both entities that calculates their position for any t that occurs after the periodic time has passed. In this equation the vehicle has acceleration of a_T meaning is breaking.

$$p_v(t) = p_v(t_r) + v_v(t_r) \cdot t + \frac{1}{2} \cdot a_B \cdot t^2 \quad (5.8)$$

$$p_o(t) = p_o(t_r) + v_o(t_r) \cdot t + \frac{1}{2} \cdot a_b \cdot t^2 \quad (5.9)$$

- If the position of both entities are equal for any value of t means that the vehicle can't stop in time, so there's a possibility of a collision. The next equation tries to calculate exactly that.

$$p_v(t) = p_o(t) \rightarrow \quad (5.10)$$

$$\frac{1}{2} \cdot (a_B - a_o) \cdot t^2 + (v_v(t_r) - v_o(t_r)) \cdot t + (p_v(t_r) - p_o(t_r)) = 0$$

- Finally, the second condition that verifies if the vehicle can stop in time is calculated by using the discriminatory of the quadratic equation above 5.10 to discover if the above has any solution.

$$a \cdot t^2 + b \cdot t + c = 0 \quad (5.11)$$

$$b^2 - 4ac < 0 \quad (5.12)$$

Results and Conclusion

The first condition 5.5 and second condition 5.10 together make a valid methodology to detect when the automatic emergency breaking system should make a decision, such methodologies can become very complex, very fast, and that is why it's important to have hybrid system tools at hand to be able to simulate these, validate them, and iterate over them, for example, in a simple thought experiment it may have seemed that condition one was enough to satisfy the prevention of a collision, something that it's not true. Such methodologies can then be programmed and simulated in Lince:

- Initial state of the system, we consider the obstacle to not be moving so it's velocity is zero and acceleration not even considered.

```

1 // Initial Position and velocity of vehicle.
2 p:=0; v:=20;
3 // Initial Postion, velocity of obstacle.
4 pl:=50; vl:=0;
5 // Acceleration and Breaking values of vehicle.
6 aT:=-9.8; aA:=6;
7 // Periodic decision time for the system.
8 sampling_time:=0.1;
```

2. A cycle while that loops as long as the vehicle is moving

```
1   while v>0 do{
```

3. Verification of the first 5.5 and second condition 5.10

```
1   if ((p + v*sampling_time + aA/2*sampling_time^2 < pl+v1*
      sampling_time ) &&
2   (((v+aA*sampling_time-v1)^2 - 4*(p+v*sampling_time+aA/2*(
      sampling_time^2)-(pl +v1*sampling_time))*(aT/2)) < 0))
```

4. If a collision is not possible then don't intervene and the vehicle keeps accelerating.

```
1       then p'=v,v'=aA for sampling_time;
```

5. If a collision is detected then intervene and start breaking until the vehicle stops.

```
1   else {
2       p'=v,v'=aT until_0.01 (v<=0);}
3   }
```

In the end, since Lince can simulate 48 and validate the automatic emergency breaking system above in the specific scenario that it was programmed, the user can try different scenarios varying in velocity, sampling times, and distance to the obstacle, even using bigger sampling times to simulate the reaction of human beings against the autonomous one. The automatic breaking system is a perfect example that highlights the usefulness of hybrid system tools like Lince, therefore being why it's given such importance in this dissertation.

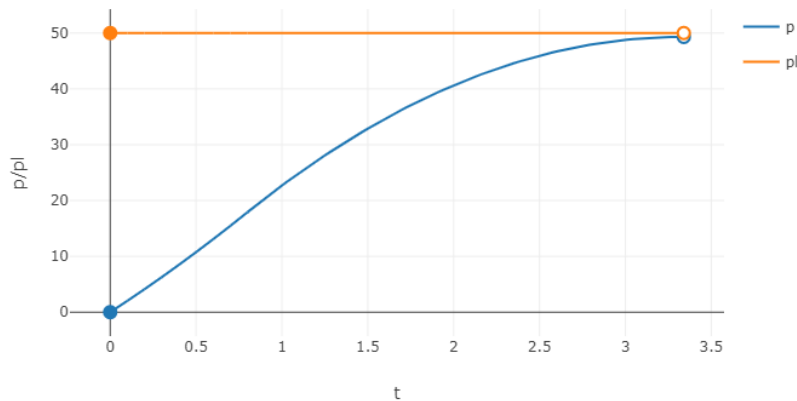


Figure 48: Automatic Emergency Breaking - Lince Output

5.4.2 Adaptive Cruise Control

Introduction and Methods

An adaptive cruise control is a decision making mechanism in autonomous driving that aims to maintain a vehicle velocity in order to always establish a safe distance from another vehicle that is driving in the front, making sure that in a urgent situation the adaptive cruise control can realize a safe and reliable stop to avoid collisions. An adaptive cruise control relies once again on several radars to obtain information around the vehicle and aims to improve safety, reduce driver's fatigue and help on long distance travelings.

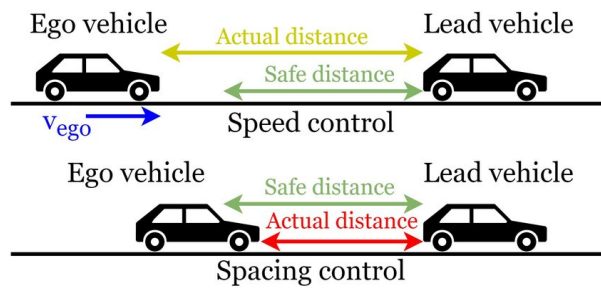


Figure 49: Adaptive Cruise Control [20]

In order to test how an adaptive cruise mechanism can be useful in several scenarios, we once again test the methodology developed in Lince environment to guarantee its validation. In this scenario, a vehicle equipped with an adaptive cruise control is driving before a leader vehicle that is moving with a constant velocity, the objective of the mechanism is for the cruise control to always maintain a minimum safety distance from the car in front, in order to do that, the adaptive cruise control has the power to decelerate adjusting its velocity to the same values as the leader.

Results and Conclusion

1. The developed programs starts by declaring the initial position and velocity of both cars, being p and v the position and velocity of the car equipped with an adaptive cruise control, p_l and v_l being the position and velocity of the leading vehicle. We consider, initially, the velocity of the leading vehicle (10) to be inferior to the vehicle with the cruise control (20).

```
1  p := 0; v := 20;  
2  p_l := 30; v_l := 10;
```

2. It's declared the breaking capacities of the vehicle with the adaptive cruise aT , the leading vehicle is on constant velocity, then its acceleration is zero $aL = 0$.

```
1      aT:= -8; aL:= 0;
```

3. It is necessary to declare a periodic time in which the adaptive cruise control will make a decision, and it's also necessary to establish a desired safety distance from the leading vehicle.

```
1      safety_distance := 10;
2      sampling_time:= 0.1;
```

4. The programs enters a loop from which to make periodic decisions.

```
1      while ( true ) do {
```

5. Following the equations similar to 5.12 and 5.5, the adaptive cruise control verifies if the vehicle can cause a collision with the position of the vehicle in the front minus the safety security, in other words, this condition verifies if the vehicle will surpass the safety distance.

```
1      if ((p + v*sampling_time < ( pl - safety_distance ) + vl *
          sampling_time + aL/2*sampling_time^2)
2      && ( ( ( v - vl + ( -aL)*sampling_time )^2 - 4*(p - ( pl -
          safety_distance ) + (v - vl )*sampling_time +
3      ( -aL) /2*sampling_time^2) *(aT-aL)/2 ) <0))
```

6. If the safety distance will not be trespassed then maintain a constant velocity $v' = 0$

```
1      then p'=v , v'=0 , pl'=vl , vl' =aL for sampling_time;
```

7. If the safety distance will be trespassed then the adaptive cruise will decide to perform a deceleration.

```
1      else p'=v , v'=aT, pl'=vl , vl'=aL for sampling_time;
2      }
```

The output of running this program can be observed in the figure 50, Lince can be used to validate the methodology of the adaptive cruise control in this scenario, however, it's capabilities are full seen when different scenarios are tested, for example, the initial positions of both vehicles could be changes as in their velocities, conclusions could be taken about the power necessary in the breaks of the vehicle with the adaptive cruise control and also how fast it's sensor capabilities should be to cover normal speeds and scenarios of real life situations, furthermore, safety distances could be calculated depending on the velocity of both cars and situations in which the leading car is accelerating or decelerating could also be tested. These are all possibilities that Lince allows to take conclusions from and give information about

the materials, variables and methodologies to use while developing an adaptive cruise control, enforcing the idea that hybrid system simulation tools are a must in the development of these.

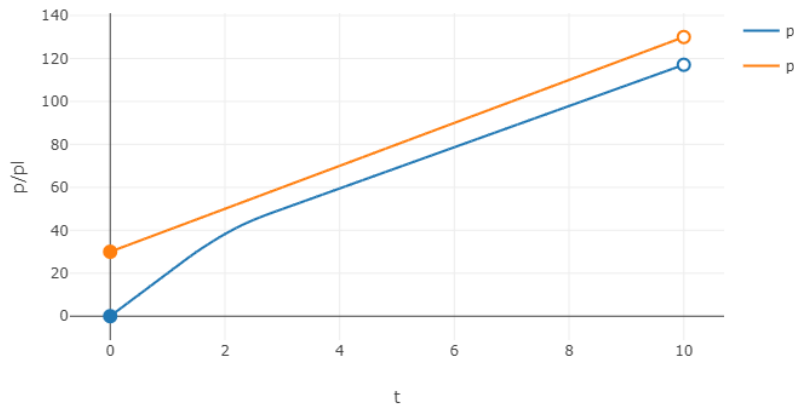


Figure 50: Adaptive Cruise Control - Lince Output

5.4.3 Missile and Target

Introduction and Methods

The Missile and Target system is a decision making problem in autonomous driving that aims to change the direction of a missile in a way that eventually reaches a position from which a target is occupying, of course, such directional problem could be derived to normal vehicles or any other robotics movement. We'll be exploring a very simple missile and target problem with the following characteristics:

- The missile will have a constant velocity.
- The missile will be moving in a 2D space using the linear equations of the Dubins path previously defined [5.2](#).
- The decision mechanism will have the opportunity to perform rotations in both directions with a predetermined angular velocity.
- The target will be in a stationary position.

With all restrictions in place to facilitate a missile and target system easy to explain and develop a methodology was created for the system to make the decision between turning to the right, left, or not turning. Mathematically, the problem involves comparing the direction of the velocity vector of the missile with the

desired one, the objective of the system is for the missile velocity to always point to the target. So there's a need to compare the vector of the missile velocity to the desired vector that connects the target to the missile, this desired vector is calculate by the following equation:

$$\vec{P}_{MT} = \vec{P}_T - \vec{P}_M \quad (5.13)$$

In equation 5.13, $\vec{P}_T$ is the position of the target and $\vec{P}_M$ is the position of the missile. After getting both the desired orientation for the velocity vector in $\vec{P}_{MT}$ and the actual velocity vector $\vec{V}_M$, there's a group of vector mathematical functions that can be applied to compare both vectors and make the desired decisions, for example, the cross product of both vectors gives information about which direction to turn.

$$\begin{cases} \vec{V}_M \times \vec{P}_{MT} \geq 0, & \text{turn right;} \\ \vec{V}_M \times \vec{P}_{MT} < 0, & \text{turn left.} \end{cases} \quad (5.14)$$

Remark: Notice that the cross product of two vectors is not commutative, so $V_M \times \vec{P}_{MT} \neq \vec{P}_{MT} \times V_M$. Deciding which direction to turn the velocity vector to coincide with the target position is opposite from deciding which direction to turn the the vector of the target position to coincide with the velocity vector.

So the decision between turning left or right can be made following these calculations 5.14, however, decision between not turning at all in case the angle is negligible could also be made by calculating the angle between these two vectors with the following equation

$$\alpha = \arccos \left(\frac{\vec{V}_M \cdot \vec{P}_{MT}}{|\vec{V}_M| |\vec{P}_{MT}|} \right) \quad (5.15)$$

In the end, the desired behavior follows the one represented in figure 51, at each period of time, the control mechanism of the missile will compare the direction of it's velocity vector, to the position of the target, and turn left or right accordingly for an amount of time until it makes the decision again.

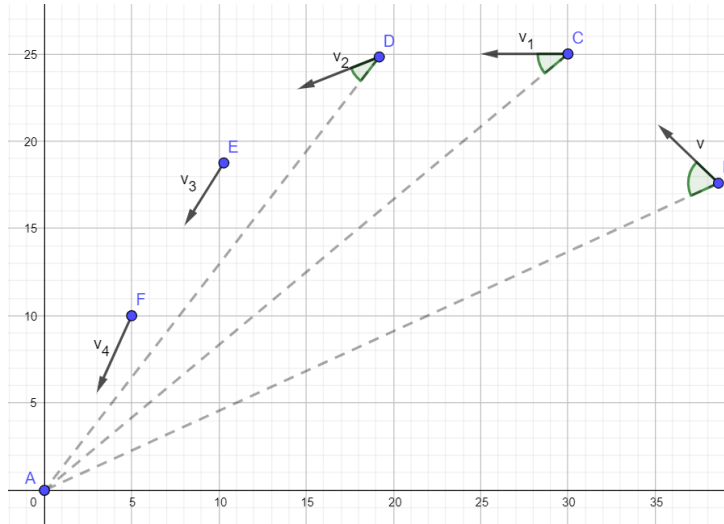


Figure 51: Missile and Target

Results and Conclusion

1. To encode this simple target and missile system, first we declare the initial position of both the missile and the target, the velocity vector of the target is also declared.

```
1    px:=15; py:=10; vx:=-3; vy:=0;
2    pxf:= 0; pyf:=0;
```

2. Next, some important variables are also declared, the missile turning capabilities in the form of it's angular velocity for both sides, the periodic time from which the system makes a decision, and the negligible angle from which to decide if the missile should turn or not.

```
1    va:=(1/10)*pi();
2    time:=0.5; i_angle:=(pi()/20);
```

3. Followed by some auxiliary variables to make the calculations above, including the angle between both vectors, their cross and dot products.

```
1    vxf:= 0; vyf:=0;
2    angle:=0; dot_product:=0; cross_product:=0; rotation:=0; aw:=0;
```

4. The system enters a loop and starts by calculating the vector $\vec{P}_{MT}$ [5.13](#)

```
1    while true do {
2        vxf:=pxf-px; vyf:=pyf-py;
```

5. After that the program makes all calculations between the two vectors described above [5.14](#) and [5.15](#)

```

1 dot_product := (vx * vxf) + (vy * vyf);
2 cross_product := (vx * vyf) - (vy * vxf);
3 angle := arccos(dot_product / ((sqrt (vx^2 + vy^2)) * (sqrt (vxf^2
+ vyf^2))));

```

6. The missile makes a decision about if it should rotate to the left or the right according to the crossing product of the vectors [5.14](#)

```

1 if cross_product < 0 then rotation := -1; else rotation := 1;

```

7. If the angle between the vectors is negligible then the system decides to not turn at all deciding for a angular velocity of zero, if the angle is not negligible then decide if the angular velocity is positive or negative according to the decision made in the previous point.

```

1 if (angle < i_angle)
2 then aw:=0;
3 else aw := rotation * va;

```

8. Lastly, the system evolves for a periodic time according to the Dubins path [5.2](#), from which it makes the decision again.

```

1 px'=vx, py'=vy, vx' = - aw * vy, vy' = aw * vx for time;

```

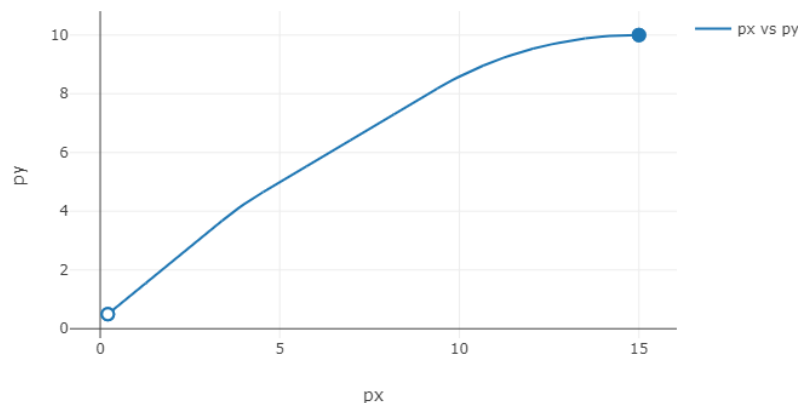


Figure 52: Missile and Target - Lince Output

In the the figure 52, we can observe Lince's output from running the program above, representing the path of the missile, it can be observed the validation of the methodology for the missile clearly tries to pursuit the target in the position of $(0, 0)$ making the right turning choices. The objective of this exercise was to once again to prove Lince's capabilities in simulating autonomous driving hybrid systems, however, this simple system could be expanded to model other and more complex scenarios, specially the ones from which the target is moving, scenarios from which the missile tries to predict the target's path, scenarios in which the missile needs to avoid other missiles, or even, in the future, a distributed system from which the target also makes decisions to avoid the collision with the missile. Simpler then this, it's also more realistic that the missile has different angular velocities, travels in a 3D space, or starts in different initial positions and velocities. Once again, Lince allows to take found conclusions on the best specifications the missile should have to efficiently pursuit the target without the need to realize experiments in real life. Conclusions can be taken about the necessary turning capabilities of the missile taken into account it's traveling velocity, the necessary rapidness of the sensors and decision making skills, how much should the negligible angle should be in order to not make unnecessary turnings among other things.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

Hybrid systems are each day, more and more integral to the modern life-style of our society, these systems, capable of modeling computational algorithms with characteristics of the physical world are pivotal in industries ranging from automotive to aerospace, healthcare, and manufacturing, all of them requiring computational control mechanisms of some kind and their respective safety validation and simulation to ensure their correctness, efficient development and implementation that covers a large variety of problems in all sectors of our society. Hybrid systems are mathematical models, characterized by their blend of continuous and discrete dynamics. By modeling both continuous changes and discrete events, using the right tools, hybrid systems allow for a comprehensive modeling, validation and simulation of real life mechanisms that interact with the physical world, making the study of these essential in the development of this technology. Lince is an emerging tool in the area, designed to provide lightweight simulation for hybrid systems through a specialized programming language. Lince aims to make accessible for users the validation and optimization of systems efficiently in a way not explored in the market before.

During the realization of this thesis, a taxonomy of hybrid systems was created to better understand and categorize its capabilities and limitations. This taxonomy has shown to play a crucial role in defining how Lince interacts with different types of hybrid systems. Throughout the study, this taxonomy revealed that Lince excels in simulating several behaviours of hybrid system like linear continuous dynamics, determinism, and time triggered actions. However, it faces huge limitations when dealing with non-linear dynamics, non-deterministic or stochastic systems, distributed systems, and event triggered actions. To simulate non-linear dynamics, users must first simplify them into a linear continuous dynamic. Similarly, non-deterministic systems are impossible to simulate and can only be converted into a possible and very limited version of it's deterministic equivalent, on the other hand, event triggered actions can become a huge error inducing mechanism in Lince being that they are difficult to detect. The taxonomy developed

during this study has significantly improved the process of converting various hybrid systems into Lince's programming language, giving a clear guideline of mental steps the user should do before starting to implement on Lince, specially anticipate the limitations of the tool.

By categorizing systems according to the taxonomy and understanding how Lince interacts with each category, users can more effectively choose the best modeling approach. This helps identify potential errors and limitations in the system before programming begins, streamlining the development process. In this study, a research was also explored on a systematic conversion of commonly used automata directly into Lince, while this has been proved incomplete until further upgrades, it was a great thought experiment understanding the behaviors of hybrid automata and hybrid systems in general and offering a clear path of possible improvements to the tool in the future.

Given the current limitations of Lince, the taxonomy points to clear areas of improvement, as it was suggested in a theoretical new expanded syntax [8](#) for plausible future implementations that will turn possible the simulation of new types of hybrid systems previously impossible. like distributed, non-deterministic or stochastic systems. Furthermore other possible changes could include better detection and handling of event triggered actions and non-linear continuous dynamics. These improvements would significantly expand the tool's capabilities, making it applicable to a broader range of complex systems.

In this dissertation, several examples of hybrid systems were introduced [9](#) and simulated in Lince, this array of varied examples could also serve as a bestiary of very simple hybrid systems, all presenting distinct behaviors, that together, could encompass all different identified categories of hybrid systems and facilitate the assessment and experimentation of other hybrid system tools, in this respect, we could observe Lince capabilities while simulating mostly movement, in examples like the automatic breaking system and the adaptive cruise control, while others examples like the light switch and the thermostat showed pretty obviously that Lince was not able to deal with several behaviors like distributed and non-deterministic systems.

Despite its limitations, Lince has proven to be a valuable tool for lightweight simulations of hybrid systems. It provides a fast and easy way to validate simpler hybrid systems and optimize algorithms that would otherwise be challenging and time consuming to test in real-world environments, Lince allows the user to test different control algorithms and optimize their behavior and values, to quickly realize the properties their hybrid systems should contain, before making monetary decisions and waste precious resources. By offering a simplified approach to hybrid system simulation, Lince enables users to efficiently explore and refine their designs, making it a useful resource for researchers and engineers working in the field.

6.2 Future Work

During the execution of this dissertation potential improvements were also identified, and it's important to list all of these in a concise matter to distinguish a clear path for the future of this tool

- **Improvement of the efficiency of the tool:** Throughout the realization of this project, it was clear that Lince appears to be quite inefficient in speed while compiling and executing it's programs, although a very basic suggestion, this is a clear urgent path of improvement.
- **Improvement of Lince's syntax:** Lince's language is quite basic, for example, there's no possibility of creating functions, no compartmentalization of code, turning it very illegible in scalable or complex systems.
- **Distributed Systems:** Lince syntax language could be expanded to include operators like the parallelism of programs and the use of channels [8](#), creating the possibility of the implementation of a whole new type of hybrid systems. Although inefficient, because distributed systems can be compiled into a single system, there's a possible future case study of doing such in the tool.
- **Non-Deterministic Discrete Dynamics:** Lince syntax language could be expanded to include non-deterministic operators, specially those who relies on random probabilistic choices to present one possible output at each execution [8](#).
- **Non-Deterministic Continuous Dynamics:** Some very specific hybrid systems like the Brownian movement rely on stochastic continuous dynamics, this is a clear field of study that contains immense research sources, and unfortunately, not many hybrid system tools have it implemented, creating a big opportunity for Lince to be one of the few first.
- **Event Triggered Actions:** The detection of conditions during the continuous behavior of the system is a hard mechanism to implement, however, it's a worth improvement considering the immense quantity of hybrid systems that relies on these, if not possible, Lince would benefit immensely if the detection of these conditions would be more precise, even if using external tools to do so.
- **Non Linear Continuous behavior:** Although such improvements were already studied [[33](#)], these are not fully implemented yet and could be more accessible, turning possible the execution of the original Dubins path [5.1](#) for example.

- **More Case Studies:** More complex and varied examples of hybrid systems would greatly show to the market the full capabilities of the tool and also call attention to the usage of it.

	Linearity	Branching Type	Triggered Actions
Projectile Motion 23	Linear	Deterministic	
Dubins Path 24	Non-Linear	Deterministic	
Distributed Breaking 4.2.1	Linear	Distributed	Time Triggered Interrogations
2D Cruise Control 4.2.2	Linear	Deterministic	Time Triggered Interrogations
Breaking Or Accelerating 4.2.3	Linear	Non-Deterministic Switching	Time Triggered Changes
Thermostat 28	Linear	Non-Deterministic Time	
Brownian Movement 29	Linear	Stochastic Continuous dynamic	
Breaking And Accelerating 4.3.1	Linear	Deterministic	Time Triggered Changes
Bouncing Ball 32	Linear	Deterministic	Event Trigger Action
Light Switch 4.3.3	Linear	Distributed	Controlled Triggered Signals
Controlled Velocity 4.3.3	Linear	Distributed	Controlled Triggered Values
Water Tank 35	Linear	Deterministic	Event Triggered Actions
Gas Burner 39	Linear	Non-Deterministic Time	
Wall Collision 45	Linear	Deterministic	Event Triggered Actions
Automatic Breaking System 47	Linear	Deterministic	Time Triggered Interrogations
Adaptive Cruise Control 49	Linear	Deterministic	Time Triggered Interrogations
Missile and Target 51	Linear	Deterministic	Time Triggered Interrogations

Table 9: Examples of hybrid systems and their categories

Bibliography

- [1] Alessandro Abate Aaron D. Ames and Shankar Sastry. *Sufficient Conditions for the Existence of Zeno Behavior*.
- [2] Rajeev Alur and David L. Dill. "A theory of timed automata. In: Theoretical Computer Science 126.2". In: 1994, pp. 183–235.
- [3] *ArcaTools.Org/Lince*. 2023. URL: <http://arca.tools.org/assets/lince.html#fulllince> (visited on 10/03/2024).
- [4] Rick Durrett. *Brownian Motion in Probability: Theory and Examples*. 2019.
- [5] Richard Feynman. *The Brownian Movement, The Feynman Lectures of Physics*. 1964.
- [6] Peter Fritzson. *Principles of Object Oriented Modeling and Simulation with Modelica 3.3: A Cyber-Physical Approach*. John Wiley & Sons, 2014.
- [7] Nathan Fulton et al. "KeYmaera X: An Axiomatic Tactical Theorem Prover for Hybrid Systems". In: In Amy Felty and Aart Middeldorp, 2015.
- [8] Mohamed M. Bayoumi Gino Labinaz and Karren Rudie. M. "Modeling and control of hybrid systems: A survey". In: 1996.
- [9] *gitHub Lince*. 2023. URL: <https://github.com/arcalab/lince> (visited on 10/03/2024).
- [10] Sergey Goncharov, Renato Neves, and José Proença. *Lince: Lightweight Prototyping of Hybrid Programs*.
- [11] Sergey Goncharov, Renato Neves, and Jose Proença. "Implementing hybrid semantics: From functional to imperative In International Colloquium on Theoretical Aspects of Computing". In: Springer, 2020, pp. 262–282.
- [12] Peter Höfner. *Algebraic Calculi for Hybrid Systems*. 2009.
- [13] Jeffrey D. Ullman John E. Hopcroft Rajeev Motwani. *Introduction to Automata Theory, Languages, and Computation*. 2001.
- [14] Harold Klee and Randal Allen. *Simulation of dynamic systems with MATLAB and Simulink*. CRC Press, 2007.
- [15] Thomas Krilavicius. *Bestiarium of hybrid systems*. 2005.
- [16] Thomas Krilavicius. "Hybrid Techniques for Hybrid Systems. PhD thesis, University of Twente". In: (2006).
- [17] Tomas Krilavicius. "Hybrid Techniques for Hybrid Systems". In: 2006.
- [18] Aniello Lampo, Miguel Ángel García March, and Maciej Lewenstein. *Classical Brownian Motion*. 2019.
- [19] Edward A. Lee. "The Past, Present and Future of Cyber-Physical Systems: A Focus on Models". In: (2015).

- [20] Moradi Mehrdad et al. *Exploring Fault Parameter Space Using Reinforcement Learning-based Fault Injection*. 2020.
- [21] George Meyer Michael Heymann Feng Lin and Stefan Resmerita. *Analysis of Zeno Behaviors in Hybrid Systems*.
- [22] Vivek S. Borkar Michael S. Branicky and Sanjoy K. Mitter. "A unified framework for hybrid control: Background, model, and theory." In: Springer, 1994.
- [23] Pieter Mosterman. "An overview of hybrid simulation phenomena and their support by simulation packages." In: Springer, 1999, pp. 165–177.
- [24] Simin Nadjm-Tehrani. *Time-Deterministic Hybrid Transition Systems*.
- [25] Renato Neves. "Hybrid programs. PhD thesis, Minho University". In: (2018).
- [26] JAMES NUTARO. *Towards a theory of super dense time in simulation models*. 2020.
- [27] Zohar Manna Oded Maler and Amir Pnueli. "From Timed to Hybrid Systems. In Proceedings of the Real-Time: Theory in Practice". In: 1992, pp. 447–484.
- [28] André Platzer. *Logical Foundations of Cyber Physical Systems*. Springer Nature Switzerland.
- [29] Andre Platzer. *KeYmaera X Tutorial*. URL: <https://keymaerax.org/Xtutorial.html>.
- [30] Andre Platzer. *Stochastic Differential Dynamic Logic for Stochastic Hybrid Programs*. 2011.
- [31] Luís S. Barbosa Renato Neves. "Languages and Models for Hybrid Automata: A Coalgebraic Perspective". In: (2018).
- [32] Bhargav Kothapalli Shishir Nadubettu Yadukumar and Aaron D. Ames. *Zeno Behavior in Electromechanical Hybrid Systems: From Theory to Experimental Validation*.
- [33] Ricardo da Silva Correia. *Simulation of Hybrid Systems Regulated by Newtonian Mechanics*. 2023.
- [34] Shuling Wang et al. *Formally Verified C Code Generation from Hybrid Communicating Sequential Processes*. 2024.

Part III

Appendices

Chapter A

Automata to Lince

A.1 Haskell Automata Compiling

```
1 import Data.List (nub)
2
3 -----
4 -- Automaton Data Structure and respective functions
5 -----
6
7 -- Location is identified by a Integer (Helps with Kleene algorithm)
8 type Location = Int
9 -- A transition is represented by a Char, a sequence of Labels is a string.
10 -- A string is an execution of the system, as long as it's accepted by RE.
11 type Label = Char
12 -- A transition is a connection between 2 locations.
13 type Link = (Location, Label, Location)
14
15
16 -- Function that given a label, returns the location in which the
17 -- transition leaves from
18 -- Note: Each transition is represented by a label, there can't be 2
19 -- transitions with the same label.
20 getLocationFrom :: Label -> [Link] -> Location
21 getLocationFrom l [] = error "Label Not Found"
22 getLocationFrom a ((l1,b,l2):t) = if (a==b) then l1 else getLocationFrom a
23 t
24
25 -- Function that given a label and a location, returns the next location
26 -- resulting from doing that transition
27 getNextLocation :: Label -> Location -> [Link] -> Location
28 getNextLocation _ l [] = error "Link not found"
29 getNextLocation a loc ((l1,b,l2):t) = if (a==b && loc==l1) then l2 else
30 getNextLocation a loc t
31
32 -- Function that tests if two locations are connected by a transition
33 hasLink :: Location -> Location -> [(Location,Label,Location)] -> Bool
34 hasLink l1 l2 [] = False
35 hasLink l1 l2 ((a,b,c):t) =
36 if (a == l1 && c==l2) then True else hasLink l1 l2 t
37
38 -- Function that calculates all transition sequences up to certain size
39 findLabelSequences :: [(Location, Label, Location)] -> Int -> Location ->
40 [[Label]]
41 findLabelSequences links n start
42 | n == 0 = [[]] -- Base case: return an empty sequence
```

```

37 | otherwise = [[]] ++ -- Include the empty for paths of size less than n
38 [ label : path
39 -- Traverse all links that start at current location
40 | (from, label, to) <- links
41 , from == start
42 -- Find sequences of length n-1 from the next location.
43 , path <- findLabelSequences links (n - 1) to
44 ]
45
46 -- Function that calculates the sequence of locations a sequence of
    transitions passes through.
47 locationFromSequence :: Location -> [Label] -> [Location]
48 locationFromSequence l s = l : (loc l s getLinkList)
49 where
50 loc l [] a = []
51 loc l ['.',''] a = []
52 loc l ('.':t) a = loc l t a
53 loc l (h:t) a = let r = (getNextLocation h l a) in r : loc r t a
54
55 -- -----
56 -- Information about the Automaton Model
57 -- -----
58
59 -- Function that returns transition sequence desired for Lince to simulate
60 -- Note: "a.b" means doing a wait for p time and try to do a again (delay)
61 getSequence :: String
62 getSequence = "ab"
63
64 -- Initial state of the Automaton
65 -- Note: "Example y:=0;t:=0;"
66 getInit :: String
67 getInit = "x:=0;"
68
69 -- Initial Location of the automaton
70 -- A location always has a natural number [1..]
71 getInitLocation :: Int
72 getInitLocation = 1
73
74 -- Final Location of the automaton (For Kleene Algorithm)
75 getFinalLocation :: Int
76 getFinalLocation = 3
77
78 -- How many location the automaton has (For Kleene Algorithm)
79 getLocationSize :: Int
80 getLocationSize = 3
81
82 -- Precision of detection of guards and invariant.
83 -- Note: If precision = "0.1" then it will use until_0.1
84 getPrecision :: String
85 getPrecision = "0.1"
86
87 -- Transitions of the Automaton
88 getLinkList :: [Link]
89 getLinkList = [(1,'a',2),(2,'b',3)]
90
91 -- Function that returns the continous flow of each location
92 -- Example: "x'=1, y'=3" Note: Don't forget the commas dividing each
    variable.

```

```

93 -- Note: If there's none then should return the empty string. Same for all
    the others.
94 getCont :: Location -> String
95 getCont 1 = "x'=1 "
96 getCont 2 = "x'=2 "
97 getCont 3 = "x'=3 "
98 getCont _ = ""
99
100 -- Function that returns the invariant of each location
101 -- Example: x>=5 && y<=10
102 getInv :: Location -> String
103 getInv 1 = "x<=5 "
104 getInv 2 = "x<=5 "
105 getInv 3 = "x<=5 "
106 getInv _ = ""
107
108 -- Function that returns the reset of each transition
109 -- Example x:=0; y:=0; Note: Don't forget the commas dividing and in the
    ending.
110 getAssign :: Label -> String
111 getAssign 'a' = "x:=0; "
112 getAssign 'b' = "x:=0; "
113 getAssign _ = ""
114
115 -- Function that returns the guard of each transition
116 -- Example "x==5 && y>5"
117 getGuard :: Label -> String
118 getGuard 'a' = "x==5 "
119 getGuard 'b' = "x==5 "
120 getGuard _ = ""
121
122 -- -----
123 -- Function that compile Lince Code
124 -- -----
125
126 -- Cases added for : Automaton with no continuous behaviour but guards, to
    make ifs
127 --      Automaton with no continuous behavior nor guards (Classic DFA)
128 --      Of course more cases can be taken into account but they'll be
    ignored for
129 --      simplification purposes.
130
131
132 -- Function that receives an input string and returns Lince code from
    doing that sequence of transitions
133
134 -- Function that compiles the last symbol of a string.
135 compile :: [Label] -> String
136 compile [a] = let l = (getLocationFrom a getLinkList)
137 in getCont l ++ " until_" ++ getPrecision ++ "(" ++ getGuard a ++ ") || !("
    ++ getInv l ++ ")); \n if (" ++ getGuard a ++ ") then " ++ getAssign a
    ++ "else deadlock:=1;\n"
138
139 -- Function that compiles a delay of a transition
140 compile (h:'.':t) = let l = (getLocationFrom h getLinkList)
141 in getCont l ++ " until_" ++ getPrecision ++ "(" ++ getGuard h ++ ") || !("
    ++ getInv l ++ ")); \n if (" ++ getGuard h ++ ") then { \n" ++ getCont l
    ++ " for " ++ getPrecision ++ ";" ++ (compile (h:t)) ++ " } else deadlock

```

```

:=1;\n"
142
143 -- Function that compiles the head symbol of a string and calls recursive.
144 compile (h:t) = let l = (getLocationFrom h getLinkList)
145 in getCont l ++ " until_" ++ getPrecision ++ (" ++ getGuard h ++ ") || !(
    ++ getInv l ++ ")); \n if (" ++ getGuard h ++ ") then { \n" ++ getAssign
    h ++ (compile t) ++ " } else deadlock:=1;\n"
146
147 -- Function that compiles a regular expression into Lince code
148
149 -- Function that compiles a single transition label
150 compileR :: RE -> String
151 compileR (Symbol a) = let l = (getLocationFrom a getLinkList)
152 in getCont l ++ " until_" ++ getPrecision ++ (" ++ getGuard a ++ ");" ++ (
    getAssign a)
153
154 -- Function that compiles the kleene star
155 compileR (Star r) = "while (true) do {" ++ compileR r ++ " }"
156
157 -- Function that compiles de concatenation of 2 regular expressions
158 compileR (Concat r1 r2) = compileR r1 ++ compileR r2;
159
160
161 -- Function that returns Lince Code with an urgent methodology of doing the
    first transition that is detected (less complex, doesn't need sequence
    of transitions).
162 -- Creates a variable state equal to a location, enters a loop (while true
    ) that detects state and executes the code of that location until a
    transition is enabled
163 compileU :: String
164 compileU = getInit ++ "\nstate:=" ++ (show getInitLocation) ++ "\ndeadlock:=0;
    \nwhile (deadlock==0) do {" ++ compileU' getLocationSize ++ "\n\n}"
165
166 compileU' :: Int -> String
167 compileU' 0 = ""
168 compileU' n = "\n\nif (state == " ++ (show n) ++ " then {" ++ (getCont n) ++
    " until_" ++ getPrecision ++ " (!( " ++ (getInv n) ++ " )" ++ concat [" ||
    (" ++ (getGuard label) ++ " )" | (from,label,to) <- getLinkList, from ==
    n ] ++ " )" ++ "\n      if !( " ++ (getInv n) ++ " ) then deadlock:=1; else
    {" ++ compileU' [ (label,to) | (from, label,to) <- getLinkList, from ==
    n ] ++ "\n} else deadlock := deadlock;" ++ compileU' (n-1)
169
170 compileU'' :: [(Label,Location)] -> String
171 compileU'' [] = "deadlock:=deadlock;"
172 compileU'' [(l,t)] = "\n      if (" ++ (getGuard l) ++ ") then {" ++ (
    getAssign l) ++ "state:=" ++ (show t) ++ ";" ++ " else { deadlock :=
    1;}"
173 compileU'' ((l,t):xs) = "if (" ++ (getGuard l) ++ ") then " ++ (getAssign l)
    ++ " state:=" ++ (show t) ++ ";" ++ "else{" ++ compileU'' xs ++ "}"
174
175 -----
176 -- Regular Expression data structure
177 -----
178 data RE = Epsilon          --
179 | Empty                    --
180 | Symbol Char              -- a single symbol
181 | Concat RE RE             -- Concatenation of two REs
182 | Alt RE RE                -- Alternation (a | b)

```

```

183 | Star RE          -- Kleene star (a*)
184 deriving (Eq)
185
186 instance Show RE where
187 show Epsilon      = "ε"
188 show Empty        = "∅"
189 show (Symbol c)   = [c]
190 show (Concat re1 re2) = show re1 ++ show re2
191 show (Alt re1 re2)   = "(" ++ show re1 ++ " | " ++ show re2 ++ ")"
192 show (Star re)      = "(" ++ show re ++ ")*"
193
194 -- Function that returns Rij when k==0, aka direct connection between two
    location
195 getRE :: Location -> Location -> RE
196 getRE l1 l2 = getRE2 l1 l2 getLinkList
197 where
198 getRE2 l1 l2 [] = if l1 == l2 then Epsilon else Empty
199 getRE2 l1 l2 ((a,b,c):t)
200 | a == l1 && c==l2 = if (hasLink l1 l2 t || l1==l2) then Alt (Symbol b) (
    getRE2 l1 l2 t) else (Symbol b)
201 | otherwise = getRE2 l1 l2 t
202
203 -- Functions that simplify Regular Expression
204 -- Note: Not perfect
205 simplify :: RE -> RE
206 simplify Epsilon = Epsilon
207 simplify Empty = Empty
208 simplify (Symbol c) = Symbol c
209
210 -- Simplifying Concat
211 simplify (Concat re1 re2) =
212 let re1' = simplify re1
213     re2' = simplify re2
214 in case (re1', re2') of
215 (Empty, _) -> Empty          -- Empty.R = Empty
216 (_, Empty) -> Empty         -- Same as above
217 (Epsilon, re) -> re         -- Epsilon.R = R
218 (re, Epsilon) -> re         -- Same as above
219 -- R*R* = R*
220 (Star r1'', Star r2'') | r1'' == r2'' -> Star r1''
221 -- (ε+R)R* = R*
222 (Alt Epsilon r1'', Star r2'') | r1'' == r2'' -> Star r2''
223 (Alt r1'' Epsilon, Star r2'') | r1'' == r2'' -> Star r2''
224 (Star r1'', Alt Epsilon r2'') | r1'' == r2'' -> Star r2''
225 (Star r1'', Alt r2'' Epsilon) | r1'' == r2'' -> Star r2''
226 _ -> Concat re1' re2'      -- Otherwise, keep the concatenation
227
228 -- Simplifying Alt
229 simplify (Alt re1 re2) =
230 let re1' = simplify re1
231     re2' = simplify re2
232 in case (re1', re2') of
233 (Empty, re) -> re          -- Empty alternated with x is x
234 (re, Empty) -> re         -- Same as above
235 (re1'', re2'') | re1'' == re2'' -> re1'' -- R + R = R
236 -- R*a | a = R*a
237 ((Concat (Star r'') a'), a'') | a''==a'' -> (Concat (Star r'') a')
238 ((Concat a' (Star r'')), a'') | a''==a'' -> (Concat a' (Star r''))

```

```

239 _ -> Alt re1' re2'                -- Otherwise, keep the alternation
240
241 -- Simplifying Star
242 simplify (Star re) =
243 let re' = simplify re
244 in case re' of
245   Epsilon -> Epsilon                -- Star of Epsilon is Epsilon
246   Alt Epsilon re'' -> Star re''     -- (e+R)* = R*
247   Alt re'' Epsilon -> Star re''     -- Same as above
248   _       -> Star re'              -- Otherwise, keep the star
249
250 -- Function that shambles order of RE
251 -- Note: Not perfect
252
253 shamble :: RE -> RE
254 -- R1.(R2.R3) = (R1.R2).R3
255 shamble (Concat a (Concat b c)) =
256 let a' = shamble a
257     b' = shamble b
258     c' = shamble c
259 in (Concat (Concat a' b') c')
260
261 shamble (Concat (Concat a b) c) =
262 let a' = shamble a
263     b' = shamble b
264     c' = shamble c
265 in (Concat a' (Concat b' c'))
266
267 shamble (Concat a b) = Concat (shamble a) (shamble b)
268
269 -- (R | R) | R = R | (R | R)
270 shamble (Alt (Alt a b) c) =
271 let a' = shamble a
272     b' = shamble b
273     c' = shamble c
274 in (Alt a' (Alt b' c'))
275
276 shamble (Alt a (Alt b c)) =
277 let a' = shamble a
278     b' = shamble b
279     c' = shamble c
280 in (Alt (Alt a' b') c')
281
282
283 shamble (Alt a b) = (Alt (shamble a) (shamble b))
284 shamble (Star a) = Star (shamble a)
285 shamble a = a
286
287 -- Upgraded simplify by shambling order of RE (Not Perfected)
288 simplifyShamble :: RE -> RE
289 simplifyShamble a = simplify (shamble (simplify a))
290
291 -----
292 -- Kleene Algorithm
293 -----
294
295 -- Kleene algorithm for multiple initial and accepting location
296 allkleene :: Int -> [Location] -> [Location] -> RE

```

```

297 allkleene k l1 l2 = allkleene' [ kleene k x y | x <- l1, y <- l2]
298 where
299 allkleene' [] = Empty
300 allkleene' [a] = a
301 allkleene' (a:t) = Alt a (allkleene' t)
302
303 -- Kleene algorithm for one initial and one accepting location
304 kleene :: Int -> Location -> Location -> RE
305 kleene 0 a b = getRE a b
306 kleene n a b = Alt
307 (
308   Concat (kleene (n-1) a n)
309   (Concat (Star (kleene (n-1) n n))
310     (kleene (n-1) n b)
311   )
312 )
313 (kleene (n-1) a b)
314
315 -- -----
316 -- Auxiliar functions that add combinations of delays to transitions
317 -- -----
318 -- Helper function to insert '.' at every possible position in the string
319 insertAtAllPositions :: Char -> String -> [String]
320 insertAtAllPositions c str = [before ++ [c] ++ after | i <- [1..length str
321   ], let (before, after) = splitAt i str]
322
323 -- Function to generate all possible variations of a string with '.'
324   inserted up to n times
325 insertAllPossibleN :: Int -> String -> [String]
326 insertAllPossibleN 0 str = [str] -- Base case: no more insertions
327 insertAllPossibleN n str = nub ( concatMap (\s -> insertAllPossibleN (n-1)
328   s) (insertAtAllPositions '.' str))
329
330 -- Main function to apply the transformation to a list of strings
331 addAllPossible :: Int -> [String] -> [String]
332 addAllPossible n strs = concatMap (insertAllPossibleN n) strs
333
334 -----
335 -----
336 main = do
337   putStrLn ("Kleene From Init to Final: ")
338   print (simplifyShamble (kleene getLocationSize getInitLocation
339     getFinalLocation))
340   putStrLn ("All possible runs of Automaton up to size 3:")
341   print (findLabelSequences getLinkList 3 getInitLocation)
342   putStrLn ("All possible runs of Automaton up to size 3 and 3 delays:")
343   print (addAllPossible 3 (findLabelSequences getLinkList 3 getInitLocation))
344   putStrLn ("Locations from doing " ++ getSequence ++ ":")
345   print (locationFromSequence getInitLocation getSequence)
346   putStrLn ("\nLince code for deterministic Automata:")
347   print ( compilerR (simplifyShamble (kleene getLocationSize getInitLocation
348     getFinalLocation)))
349   putStrLn ("\nLince code for run " ++ getSequence ++ " :\n" ++ getInit ++ "\n"
350     ++ "deadlock:=0;\n" ++ compile getSequence)
351   putStrLn ("\nLince code for urgent method: \n" ++ compileU)

```

A.2 Urgent Methodology algorithm

Algorithm 1 Old Algorithm for conversion of Automata to Lince - Urgent methodology

```
1: variables  $\leftarrow$  Init
2: state  $\leftarrow$  Init
3: deadlock  $\leftarrow$  false
4: while deadlock == false do
5:   for all  $l \in L$  do
6:     if state =  $l$  then
7:        $f(l)$  until  $\bigvee \{guard(L1, \Sigma, L2) \mid (L1, \Sigma, L2) \in E \wedge (L1 = l)\} \vee \neg inv(l)$ 
8:       if  $\neg inv(l)$  then
9:         deadlock  $\leftarrow$  true
10:      else
11:        for all  $j \in \{(L1, \Sigma, L2) \in E \mid (L1 = l)\}$  do
12:          if  $guard(j)$  then
13:             $assign(j)$ 
14:            state  $\leftarrow$   $L2$ 
15:          end if
16:        end for
17:      end if
18:    end if
19:  end for
20: end while
```

A.3 Examples

A.3.1 Deterministic Semaphore

```
1  x := 0;
2  while true do {
3    x' = 1 until_1 (x == 30); x := 0;
4    x' = 1 until_1 (x == 2); x = 0;
5    x' = 1 until_1 (x == 30); x = 0;
6  }
```

A.3.2 Automaton Example

```
1  x:=0;
2  x'=1 until_0.1 (x==3 || !(x<=5));
3  if (x==3) then {
4    x:=0; x'=1 until_0.1 (x==1 || !(x<=5));
5    if (x==1) then {
6      x:=x+1; x'=1 until_0.1 (x==1 || !(x<=5));
7      if (x==1) then x:=1;
8      else deadlock:=1;
9    }else deadlock:=1;
10 }else deadlock:=1;
```


Chapter B

Lince Examples

B.1 Taxonomy

Projectile Motion

```
1 // Linear System
2 x:=0; y:=0; vx:=5; vy:=5; g:=9.8;
3 x'=vx, y'=vy, vx'=0, vy'=-g for 10;
```

Dubins Aircraft Dynamic

```
1 // Non-Linear System
2 x:=0; y:=0; v:=10; a:=0; w:=pi()/10;
3 x' = v * cos a, y = v * sin (a), a' = w for 10;
```

Distributed Breaking

```
1 // Distributed Hybrid System
2 // Lince does not support:
3 p1:=-10; p2:=10; v1:=1; v2:=-1; a1:=0; a2:=0;
4 {
5 while true do {
6   p1'=v1, v1'=a1 for 0.5;
7   if (p2-p1<10) then a1:=-1; else a1:=1;}
8 } || {
9 while true do {
10  x2'=v2, v2'=a2 for 1;
11  if (p2-p1<15) then a2:=1; else a2:=-1;}
12 }
13 // Equivalent that Lince supports:
14 pa:=-10; pb:=10; va:=1; vb:=-1; aa:=0; ab:=0;
15
16 while true do {
17   pa'=va, va'=aa, pb'=vb, vb'=ab for 0.5;
18   if (pb-pa<10) then aa:=-1; else aa:=1;
19   pa'=va, va'=aa, pb'=vb, vb'=ab for 0.5;
20   if (pb-pa<10) then aa:=-1; else aa:=1;
21   if (pb-pa<15) then ab:=1; else ab:=-1;
22 }
```

2D Cruise Control

```
1 // Deterministic System.
2 x:=0; y:=0;
3 vx:=2; vy:=2;
4 ax:=5; ay:=5;
5 while true do {
6   if vx<=10 then ax:=5; else ax:=-4;
7   if vy<=10 then ay:=7; else ay:=-2;
8   x'=vx, vx'=ax, y'=vy, vy'=ay for 1;
9 }
```

Breaking Or Accelerating

```
1 // Non-Deterministic Switching
2 // Lince does not support:
3 x:=0; v:=2; a:=0;
4 x'=v, v'=a for 30;
5 (a:= -1) + (a:=1);
6 x'=v, v'=a for 30;
```

Thermostat

```
1 // Non-Deterministic in Time
2 // Lince does not support
3 l:=10; h:=5; K:=1;
4 tempMin:=5; tempMax:=20;
5 tempOn:=10; tempOff:=10;
6 while (true) do {
7   l' = -K * l until+ (l>tempMin && l<tempOn);
8   l' = K*(h-l) until+ (l>tempOff && l<tempMax);
9 }
```

Brownian Motion

```
1 // Stochastic continuous movement
2 // Lince does not support
3 x:=0; v:=10; u:=3;
4 x'= v + u * W for 10;
```

Breaking and Accelerating

```
1 // Time triggered change
2 x:=0; v:=2; a:=1;
3 while true do {
4   x'=v, v'= a for 30;
5   a := - a;
6 }
```

Breaking If

```
1 // Time triggered Interrogation
2 x:=0; v:=2; a:=1;
3 while (true) do {
4     x'=v, v'= a for 1;
5     if (v>10) then a:=-1 else a:=a;
6 }
```

Bouncing Ball

```
1 // Event triggered action - Wrong Simulation
2 h:=10; v:=0; a:=-9.8;
3 while (true) do {
4     h'=v, v'= a until (h == 0);
5     v:=-0.5*v;
6 }
7 // Equivalent that Lince supports
8 h:=10; v:=0; a:=-9.8;
9 while (true) do {
10    h'=v, v'= g until (h < 0);
11    h:=0; v:=-0.5*v;
12 }
```

Light Switch

```
1 // Distributed Systems and Controlled Signals
2 // Lince does not support
3 c1:=0; c2:=0; light:=0;
4 {
5     while true do {
6         c1'=1 until ch?
7         light = 1 - light;
8     }
9 } || {
10    while true do {
11        c2'=1 for 10;
12        c2'=1 until ch!
13    }
14 }
```

Controlled Velocity

```
1  // Distributed Systems and Controlled Valued Signals
2  // Lince does not support
3  c1:=0; c2:=0; x:=0; v:=0; a:=0;
4  {
5      while true do {
6          x'=v; v'=a until (a)ch?
7      }
8  } || {
9      while true do {
10         c2'=1 for 1;
11         a:= a+1;
12         c2'=1 until (a)ch!
13     }
14 }
```

B.2 Discussions

Water Tank

```
1  x:=0; y:=1;
2  while (true) do {
3      x' = 1, y' = 1 until_0.5 (y==10); x:=0;
4      x' = 1, y' = 1 until_0.5 (x==2);
5      x' = 1, y' = -2 until_0.5 (y==5); x:=0;
6      x' = 1, y' = 1 until_0.5 (x==2);
7  }
```

Thermostat

```
1  temp:=25; k:=0.2 ; h:=100;tempOn:=20; tempOff:=30;
2  tempMin:=15; tempMax:=35;
3  while (true) do {
4      temp'= -k * temp until_1 (temp<tempOn);
5      temp'= k * (h -temp) until_1 (temp>tempOff);
6  }
```

Gas Burner

```
1 // Compiling string "a.ba"
2 x:=0;y:=0;z:=0;
3 deadlock:=0;
4 x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x<=1));
5 if (x==0.2 || x==0.5 || x==1) then {
6   x'=1,y'=1,z'=1 for 0.1;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x
   ==1) || !(x<=1));
7   if (x==0.2 || x==0.5 || x==1) then {
8     x:=0;x'=1,y'=1 until_0.1 ((x==30) || !(x<=60));
9     if (x==30) then {
10      x:=0;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x
      <=1));
11      if (x==0.2 || x==0.5 || x==1) then {
12        x:=0;x'=1,y'=1 until_0.1 ((x==30) || !(x<=60));
13        if (x==30) then x:=0;else deadlock:=1;
14      } else deadlock:=1;
15    } else deadlock:=1;
16  } else deadlock:=1;
17 } else deadlock:=1;
18 -----
19 // Compiling string "aba.ba..b"
20 x:=0;y:=0;z:=0;
21 deadlock:=0;
22 x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x<=1));
23 if (x==0.2 || x==0.5 || x==1) then {
24   x:=0;x'=1,y'=1 until_0.1 ((x==30) || !(x<=60));
25   if (x==30) then {
26     x:=0;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x<=1));
27     if (x==0.2 || x==0.5 || x==1) then {
28       x'=1,y'=1,z'=1 for 0.1;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 ||
       x==1) || !(x<=1));
29       if (x==0.2 || x==0.5 || x==1) then {
30         x:=0;x'=1,y'=1 until_0.1 ((x==30) || !(x<=60));
31         if (x==30) then {
32           x:=0;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x==0.5 || x==1) || !(x
           <=1));
33           if (x==0.2 || x==0.5 || x==1) then {
34             x'=1,y'=1,z'=1 for 0.1;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x
             ==0.5 || x==1) || !(x<=1));
35             if (x==0.2 || x==0.5 || x==1) then {
36               x'=1,y'=1,z'=1 for 0.1;x'=1,y'=1,z'=1 until_0.1 ((x==0.2 || x
               ==0.5 || x==1) || !(x<=1));
37               if (x==0.2 || x==0.5 || x==1) then {
38                 x:=0;x'=1,y'=1 until_0.1 ((x==30) || !(x<=60));
39                 if (x==30) then x:=0;else deadlock:=1;
40               } else deadlock:=1;
41             } else deadlock:=1;
42           } else deadlock:=1;
43         } else deadlock:=1;
44       } else deadlock:=1;
45     } else deadlock:=1;
46   } else deadlock:=1;
47 } else deadlock:=1;
```

Dubins Path

```
1 // Time Discretization
2 px:=0; py:=0; vx:=0; vy:=0;
3 a:=0; w:=pi()/10; v:=1;
4 while true do {
5     a:=a+w;
6     vx:=v*cos(a);
7     vy:=v*sin(a);
8     px'=vx, py'=vy for 1;
9 }
10
11 // Linear Equivalent
12 px:=0; py:=0; vx:=1; vy:=0;w:=pi()/10;
13 px'=vx, py'=vy, vx'=-w * vy, vy'=w*v for 20;
```

Wall Collision

```
1 x:=0;y:=3;vx:=2;vy:=0;vtemp:=0;
2 x'=vx, y'=vy until (y<((2*x)+0));
3 vtemp:=vx;
4 vx := vx - (2*m*(vx*m - vy))/((m^2)+1);
5 vy := vy + (2*(vtemp*m - vy))/((m^2)+1);
6 x'=vx, y'=vy for 2;
```

Automatic Emergency Breaking

```
1 // Initial Position and velocity of vehicle.
2 p:=0; v:=20;
3 // Initial Position , velocity of obstacle.
4 pl:=50; vl:=0;
5 // Acceleration and Breaking values of vehicle.
6 aT:=-9.8; aA:=6;
7 // Periodic decision time for the system.
8 sampling_time:=0.1;
9 while v>0 do{
10     if ((p + v*sampling_time + aA/2*sampling_time^2 < pl+vl* sampling_time)
11         && (((v+aA*sampling_time -vl)^2 - 4*(p+v*sampling_time+aA/2*(
12             sampling_time^2)-(pl +vl*sampling_time))*(aT/2)) < 0))
13         then p'=v,v'=aA for sampling_time;
14     else {
15         p'=v,v'=aT until_0.01 (v<=0);
16     }
```

Adaptive Cruise Control

```
1  p:= 0; v:= 20;
2  pl:= 30; vl := 10;
3  aT:= -8; aL:= 0;
4  safety_distance := 10;
5  sampling_time:= 0.1;
6  while ( true ) do {
7      if ((p + v*sampling_time < ( pl - safety_distance ) + vl *
          sampling_time + aL/2*sampling_time^2) && ( ( ( v - vl + (-aL)*
          sampling_time )^2 - 4*(p - ( pl - safety_distance ) + (v - vl )*
          sampling_time + ( -aL) /2*sampling_time^2) *(aT-aL)/2 ) <0))
8          then p'=v , v'=0 , pl'=vl , vl' =aL for sampling_time;
9          else p'=v , v'=aT, pl'=vl , vl'=aL for sampling_time;
10     }
```

Missile and Target

```
1  px:=30; py:=0; vx:=0; vy:=2;
2  pxf:= 0; pyf:=0;
3
4  va:=(1/10)*pi();
5  time:=0.5; i_angle:=(pi()/10);
6
7  aw:=0; vxf:= 0; vyf:=0;
8  angle:=0; dot_product:=0; cross_product:=0; rotation:=0;
9
10 while true do {
11     vxf:=pxf-px; vyf:=pyf-py;
12     dot_product := (vx * vxf) + (vy * vyf);
13     cross_product := (vx * vyf) - (vy * vxf);
14     angle := arccos(dot_product / ((sqrt (vx^2 + vy^2)) * (sqrt (vxf^2 +
        vyf^2))));
15
16     if cross_product < 0 then rotation := -1; else rotation := 1;
17
18     if (angle<i_angle)
19         then aw:=0;
20         else aw := rotation * va;
21
22     px'=vx, py'=vy, vx'= - aw * vy, vy'= aw * vx for time;
23 }
```